


# 2026年5月北美芯片设计/验证题目汇总

根据同学回忆整理 · 98道真题含答案与解析

Apple · Qualcomm · Google · Nvidia · ByteDance · Broadcom

Hudson River Trading · Etched · Amazon · Curris Logic · Black Sesame

# 目录


## Digital Design

Q1. Setup/Hold Time: Concepts and Violation Resolution
Q2. CDC: Synchronizer Placement
Q3. RTL Bug Fix: Synchronous FIFO
Q4. RTL Bug Fix: Power Manager (CDC + Blocking/Non-blocking)
Q5. Safety-Aware RTL Design: Techniques and Applications (designer interview question)
Q6. Security-Aware RTL Design: Techniques and Applications (designer interview question)
Q7. Low Power RTL Design: Common Techniques and Applications (designer interview question)
Q8. Lockstep RISC-V: Implementation and Power Impact (designer interview question)
Q9. Low Power Techniques Overview (designer interview question)
Q10. Multi-Voltage Domain and DVFS Design Details (designer interview question)
Q11. LRU and PLRU Cache Replacement: 16-Way Implementation (designer interview question)
Q12. Parity Check: 16-bit Data Implementation and Gate Layers (designer interview question)
Q13. Digital Design: Build an Inverter Using Gates X and Y
Q14. Verilog: Implement Fibonacci Sequence
Q15. Digital Design: NOR + Two D-FFs Circuit Waveform
Q16. Debug: Multiple CPUs Reading Inconsistent Data
Q17. Computer Architecture: MOESI Protocol -- Two Processors Writing Same Cache Line
Q18. Computer Architecture: Cache Types and Write-Back Policy
Q19. Analog/Digital: CMOS Inverter with PMOS and NMOS
Q20. Analog Circuit: RC Low-Pass Filter
Q21. Digital Design: 2-to-1 MUX Using Only NAND Gates
Q22. Verilog: Sliding Window -- Count 1s in Last 100 Bits > 50
Q23. Digital Design: 2s Complement Bit Width for A-B
Q24. Digital Design: Divide-by-3 Clock with 50% Duty Cycle
Q25. Digital Design: FIFO Full and Empty Detection
Q26. Digital Design: Synchronous vs Asynchronous Reset -- When to Use Each
Q27. Digital Design: Setup/Hold Violation Analysis and Checker
Q28. Digital Design: FSM for a Digital Lock with RTL and Assertions
Q29. Computer Architecture: ROB and CFC -- Out-of-Order Execution with In-Order Commit
Q30. Computer Architecture: Two Kinds of RAT and What They Solve
Q31. Computer Architecture: Purpose of CFC in Tomasulo Machine
Q32. Computer Architecture: MOESI vs MESI -- The O State and Performance
Q33. Computer Architecture: MOESI State Transition Table for 4-Core CMP

## SystemVerilog — Constraints & Randomization

Q34. Constraint: Address Alignment with CMD Signal
Q35. Constraint: Dynamic Array Sum and 2D Array Row Constraint
Q36. Constraint: Various Array Constraints
Q37. SV Constraint: 1D/2D Array Size, Value, and Ordering
Q38. SV Constraint: Dynamic Array with Sum Equal to 20
Q39. SV Constraint: Non-Overlapping Rectangle Regions in 2D Array
Q40. SV Constraint: Exactly One Consecutive Run of 1s in 16-bit Value
Q41. SV Constraint: rand bit Array with Odd Number of 1s
Q42. SV Constraint: 300-Element Array, Values 0-5, Each >= 40 Times, No Consecutive 0s
Q43. SV Constraint: 300-Element Array, 0 May Repeat, Non-Zero Values Must Not Be Consecutive

Q44. SV Constraint: Memory Access with Excluded Range
Q45. SV Constraint: Repeatedly Randomize Non-Overlapping Memory Regions Until Full
Q46. SV Constraint: Symmetric Array Using Helper Function
Q47. SV Constraint: Constraint Inheritance and constraint_mode()

### SystemVerilog — Assertions (SVA)

Q48. SVA: Analyze a Complex Property
Q49. SVA: Glitch Detection -- Signal HIGH for Less Than 2.5ns
Q50. SVA: valid Must Stay High Until ready
Q51. SVA: Sequence Assertion -- cat Followed by 3 or 4 dogs
Q52. SVA: Clock Period Verification -- Within 5% of Expected Period
Q53. SVA: Clock Period Check When expected_clock_period is Async
Q54. SVA: APB Protocol Assertion -- PSEL to PENABLE to PREADY
Q55. SVA: B Toggles 2 Cycles After A Toggles

### SystemVerilog — OOP

Q56. OOP: Inheritance, Polymorphism, and Singleton Pattern
Q57. OOP: UVM Sequence Bug Analysis
Q58. OOP: Singleton Class and Abstract Class
Q59. SV OOP: Class Handle -- Pass by Value vs Pass by ref

### SystemVerilog — Coding

Q60. SV Coding: Edge Detector (All Edges and Rising Edge)
Q61. SV Coding: Monitor and Testbench for Interrupt Protocol
Q62. SV Coding: Covergroup for 8-bit Signal
Q63. SV Coding: AHB Interface with Clocking Blocks
Q64. SV Coding: fork/join_none Loop Variable Capture
Q65. SV Coding: fork/join Variations and wait fork
Q66. SV Coding: fork/join -- Call Function After Any Two of Three Tasks Complete
Q67. Verilog: Falling Edge Detector and Its Synthesized Circuit
Q68. UVM: Architecture Overview

### UVM

Q69. UVM: run_phase Ordering and drop_objection Behavior
Q70. UVM: All Phases -- Which are Tasks vs Functions
Q71. UVM: Inter-process Synchronization Methods
Q72. UVM: Code Coverage Types
Q73. UVM: DMA Scoreboard Implementation
Q74. UVM: Design a DMA Engine uvm_sequence_item
Q75. SV: Associative Array vs Regular Array in UVM Scoreboard
Q76. UVM: Fix Scoreboard to Support Byte-Enable Writes
Q77. SV/C++: Data Structure for Sparse Memory in Verification
Q78. Verification: Handling Concurrent Writes to Sparse Memory Model
Q79. UVM: Scoreboard Design for 10-Input to 20-Output Port Mapping
Q80. UVM: Phase Mechanism -- What It Is and How It Works
Q81. UVM: Objection Mechanism -- What It Is and How It Works
Q82. UVM: Scoreboard with Timeout and Duplicate Detection (A/B Pairing)
Q83. Python: Parse comma-separated names into a list

### Algorithms & Scripting

Q84. Python: Extract error lines using regex
Q85. C: Check if the N-th bit is 1
Q86. Algorithm: Find the Element That Appears Once (XOR Trick)

Q87. C++ Algorithm: Count Pairs with Equal Digit Sum
Q88. C++ Design: Container with ADD, EXISTS, GET_COUNT
Q89. C++ Algorithm: Palindrome Vowel Removal
Q90. C++ Algorithm: Array Pair Product Query with Point Update
Q91. C++: Reverse a Linked List
Q92. C: memcpy Safety Issues and Best Practices
Q93. Test Plan: SoC with Encryption/Decryption Module

## Verification — Test Plan

Q94. Algorithm: 100 Switches Toggle Problem -- How Many Are ON
Q95. Verification: SoC-Level Verification with CPU, DMA, and Peripherals
Q96. Verification: Test Plan for Address-Based Routing DUT (A/B Client)

## Behavior Questions

Q97. Behavior Question: Describe Your Most Proud Project
Q98. Behavior Question: Describe an Interesting Bug You Encountered

## Digital Design

### Q1. Setup/Hold Time: Concepts and Violation Resolution

QUESTION

(a) Explain the concepts of setup time and hold time.

(b) What causes setup and hold violations?

(c) How do you resolve each type of violation before and after tapeout?

(d) Given a specific circuit, calculate the maximum and minimum operating frequency.

ANSWER

```
// (a) Concepts:
 // Setup time (t_su): data must be STABLE before the clock edge
 //    by at least t_su. Violating setup -> data sampled incorrectly.
 // Hold time (t_h): data must REMAIN STABLE after the clock edge
 //    for at least t_h. Violating hold -> data corrupted immediately
 //    after the edge.

 // (b) Causes:
 // Setup violation: data path too SLOW (long combinational delay).
 //    Tclk < t_cq + t_logic + t_su
 // Hold violation: data path too FAST (short combinational delay).
 //    t_cq + t_logic < t_h + t_skew

 // (c) Resolution:
 // Setup violation - before tapeout:
 //    - Reduce logic depth (pipeline the path)
 //    - Use faster cells (LVT)
 //    - Increase Vdd
 //    - Reduce clock frequency
 // Setup violation - after tapeout:
 //    - Reduce operating frequency (most effective)
 //    - Increase supply voltage
 //    - Lower operating temperature (Fast PVT corner)

 // Hold violation - before tapeout:
 //    - Insert delay buffers on short paths
 //    - Use slower cells (HVT) on the data path
 //    - Fix clock skew via CTS
 // Hold violation - after tapeout:
 //    - Increase supply voltage (speeds up path, paradoxically helps)
 //    - Adjust clock phase
 //    - NOTE: hold violations CANNOT be fixed by reducing frequency

 // (d) Max/Min frequency calculation:
 // Max frequency (setup constraint):

 //    Tclk_min = t_cq + t_logic_max + t_su + t_skew
 //    f_max = 1 / Tclk_min
 //
 // Min frequency is not constrained by hold in typical cases.
 // Hold constraint (independent of frequency):
 //    t_cq + t_logic_min > t_h + t_skew
```

解释

Setup 和 Hold 是数字设计最基础也最重要的时序概念，面试必考。

关键区别：Setup violation 与频率相关，降频就能修复；

Hold violation 与频率无关，无论多低的频率都无法靠降频解决，

这是 tapeout 后最危险的 violation。

计算频率时注意：setup 约束决定最高频率，hold 约束与频率无关，

hold 的计算要用最小路径延迟（best case），而 setup 用最大延迟（worst case）。

## Q2. CDC: Synchronizer Placement

QUESTION

A signal crosses from clock domain A to clock domain B.

The synchronizer (two-FF chain) should be clocked by which clock:

clkA or clkB? Explain why.

ANSWER

```
// Answer: the synchronizer must be clocked by clkB (the DESTINATION clock).

 // Why clkB:
 // The purpose of the synchronizer is to resolve metastability
 // BEFORE the signal is used in domain B.
 // The two flip-flops need to settle (resolve metastability) within
 // one clkB period before the signal is sampled by domain B logic.
 //
 // If clocked by clkA: the output would still be asynchronous to clkB,
 // and domain B logic would sample an unsynchronized signal -> metastability
 // is NOT resolved, defeating the entire purpose.
 //
 //   clkA domain             clkB domain
 //   +---------+             +------+      +------+
 //   | FF_src   |--------->| FF1      |--->| FF2    |---> to clkB logic
 //   +---------+             +------+      +------+
 //    ^clkA                   ^clkB         ^clkB
 //
 // FF1 may go metastable, but has one full clkB cycle to resolve.
 // FF2 samples the (hopefully resolved) output of FF1.
 // Both FF1 and FF2 are clocked by clkB.
```

解释

这道题是 CDC 最经典的基础题，答案是目标时钟域（clkB）。

原理：同步器的作用是让亚稳态在被目标域逻辑采样之前有足够时间分辨。

如果用源时钟 clkA 驱动同步器，输出信号对 clkB 来说仍然是异步的，

毫无意义。同步器的两个 FF 都必须用 clkB 驱动，

这样 FF1 的亚稳态有一个完整的 clkB 周期来分辨，FF2 再采一拍确保稳定。

## Q3. RTL Bug Fix: Synchronous FIFO

QUESTION

The following synchronous FIFO has multiple bugs. Find and fix them.

```
module xyz #(parameter DEPTH = 16) (
   input    logic clk, rst_n, wr_en, rd_en,
   input    logic wr_data,
   output logic rd_data,
   output logic full, empty
);
   logic mem [0:DEPTH-1];
   logic [$clog2(DEPTH)-1:0] rd_ptr, wr_ptr;
   logic [$clog2(DEPTH):0]        count;


   always_ff @(posedge clk) begin
     if (!rst_n) wr_ptr <= 0;
     else if (wr_en) begin
         mem[wr_ptr] <= wr_data;
         wr_ptr <= (wr_ptr + 1);
     end
   end


   always_ff @(posedge clk) begin
     if (!rst_n) rd_ptr <= 0;
     else if (rd_en) begin
         rd_data <= mem[rd_ptr];
         rd_ptr <= (rd_ptr + 1);
     end
   end


   always_ff @(posedge clk) begin
     if (!rst_n) begin
         count <= 0;
         empty <= 0;     // BUG
         full   <= 0;
     end else begin
         if (wr_en && !full && (!rd_en || empty))
           count <= count + 1;
         else if (rd_en && !empty && (!wr_en || full))


           count <= count - 1;
         empty <= (count == 0);      // BUG
         full   <= (count == DEPTH);
     end
   end
endmodule
```

ANSWER

```
// Bug 1: Data width mismatch
// wr_data and rd_data are declared as 'logic' (1-bit) but should
// be parameterized. Assuming 8-bit data for fix:
// input    logic [7:0] wr_data,
// output logic [7:0] rd_data,
// logic [7:0] mem [0:DEPTH-1];

// Bug 2: empty not initialized to 1 on reset
// On reset, FIFO is empty, so empty should be 1, not 0.
// if (!rst_n) begin
//   count <= 0;
//   empty <= 1;    // FIXED: FIFO is empty after reset
//   full   <= 0;
```

```
// end

// Bug 3: empty/full flags use stale count (off-by-one cycle)
// count is updated with <=, so empty/full based on count
// in the same always_ff block see the OLD count value.
// Fix: use combinational (next-cycle) count or use direct comparison:
//
// Option A: combinational flags (preferred)
// assign empty = (count == 0);
// assign full   = (count == DEPTH);
//
// Option B: use count+1 / count-1 inside the sequential block
// This avoids the one-cycle lag.

// Bug 4: pointer wrap-around missing
// wr_ptr <= (wr_ptr + 1) should wrap around at DEPTH-1:
// wr_ptr <= (wr_ptr == DEPTH-1) ? 0 : wr_ptr + 1;
// rd_ptr <= (rd_ptr == DEPTH-1) ? 0 : rd_ptr + 1;

// Bug 5: Count logic is incorrect for simultaneous read+write
// when the FIFO is full or empty.
//

// WRONG logic (our original answer):
//    if (wr_en && !rd_en && !full)    count <= count + 1;
//    else if (rd_en && !wr_en && !empty) count <= count - 1;
//
// This has TWO silent bugs:
//
// Case A: wr_en=1, rd_en=1, empty=1 (write+read when FIFO is empty)
//    -> Write is valid (!full), read is blocked (empty).
//    -> Expected: count +1
//    -> Wrong code: first branch FALSE (rd_en=1 breaks !rd_en),
//                   second branch FALSE -> count unchanged! BUG.
//
// Case B: wr_en=1, rd_en=1, full=1 (write+read when FIFO is full)
//    -> Write is blocked (full), read is valid (!empty).
//    -> Expected: count -1
//    -> Wrong code: first branch FALSE (!full=0),
//                   second branch FALSE (wr_en=1 breaks !wr_en) -> count unchanged! BUG.
//
// CORRECT FIX: decouple wr/rd validity, then combine:
//    wr_ok = wr_en && !full        (actual write will happen)
//    rd_ok = rd_en && !empty       (actual read will happen)
//    if      (wr_ok && !rd_ok) count <= count + 1;    // write only
//    else if (rd_ok && !wr_ok) count <= count - 1;    // read only
//    // both valid simultaneously -> count unchanged (correct)
//    // neither valid               -> count unchanged (correct)

// FIXED MODULE (corrected count logic):
module xyz #(
  parameter DEPTH   = 16,
  parameter DATA_W = 8
) (
  input    logic               clk, rst_n, wr_en, rd_en,
  input    logic [DATA_W-1:0] wr_data,
  output logic [DATA_W-1:0] rd_data,
  output logic                full, empty

);
  logic [DATA_W-1:0]           mem [0:DEPTH-1];
  logic [$clog2(DEPTH)-1:0] rd_ptr, wr_ptr;
  logic [$clog2(DEPTH):0]      count;
```

```
  // Actual transaction signals (decoupled from raw enables)
  wire wr_ok = wr_en && !full;
  wire rd_ok = rd_en && !empty;

  always_ff @(posedge clk) begin
     if (!rst_n) wr_ptr <= 0;
     else if (wr_ok) begin
        mem[wr_ptr] <= wr_data;
        wr_ptr <= (wr_ptr == DEPTH-1) ? 0 : wr_ptr + 1;
     end
  end

  always_ff @(posedge clk) begin
     if (!rst_n) rd_ptr <= 0;
     else if (rd_ok) begin
        rd_data <= mem[rd_ptr];
        rd_ptr <= (rd_ptr == DEPTH-1) ? 0 : rd_ptr + 1;
     end
  end

  always_ff @(posedge clk) begin
     if (!rst_n) count <= 0;
     else begin
        if     (wr_ok && !rd_ok) count <= count + 1;
        else if (rd_ok && !wr_ok) count <= count - 1;
        // simultaneous wr_ok && rd_ok: count unchanged (correct)
     end
  end

  assign empty = (count == 0);

  assign full     = (count == DEPTH);
endmodule

// --- EXTENSION: Support simultaneous wr+rd when full or empty ---
//
// Scenario 1: FIFO FULL, wr+rd simultaneously
//    Standard behavior: write blocked (full), only read proceeds -> count-1
//    Extended behavior: allow write AND read simultaneously even when full.
//    -> Read frees one slot; write fills it in the same cycle.
//    -> Count stays the same; old LRU data out, new data in.
//    -> RTL: relax wr guard: wr_ok_ext = wr_en && (!full || rd_en)
//           (if read is also happening, a slot is being freed simultaneously)
//
// Scenario 2: FIFO EMPTY, wr+rd simultaneously -> BYPASS mode
//    Standard behavior: read blocked (empty), only write proceeds -> count+1
//    Extended behavior: bypass - data_in goes directly to data_out.
//    -> Nothing written to FIFO; count stays 0.
//    -> RTL: bypass mux on rd_data output.
//
// FIFO with bypass (empty passthrough):
module xyz_bypass #(
  parameter DEPTH     = 16,
  parameter DATA_W = 8
) (
  input   logic                clk, rst_n, wr_en, rd_en,
  input   logic [DATA_W-1:0] wr_data,
  output logic [DATA_W-1:0] rd_data,
  output logic                 full, empty
);
  logic [DATA_W-1:0]           mem [0:DEPTH-1];
  logic [$clog2(DEPTH)-1:0] rd_ptr, wr_ptr;
```

```
  logic [$clog2(DEPTH):0]      count;

  // Bypass condition: read requested AND fifo is empty AND write happening
  // -> forward wr_data directly to rd_data, skip the FIFO

     wire bypass = rd_en && empty && wr_en;

     // Only write to FIFO if NOT in bypass mode (bypass avoids incrementing count)
     wire wr_ok    = wr_en && !full && !bypass;
     wire rd_ok    = rd_en && !empty;

     always_ff @(posedge clk) begin
      if (!rst_n) wr_ptr <= 0;
      else if (wr_ok) begin
           mem[wr_ptr] <= wr_data;
           wr_ptr <= (wr_ptr == DEPTH-1) ? 0 : wr_ptr + 1;
      end
     end

     always_ff @(posedge clk) begin
      if (!rst_n) rd_ptr <= 0;
      else if (rd_ok) begin
           rd_ptr <= (rd_ptr == DEPTH-1) ? 0 : rd_ptr + 1;
      end
     end

     always_ff @(posedge clk) begin
      if (!rst_n) count <= 0;
      else begin
           if      (wr_ok && !rd_ok) count <= count + 1;
           else if (rd_ok && !wr_ok) count <= count - 1;
      end
     end

     assign empty = (count == 0);
     assign full    = (count == DEPTH);

     // Bypass mux: when bypass, forward wr_data; else read from FIFO mem
     assign rd_data = bypass ? wr_data : mem[rd_ptr];

  endmodule
  // Note: bypass rd_data is combinational (same-cycle response).
  // If registered rd_data is required, add one FF on the bypass path.
```

解释

这道题考查同步 FIFO 的设计细节，有5个 bug，其中第5个最容易写错：

1. 数据位宽：wr_data/rd_data 声明为 1-bit，应该参数化为 DATA_W。

2. Reset 初始化：empty 应该复位为 1（FIFO 复位后是空的），不是 0。

3. 标志位时序：在 always_ff 里基于 count 更新 empty/full，

由于 <= 是非阻塞赋值，empty/full 看到的是上一拍的 count，

最佳做法是用 assign 做组合逻辑。

4. 指针回卷：指针递增后需要在 DEPTH-1 时回绕到 0。

5. Count 更新逻辑有隐藏 bug（最易错）：

原始写法 if (wr_en && !rd_en && !full) 在两个边界情况下会出错：

- wr+rd 且 FIFO empty：写有效，读无效，count 应该 +1，

但 !rd_en 条件为 false，导致 count 不变，bug！

- wr+rd 且 FIFO full：读有效，写无效，count 应该 -1，

但 !wr_en 条件为 false，导致 count 不变，bug！

正确做法：先分别计算 wr_ok = wr_en && !full，rd_ok = rd_en && !empty，

再用 wr_ok/rd_ok 来判断 count 的变化方向，完全解耦两个方向的逻辑。

Bypass 扩展设计的核心思想：

当 FIFO 为空且同时有读写请求时，数据可以直接从 wr_data 旁路到 rd_data，

不经过 FIFO 存储，count 保持为 0，延迟最低。

RTL 上只需要一个 bypass mux：

rd_data = bypass ? wr_data : mem[rd_ptr]

bypass 条件 = rd_en && empty && wr_en（同时读空且有写请求）。

注意：bypass 输出是组合逻辑，如果需要寄存器输出，需要额外加一级 FF。

## Q4. RTL Bug Fix: Power Manager (CDC + Blocking/Non-blocking)

QUESTION

Find and fix all bugs in the following power manager code.

Focus on clock domain issues and blocking/non-blocking assignments.

```
module power_manager(dclk, pclk, rst, eot_register,
                         wakeup, status_register);
   input dclk, pclk, rst;
   input [31:0] eot_register;
   output reg wakeup;
   output reg [31:0] status_register;
   reg [31:0] pm_counter;


   initial begin
     pm_counter = 32'd0;
     status_register = 'd0;
   end


   always @(posedge pclk) begin
     if (rst == 1'b1) begin
         pm_counter = pm_counter + 1;      // BUG
         if (pm_counter == 32'd10)
           wakeup   = 1'b1;                // BUG
         else
           wakeup = 1'b0;                  // BUG
     end else begin
         pm_counter = 'd0;                 // BUG
     end
   end


   always @(posedge pclk) begin
     status_register = proc_result(eot_register, check_value); // BUG
   end


   // dut module: check block clocked by dclk reads status_register
```

```
   // which is driven by pclk -> CDC issue
endmodule
```

ANSWER

```
// Bug 1: Blocking assignments (=) used in sequential always blocks
 // All assignments in always @(posedge clk) should use <= (non-blocking)
 // to avoid race conditions and match synthesized flip-flop behavior.

 // Bug 2: rst polarity / logic inversion
 // Counter increments when rst==1 (active-high reset asserted).
 // This is backwards: counter should run when NOT in reset.
 // Fix: if (!rst) -> run counter; else -> reset counter.

 // Bug 3: initial blocks are not synthesizable for FFs
 // Use reset logic inside always_ff instead of initial blocks.

 // Bug 4: CDC - status_register driven by pclk, read by dclk
 // eot_register (from dclk domain) crosses into pclk domain without sync.
 // status_register (pclk domain) is read by the dclk always block -> CDC.
 // Fix: add 2-FF synchronizers at both crossing points.

 // Bug 5: check_value is undeclared / undefined.

 // Bug 6: proc_result() called combinationally inside sequential block
 // with blocking assignment - should use non-blocking.

 // FIXED:
 module power_manager (
     input    logic         dclk, pclk, rst_n,
     input    logic [31:0] eot_register,
     output logic           wakeup,
     output logic [31:0] status_register
 );
     logic [31:0] pm_counter;

     // Synchronize eot_register from dclk to pclk domain
     logic [31:0] eot_sync1, eot_sync2;
     always_ff @(posedge pclk or negedge rst_n) begin
      if (!rst_n) begin eot_sync1 <= 0; eot_sync2 <= 0; end

      else            begin eot_sync1 <= eot_register; eot_sync2 <= eot_sync1; end
     end

     // Counter and wakeup: run when NOT in reset
     always_ff @(posedge pclk or negedge rst_n) begin
      if (!rst_n) begin
           pm_counter <= 0;
           wakeup      <= 0;
      end else begin
           pm_counter <= pm_counter + 1;
           wakeup      <= (pm_counter == 32'd10);
      end
     end

     // status_register updated with non-blocking assignment
     always_ff @(posedge pclk or negedge rst_n) begin
      if (!rst_n) status_register <= 0;
      else            status_register <= proc_result(eot_sync2);
     end
 endmodule
```

解释

这道题综合考查了多个 RTL 设计原则：

1. 非阻塞赋值（<=）：always_ff 中必须用 <=，否则仿真有竞争，综合行为不确定。

2. Reset 极性：if (rst==1) 里面做计数是逻辑错误，复位时应该清零而不是计数。

3. Initial block：不可综合，复位逻辑必须在 always_ff 的 reset 分支里。

4. CDC：eot_register 来自 dclk，在 pclk 域使用前必须过同步器；

status_register 在 pclk 域驱动，在 dclk 域读取也需要同步。

面试时能说出所有 CDC 路径是加分项。

## Q5. Safety-Aware RTL Design: Techniques and Applications (designer interview question)

QUESTION

In safety-critical SoC design (e.g. automotive ASIL-D, industrial SIL-3), what RTL design techniques are commonly used to detect, prevent, or tolerate hardware faults?

For each technique, describe: what it protects against, how it works at the RTL level, and a concrete application scenario.

ANSWER

```
// 1. Lockstep (Dual-Core Redundancy)
//   Protects: any fault in processor datapath (ALU, FSM, pipeline)
//   How: two identical cores execute same instruction stream;
//        compare outputs every cycle (or at writeback stage).
//        Mismatch -> assert safety error -> halt / safe state.
//   RTL detail: comparator on register-write address+data, PC,
//        memory store address+data. Shadow core delayed 1-2 cycles
//        (staggered) to avoid common-cause failures from glitches.
//   Application: automotive MCU (Cortex-R52), airbag ECU,
//        ADAS sensor fusion processor.
//
// 2. ECC (Error Correcting Code) / Parity
//   Protects: bit-flip faults in SRAM (cache, register file, DRAM)
//   How: SECDED adds check bits alongside data.
//        On read: syndrome computed; 0 = no error;
//        non-zero single-bit = correct; double-bit = report UE.
//   RTL detail: ECC encode logic on write path; decode + correct
//        logic on read path. Adds 1-2 cycles latency.
//   Application: CPU L1/L2 cache, DDR ECC DIMM, safety register file.
//
// 3. Watchdog Timer (WDT)
//   Protects: software hang, infinite loop, loss of control flow
//   How: hardware counter counting down to zero; software must
//        periodically write a "kick" value to reset the counter.
//        If SW fails to kick before timeout -> WDT fires NMI / reset.
//   RTL detail: always-on (AON) counter, separate from main clock
//        domain. Kick register write clears counter.
//        Window WDT: kick only valid in a specific time window
//        (too early or too late both trigger fault -> detect runaway).
//   Application: automotive ECU, industrial PLC, medical devices.
//
// 4. CRC / Checksum on Data Paths
//   Protects: data corruption in buses, DMA transfers, NVM storage
```

```
//   How: CRC computed on transmit side; recomputed on receive side;
//        mismatch -> data error interrupt / retry / safe state.

  //   RTL detail: parallel CRC logic (e.g. CRC-32) implemented as
  //        XOR tree. Can be pipelined for high-throughput buses.
  //   Application: CAN bus frames, PCIe TLP, flash read verify,
  //        SPI configuration data integrity.
  //
  // 5. Memory Protection Unit (MPU) / Address Checking
  //   Protects: out-of-bounds memory access, stack overflow,
  //        unauthorized peripheral access
  //   How: hardware checks every memory transaction address against
  //        configured region table; violation -> fault exception.
  //   RTL detail: comparator array on AHB/AXI address bus;
  //        region enable + base/limit registers in always-on domain.
  //   Application: RTOS task isolation, secure boot region protection,
  //        DMA channel containment.
  //
  // 6. Redundant State Machines (FSM Encoding)
  //   Protects: FSM entering illegal/undefined states due to SEU
  //   How: use one-hot or gray encoding with explicit illegal-state
  //        detection; add default case in RTL that forces FSM to
  //        safe/reset state on any undefined encoding.
  //   RTL detail:
  //        always_comb begin
  //          unique case (state)     // unique warns on missing states
  //              ...
  //              default: next_state = SAFE_STATE; // explicit safe default
  //          endcase
  //        end
  //   Application: safety-critical control FSMs (power sequencer,
  //        protocol controller, arbitration logic).
  //
  // 7. Hamming / TMR (Triple Modular Redundancy) for Critical Registers
  //   Protects: SEU in critical configuration / control registers
  //   How: TMR stores 3 copies of a register; majority voter outputs
  //        the value agreed by at least 2 out of 3.
  //        Any single upset is masked without interrupting operation.

  //   RTL detail: 3x FF arrays + combinational majority voter.
  //        Area cost: 3x register area + voter logic.
  //   Application: spacecraft / satellite processors, nuclear plant
  //        controllers, safety configuration registers.
```

解释

Safety RTL 设计的核心关注点是“系统在发生故障时如何行为”而不是“如何避免故障”。

六种主要技术的逻辑：

Lockstep = 防处理器计算错误（最全面）

ECC/Parity = 防内存位翻（最常用）

WDT = 防软件卡死/失控（必备）

CRC = 防总线/存储数据腿误（通信必备）

MPU = 防越界访问（软件隔离必备）

TMR = 防关键寄存器位翻（宇航/核电站级别）

## Q6. Security-Aware RTL Design: Techniques and Applications (designer interview question)

QUESTION

In security-sensitive SoC design, what RTL design techniques are used to protect against hardware attacks and unauthorized access? For each technique, describe: what threat it mitigates, how it works at the RTL level, and a concrete application scenario.

ANSWER

```
// 1. Secure Boot / Boot ROM Integrity
 //      Threat: attacker replaces firmware with malicious code
 //      How: on-chip ROM stores immutable first-stage bootloader.
 //           Hash (SHA-256) of next-stage image verified against
 //           public key stored in eFuse (one-time programmable) before
 //           execution is transferred.
 //      RTL detail: boot ROM mapped to protected address range (MPU lock);
 //           SHA engine and RSA/ECDSA accelerator in always-on domain;
 //           boot flow hardwired in RTL, not configurable by SW.
 //      Application: smartphone SoC secure boot, HSM (Hardware Security Module).
 //
 // 2. Physical Unclonable Function (PUF)
 //      Threat: key extraction from NVM, chip cloning
 //      How: exploits manufacturing variation in SRAM or ring oscillators.
 //           Each chip produces a unique, reproducible response to a
 //           challenge without storing the key in NVM (no key at rest).
 //      RTL detail: SRAM PUF reads power-up state of uninitialized SRAM;
 //           error-correction (fuzzy extractor) regenerates stable key
 //           from noisy PUF response. Key never stored in flash.
 //      Application: device authentication, session key derivation,
 //           anti-counterfeiting.
 //
 // 3. Side-Channel Attack Countermeasures
 //      Threat: DPA (Differential Power Analysis) extracts crypto keys
 //           by correlating power consumption with data being processed
 //      How: masking (XOR data with random mask before processing),
 //           shuffling (randomize operation order), dummy operations
 //           (insert fake ops to decorrelate power from data).
 //      RTL detail: masked AES S-box: compute S(x XOR m) XOR f(m)
 //           instead of S(x); mask m is fresh random per operation.
 //           Requires true random number generator (TRNG) on-chip.
 //      Application: payment card SoC, smart card, SIM card AES engine.
 //
 // 4. Voltage and Clock Tamper Detection
 //      Threat: fault injection via voltage glitching or clock glitching

 //        to skip instructions or induce wrong computation
 //   How: analog voltage monitors compare Vdd against thresholds.
 //        Digital clock monitors check for frequency anomalies
 //        (too fast or too slow).
 //        On detection: immediately zeroize key registers + lockdown.
 //   RTL detail: glitch detector output wired to NMI and key-zeroize
 //        logic in always-on domain. Zeroize = synchronous clear of
 //        all key/nonce registers within 1 clock cycle.
 //   Application: secure element (SE), hardware wallet, TPM chip.
 //
 // 5. Memory Encryption and Integrity
 //   Threat: DRAM snooping, cold-boot attack, bus probing
 //   How: inline AES encryption engine between CPU and DRAM controller.
 //        Message Authentication Code (MAC/HMAC) detects tampering.
 //        Nonce/counter per cache line prevents replay attacks.
 //   RTL detail: AES-XTS or AES-CTR in the memory controller;
 //        MAC stored in extra DRAM bits or interleaved with data.
```

```
//        Critical path: encryption latency adds to DRAM access time.
//   Application: Apple M-series Secure Enclave, Intel TME/MKTME,
//        AMD SME, smartphone memory encryption.
//
// 6. Secure Debug / JTAG Access Control
//   Threat: attacker uses JTAG to read memory, extract keys,
//        or inject arbitrary instructions
//   How: JTAG access gated by authentication (challenge-response
//        using on-chip key). Debug ports disabled in production fuse.
//        Different debug levels: fully open (engineering), partial
//        (RMA debug), fully locked (production).
//   RTL detail: eFuse bits control JTAG enable; authentication FSM
//        must verify response before enabling scan chain access.
//   Application: all production SoCs (Qualcomm, Apple, NVIDIA).
//
// 7. Bus Firewall / TrustZone-style Access Control
//   Threat: untrusted software or DMA engine accessing secure memory
//   How: hardware firewall on interconnect checks every transaction's

//        master ID and address against a permission table.
//        Violation -> transaction blocked + interrupt raised.
//   RTL detail: AXI firewall IP: comparator on {master_id, addr}
//        against region table; outputs allow/deny + irq.
//        Table programmed by trusted firmware in secure mode only.
//   Application: ARM TrustZone, RISC-V PMP (Physical Memory Protection),
//        automotive HSM isolation.
```

解释

Security RTL 设计的核心是“如何将密魄封封证在硬件里而不被提取”。

七种技术的防御层次：

Secure Boot = 防完整性抗抵（防替换固件）

PUF = 防密魄提取（无需存储密魄）

功耗隐蔽 = 防 DPA 攻击（防功耗信息泄漏）

电压/时钟监测 = 防物理故障注入攻击

内存加密 = 防总线探测/冗文攻击

JTAG 控制 = 防调试接口滥用（生产封锁）

Bus Firewall = 防越权访问（硬件隔离）

## Q7. Low Power RTL Design: Common Techniques and Applications (designer interview question)

QUESTION

What are the most commonly used low power RTL design techniques?

For each technique, describe: which power type it targets

(dynamic vs static), how it is implemented in RTL, and

a concrete application scenario where it is most effective.

ANSWER

```
// Power equation: P = a*C*V^2*f (dynamic) + I_leak*V (static)
//
// --- DYNAMIC POWER REDUCTION ---
//
```

```
// 1. Clock Gating (CG)
//      Targets: dynamic power (FF clock switching)
//      RTL: insert Integrated Clock Gating (ICG) cell in clock path.
//           Enable signal must be stable before clock edge (setup).
//           always_ff @(posedge clk_gated) if (en) q <= d;
//           Tool auto-inserts ICG; manual insertion for coarse blocks.
//      Savings: 20-40% of total chip dynamic power.
//      When most effective: datapaths with low activity factor
//           (e.g. video codec idle macroblock, bus interface idle).
//      Pitfall: glitch on enable -> spurious clock pulse -> metastability.
//                Enable must be registered or come from ICG with latch.
//
// 2. Operand Isolation (OI)
//      Targets: dynamic power in combinational logic
//      RTL: gate the inputs to arithmetic/logic blocks when the
//           output is not needed (not written back or not used).
//           assign alu_in_a = alu_en ? src_a : prev_alu_in_a;
//           Prevents unnecessary glitching through the ALU.
//      Savings: 10-20% additional beyond clock gating.
//      When most effective: wide ALUs, multipliers, FP units that
//           are frequently idle but still powered.
//
// 3. Pipeline Balancing and Retiming
//      Targets: dynamic power (allows voltage reduction at same throughput)
//      RTL: balance pipeline stages so no stage is the critical path.
//           Balanced pipeline -> can reduce Vdd while keeping same f.
//           P_dynamic ~ V^2 -> 10% Vdd reduction saves ~19% power.
//      When most effective: deeply pipelined cores (CPU, DSP, GPU shader).
//
// 4. Data Encoding (Bus Invert / Gray Code)
//      Targets: dynamic power on high-activity data buses

//   RTL: bus invert: if popcount(current XOR prev) > N/2,
//        invert the bus and set an invert flag.
//        Receiver XORs with invert flag to recover data.
//        Gray code for sequential counters: only 1 bit changes
//        per increment -> reduces switching on counter/address buses.
//   Savings: up to 50% on specific buses.
//   When most effective: wide processor data buses, address buses
//        with sequential access patterns (cache line fill).
//
// --- STATIC POWER REDUCTION ---
//
// 5. Power Gating (PG)
//   Targets: static leakage (cut off idle domain supply entirely)
//   RTL / Architecture: header (PMOS) or footer (NMOS) switch
//        inserted by power flow. Controlled by AON power controller.
//        RTL must define power intent in UPF:
//          create_power_domain PD_CODEC -elements {video_codec}
//          set_isolation / set_retention as needed.
//   Savings: ~100% leakage in gated domain (dominant at 7nm and below).
//   When most effective: large blocks with long idle periods
//        (GPU shader cluster, modem, display controller).
//   Pitfall: wake-up latency (usec-msec); inrush current on
//            power-up; isolation + retention cell overhead.
//
// 6. Multi-Vt Cell Assignment
//   Targets: static leakage
//   RTL -> synthesis: synthesis tool assigns HVT cells on
//        non-critical paths, LVT only on critical paths.
//        Directive: set_multi_cpu_usage / tool constraints.
//        HVT leakage is 5-10x lower than LVT.
//   Savings: significant without any RTL change.
```

```
//   When most effective: designs with many non-critical paths
//        (control logic, configuration registers, FSMs).
//
// 7. State Encoding Optimization

//   Targets: dynamic power (reduces transitions between states)
//   RTL: choose FSM encoding to minimize Hamming distance between
//        frequently-transitioning state pairs.
//        Gray encoding for sequential FSMs; one-hot for fast decode.
//        always_comb: next state logic has fewer toggle events.
//   When most effective: power-critical FSMs that cycle frequently
//        (memory controller state machine, protocol FSMs).
//
// 8. Memory Banking and Sleep Mode
//   Targets: both dynamic and static power in memories
//   RTL: divide large SRAM into banks; only activate the bank
//        being accessed (other banks in retention/sleep mode).
//        RTL controls: bank_en[N] gated by address decode output.
//   Savings: for a 4-bank memory with random access, 75% of banks
//        are idle per cycle -> ~75% less memory dynamic power.
//   When most effective: large caches, scratchpad memories,
//        register files wider than working set.
```

解释

Low power RTL 设计的八种主要手段，面试时默认按动态/静态分类回答。

动态功耗主要手段：

Clock Gating: 最常用，几乎所有项目必用，20-40% 动态功耗节省。

Operand Isolation: Clock gating 的补充，针对组合逻辑。

Data Encoding: 高活跃总线必用，如数据总线、地址总线。

静态功耗主要手段：

Power Gating: 最狂除彻底，应对大块长期闲置。

Multi-Vt: 最平衡，综合工具自动处理，无需 RTL 改动。

面试要点：能说出每种技术最适合的应用场景和潜在陷阱。

## Q8. Lockstep RISC-V: Implementation and Power Impact (designer interview question)

QUESTION

(a) What is a lockstep processor and why is it used in

safety-critical systems (e.g. automotive ASIL-D)?

(b) Describe how to implement lockstep for a RISC-V core.

What is compared, at what granularity, and what happens

on mismatch?

(c) What is the power impact? How can overhead be reduced

```
while maintaining the safety guarantee?
(d) Lockstep vs ECC+parity: differences and when to choose each.
```

ANSWER

```
// (a) What is Lockstep?
//
// Two identical cores execute the SAME instruction stream in parallel.
// Outputs are compared every cycle (or every instruction).
// If outputs differ -> hardware fault detected -> safety response.
//
// Why for ASIL-D:
//   Detects Single Event Upsets (SEUs), permanent faults (stuck-at),
//   and transient faults anywhere in the processor datapath.
//   Achieves ASIL-D / SIL-3 diagnostic coverage without formal proof
//   of every gate. Mandatory: automotive ADAS, airbag ECU, avionics.

// (b) Lockstep RISC-V Implementation
//
// Architecture:
//   Core A (main)   <-same ISA stream->   Core B (shadow, 1-2 cycles delayed)
//        |                                        |
//        +----------> Comparator <--------------+
//                          |
//                    Mismatch -> safety response
//
// What is compared at writeback each instruction:
//   Register file: write address + write data
//   Memory store:   store address + store data
//   PC after each instruction
//   Exception / interrupt signals
//
// Staggered lockstep (Core B delayed 1-2 cycles):
//   A common-cause glitch (EM pulse, power spike) hitting both cores
//   at the same physical time affects different pipeline stages ->
//   produces DIFFERENT wrong results -> mismatch -> fault detected.
//   Without stagger, same glitch at same stage gives same wrong result
//   -> undetected common-cause failure (CCF).
//
// On mismatch:

 //   1. Assert safety error output to system safety manager (NMI)
 //   2. Optionally: rollback to last checkpoint
 //   3. Hard stop / switch to safe state (ASIL-D requirement)
 //
 // RISC-V specifics:
 //   Data memory: Core A writes; Core B writes suppressed but compared.
 //   Interrupts: delivered to both cores at the same logical cycle.
 //   CSRs: compared at instruction boundary.

 // (c) Power Impact and Reduction
 //
 // Naive lockstep: ~2x dynamic power (two cores execute everything).
 // Plus ~5-10% for comparator logic.
 //
 // Power reduction techniques:
 //   1. Shadow core at lower voltage (near-threshold computing):
 //        Core B only needs to deliver results by comparison window,
 //        not in real-time. Run Core B at lower Vdd -> saves ~30-40%.
 //   2. Split lockstep: duplicate only safety-critical datapath
 //        (ALU, branch). Use ECC for memory. Overhead ~1.3-1.5x.
 //   3. Time redundancy: execute each instruction twice on same core.
 //        Halves throughput, uses 1 core. Used in ultra-low-power IoT.
 //   4. Clock gating in shadow core between comparisons.
 //
 // Real example: ARM Cortex-R52 lockstep ~40-60% power increase
 // (not 2x) due to shadow core clock gating optimizations.
```

```
// (d) Lockstep vs ECC/Parity
//
// ECC/Parity:
//   Protects specific storage (SRAM, register files, caches).
//   SECDED: corrects 1-bit errors, detects 2-bit errors.
//   Does NOT protect combinational logic (ALU, decoder, FSM).
//   Overhead: ~12.5% extra bits for 64-bit data.
//   Power: low (only on read/write paths).

//
// Lockstep:
//   Protects ENTIRE processor including combinational logic.
//   Overhead: ~2x area and power.
//   Cannot detect: faults affecting both cores identically
//        (mitigated by staggering).
//
// Choose ECC: memory protection, ASIL-B, space/power constrained.
// Choose Lockstep: ASIL-D, processor faults must be detected.
// Best practice: both -- lockstep for processor, ECC for memory.
```

解释

Lockstep 实现三个关键设计点：

1. 交错时序（stagger）：阳影核滑后 1-2 拍，使公因故障在不同流水线阶段产生不同的错误结果。

2. 比较精度：逆送阶段（writeback）比较，不需要每个流水线阶段都比较。

3. 功耗优化：阴影核降压运行是最容易实现的开销降低方式。

## Q9. Low Power Techniques Overview (designer interview question)

QUESTION

Describe the major low power techniques in modern SoC design.

For each, state: which power type it targets (dynamic vs static),

how it works, typical savings, and key tradeoffs.

ANSWER

```
// Power = P_dynamic + P_static
//    P_dynamic = a * C * V^2 * f        (switching)
//    P_static    = I_leak * V_dd        (leakage)
//
// --- DYNAMIC POWER TECHNIQUES ---
//
// 1. Clock Gating
//      Targets: dynamic power (FF switching)
//      How: AND gate in clock path; disable clock when enable inactive.
//      Savings: 20-40% of dynamic power. Most common technique.
//      Challenge: enable signal timing; glitch -> spurious clock pulse.
//      Types: fine-grain (per register), coarse-grain (per block).
//
// 2. Operand Isolation
//      Targets: dynamic power in combinational logic
//      How: gate inputs to logic when output not used (e.g. ALU inputs
//           gated when result not written back).
//      Savings: 10-20% additional beyond clock gating.
//      Challenge: identifying unused cycles; must not add latency.
//
// 3. DVFS (Dynamic Voltage and Frequency Scaling)
```

```
//      Targets: dynamic power (V^2 * f term)
//      How: reduce both V and f during low workload.
//           V cut by 20% -> power cut by ~36% (V^2 effect).
//      Savings: 50-70% at low performance states.
//      Challenge: V/f switching latency; timing closure at each corner;
//                 memory interfaces may have minimum frequency.
//
// 4. Multi-Vdd (Multiple Supply Domains)
//      Targets: dynamic and static power
//      How: non-critical blocks at lower Vdd; level shifters at crossings.
//      Savings: 20-40% on lower-voltage blocks.
//      Challenge: level shifter area; isolation cells; UPF/CPF flow.
//
// 5. Bus Encoding (e.g. bus invert, Gray code)

 //    Targets: dynamic power on high-activity buses
 //    How: encode data to minimize bit transitions (activity factor a).
 //    Savings: up to 50% on specific high-toggle buses.
 //    Challenge: encode/decode latency and area overhead.
 //
 // --- STATIC POWER TECHNIQUES ---
 //
 // 6. Power Gating
 //    Targets: static leakage (eliminates it in idle domain)
 //    How: header (PMOS) or footer (NMOS) switch cuts supply rail.
 //         State lost unless retention cells (SRPG) used.
 //    Savings: near 100% leakage in gated domain.
 //    Challenge: wake-up latency; inrush current; retention cell area;
 //               isolation cells; always-on logic required.
 //
 // 7. Multi-Vt Cell Selection
 //    Targets: static leakage
 //    How: HVT cells on non-critical paths (5-10x less leakage, slower);
 //         LVT only on critical paths (faster, more leakage).
 //    Savings: significant leakage reduction with careful path assignment.
 //    Challenge: timing closure; managed by synthesis tool automatically.
 //
 // 8. Body Biasing
 //    Targets: static (RBB) or performance (FBB)
 //    How: Reverse Body Bias (RBB) raises effective Vt -> less leakage.
 //         Forward Body Bias (FBB) lowers Vt -> faster but more leakage.
 //    Savings: RBB reduces leakage 2-5x in sleep mode.
 //    Challenge: not applicable to FinFET (body not accessible).
 //
 // --- SYSTEM LEVEL ---
 //
 // 9. Hierarchical Power Management
 //    Coarse: power-gate entire subsystems (display, modem, camera).
 //    Fine: clock-gate pipeline stages.
 //    SW integration: RTOS power states, ARM WFI/WFE instructions.

 //
 // 10. Memory Power Optimization
 //    SRAM sleep mode: retain state at near-threshold voltage.
 //    Memory banking: only activate accessed banks.
 //    Cache way power gating: power-gate unused ways.
```

解释

记忆公式：P = alpha*C*V^2*f + Ileak*Vdd，所有技术都是针对这个公式中的某一项。

面试关键要能说出每种技术针对的是动态还是静态功耗。

Clock Gating 最常用，20-40% 节能，几乎所有设计必用。

Power Gating 最彻除彻底，但唤醒延迟和状态保持是难点。

Multi-Vt 是静态功耗最平衡的手段，综合工具自动处理。

## Q10. Multi-Voltage Domain and DVFS Design Details (designer interview question)

QUESTION

(a) What is multi-voltage domain design? What special cells are required at domain boundaries and why?

(b) Describe DVFS in detail: V/F co-ordination, switching sequence, and why order matters.

(c) What are isolation cells and retention cells? When is each needed?

(d) Key UPF concepts for multi-voltage design (at least 4).

(e) Correct V/F switching sequence when scaling from high (1.1V, 1GHz) to low (0.8V, 500MHz) performance.

ANSWER

```
// (a) Multi-Voltage Domain and Special Cells
 //
 // Partition SoC into domains with different supply voltages
 // based on performance requirements:
 //    VDD_CPU    = 0.7-1.1V (DVFS)         VDD_GPU    = 0.7-1.0V (DVFS)
 //    VDD_MEM    = 1.1V (fixed)             VDD_IO     = 1.8V (fixed)
 //    VDD_AON    = 0.9V (always-on)
 //
 // Level Shifter (LS):
 //    Converts signal voltage when crossing between domains.
 //    Low-to-High (LH): input from low-V, output at high-V.
 //    High-to-Low (HL): input from high-V, output at low-V.
 //    Why needed: a 0.8V logic-1 may not be recognized as logic-1
 //    in a 1.1V domain (below switching threshold -> malfunction).
 //
 // Isolation Cell (ISO):
 //    Clamps output of power-gated domain to known value (0 or 1)
 //    when that domain is off.
 //    Without ISO: unpowered output floats -> X propagates into
 //    always-on logic -> unpredictable behavior.
 //    ISO_AND: output = input AND iso_en (clamps to 0 when off)
 //    ISO_OR:    output = input OR     iso_en (clamps to 1 when off)
 //    ISO enable signal comes from AON domain.
 //
 // Retention Cell (SRPG):
 //    FF with shadow latch powered by separate always-on supply (Vret).
 //    SAVE: transfer FF->shadow before power-down.
 //    RESTORE: transfer shadow->FF after power-up.
 //    Cost: ~2-3x area vs normal FF; use only for critical state.

 // (b) DVFS Co-ordination
 //
 // V and f are coupled: lower V -> slower gates -> lower max f.
 // Characterized OPP (Operating Performance Point) table:
 //    High perf:     1.1V / 1 GHz

//   Nominal:       1.0V / 800 MHz
```

```
//   Low power:     0.9V / 600 MHz
//   Ultra-low:     0.8V / 400 MHz
//
// Control loop:
//   1. OS monitors workload/temp/battery
//   2. Requests new OPP from DVFS controller
//   3. Controller coordinates with PMIC (voltage) and PLL (frequency)
//   4. Switches following correct sequence (see (e))

// (c) Isolation and Retention Cells
//
// Isolation Cell behavior:
//   Domain ON    (iso_en=0): output = input (pass-through)
//   Power-down sequence: assert iso_en=1 BEFORE cutting power
//                        output clamped to 0 (or 1)
//   Power-up sequence:     deassert iso_en=0 AFTER supply stable
//                        output = input (restored)
//   When needed: any signal leaving a power-gated domain into AON.
//
// Retention Cell behavior:
//   Save (going off):      save_en=1 -> shadow = FF.Q, then cut power
//   Restore (coming on): restore power, restore_en=1 -> FF.D = shadow
//   When needed: registers holding context that must survive power-down
//     (CPU registers, peripheral config, interrupt pending bits).

// (d) Key UPF Commands/Concepts
//
// 1. create_power_domain: defines a domain and its elements.
//    create_power_domain PD_CPU -elements {cpu_top}
//
// 2. create_supply_net: defines physical supply nets.
//    create_supply_net VDD_CPU -domain PD_CPU
//
// 3. set_isolation: specifies ISO cells at domain boundary.

 //    set_isolation ISO_CPU -domain PD_CPU
 //        -isolation_power_net VDD_AON -clamp_value 0 -applies_to outputs
 //
 // 4. set_retention: specifies retention strategy.
 //    set_retention RET_CPU -domain PD_CPU
 //        -retention_power_net VRET
 //        -save_signal {save_en posedge}
 //        -restore_signal {restore_en posedge}
 //
 // 5. set_level_shifter: specifies LS at boundary.
 //    set_level_shifter LS_CPU -domain PD_CPU -applies_to outputs
 //
 // 6. Power State Table (PST): enumerates valid domain state
 //    combinations (on/off/retention). Used by verification tools.

 // (e) Correct V/F Switching Sequence
 //
 // Scaling DOWN (high -> low performance): FREQUENCY FIRST
 //   Step 1: Reduce clock: 1 GHz -> 500 MHz (wait for PLL lock)
 //   Step 2: Reduce voltage: 1.1V -> 0.8V (wait for regulator settle)
 //
 //   Why frequency first?
 //   If voltage drops first (to 0.8V) while still at 1GHz:
 //   gates are slower at low V -> setup violations -> wrong results.
 //   Dropping frequency first creates timing slack; then lowering V
 //   increases delay but period is now large enough to accommodate it.
 //
 // Scaling UP (low -> high performance): VOLTAGE FIRST
```

```
//   Step 1: Raise voltage: 0.8V -> 1.1V (wait for regulator settle)
//   Step 2: Raise clock: 500 MHz -> 1 GHz
//
//   Why voltage first?
//   If frequency rises first (to 1GHz) at 0.8V:
//   gates too slow for 1GHz at low V -> setup violations.
//   Raising voltage first makes gates fast enough for higher frequency.

//
// Mnemonic:
//   Going DOWN: slow DOWN first (freq down), then reduce V.
//   Going UP:    raise V first (build margin), then speed UP (freq up).
```

解释

三种边界单元记忆方法：

Level Shifter = 不同电压域跨越信号电平转换（永远需要）

Isolation Cell = 电源关闭时防止输出浮空（接收方是常开域时需要）

Retention Cell = 电源关闭后保存状态（需要单独常开存储电源）

DVFS 切换顺序是高频考点：

降频 -> 降压：先降频为时序预留裕量，应对低电压下门活延迟的增大。

升压 -> 升频：先升压让门活展开足够快，再提高频率不产生 setup violation。

## Q11. LRU and PLRU Cache Replacement: 16-Way Implementation (designer interview question)

QUESTION

(a) How do you implement true LRU for a 16-way cache set?

How many state bits are required? What are the hardware costs?

(b) How do you implement PLRU (Pseudo-LRU) for a 16-way set

using a binary tree?

- How many layers does the tree have?

- How many bits per set?

- How do you find the victim way?

- How do you update bits after an access?

(c) Compare true LRU vs PLRU: accuracy, cost, when to choose each.

ANSWER

```
// (a) True LRU for 16-way cache
 //
 // Must track EXACT access order of all 16 ways.
 //
 // Method: age counter per way (4 bits, values 0-15)
 //    Total state: 16 ways * 4 bits = 64 bits per set
 //    (Alternative: comparison matrix: 16*15/2 = 120 bits per set)
 //
 // On cache hit (way W accessed):
 //    Reset W's age to 0 (most recent).
 //    Increment age of all ways that were "younger" than W.
 //    -> Updates up to 15 counters per access.
 //
```

```
// On cache miss (evict):
//    Find way with maximum age = LRU victim.
//    Requires 15-input comparator tree.
//
// Hardware cost (16-way, 1024-set cache):
//    State: 64 bits/set * 1024 sets = 65,536 bits = 8 KB extra.
//    Update: 15 counter updates per hit -> high power.
//    Lookup: comparator tree -> multi-cycle or large combinational.
//    Practical: true LRU rarely used beyond 4-way due to cost.

// (b) PLRU (Pseudo-LRU) Binary Tree for 16-way
//
// Structure: complete binary tree with 16 leaves (one per way).
// Each internal node stores 1 bit pointing toward the LRU subtree.
//
// Layers and bits:
//    Layer 0 (root):   1 node   = 1 bit
//    Layer 1:          2 nodes = 2 bits
//    Layer 2:          4 nodes = 4 bits
//    Layer 3:          8 nodes = 8 bits
//    Total internal nodes = 1+2+4+8 = 15 bits per set
//    (General formula: N-1 bits for N-way PLRU)

//
// PLRU tree (bit=0 -> go left, bit=1 -> go right):
//                              [b0]
//                  0/                \1
//               [b1]                      [b2]
//          0/        \1              0/      \1
//        [b3]     [b4]             [b5]     [b6]
//      0/ \1      0/ \1           0/ \1      0/ \1
//    [b7][b8][b9][b10] [b11][b12][b13][b14]
//    /\    /\    /\     /\      /\     /\     /\     /   //   W0 W1 W2 W3 W4 W5   W6   W7
  W8 ...   W15
//
// Finding victim way (4 layers = log2(16) = 4 mux levels):
//   Follow bits from root to leaf:
//   bit=0 -> go left subtree, bit=1 -> go right subtree.
//   After 4 decisions, arrive at victim way leaf.
//   Example: b0=0, b1=1, b4=0, b9=1 -> victim = W5
//     Root b0=0 -> left          -> b1
//     b1=1            -> right -> b4
//     b4=0            -> left    -> b9
//     b9=1            -> right -> W5 (victim)
//
// Updating after accessing way W (e.g. W5, path: b0,b1,b4,b9):
//   At each node on path to W, flip bit to point AWAY from W.
//   b0: W5 in right subtree -> set b0=0 (now points left, away from W5)
//   b1: W5 in right subtree -> set b1=0 (points left)
//   b4: W5 in right subtree -> set b4=0 (points left)
//   b9: W5 is right leaf             -> set b9=0 (points toward W4, not W5)
//   Only 4 bits modified per access (vs 15 for true LRU).
//
// PLRU lookup: log2(16) = 4 layers, each layer is one mux/decode.
// Total: 4 levels of combinational logic -> fast, single-cycle.

// (c) Comparison
//
// Metric               | True LRU (16-way) | PLRU (16-way)

// ---------------|-------------------|------------------
// Bits per set         | 64 bits                  | 15 bits
// Accuracy             | 100% exact               | ~96-98% match rate
```

```
// Update/access         | 15 counter writes | 4 bit writes
// Lookup latency        | Comparator tree           | 4-level mux (fast)
// Area overhead         | High                      | Low (~4x less state)
// Power                 | High                      | Low
//
// PLRU guarantees: evicted way was NOT accessed in last log2(N)
// unique accesses. Not exact LRU but miss rate penalty < 1-2%.
//
// Choose True LRU: 2-way or 4-way; miss rate critical (LLC server).
// Choose PLRU: 8-way or more; area/power constrained; embedded;
//   most modern L1/L2 caches use PLRU or similar approximations.
```

解释

关键数据：16-way True LRU = 64 bits/set，16-way PLRU = 15 bits/set。

PLRU 二叉树 4 层，每层使用 1/2/4/8 个 bit，共 15 个内部节点。

查询受害 way：4 层 mux，每层 1 个选择，速度极快。

更新：访问 way W 后，翻转路径上 4 个 bits，指向另一边（远离 W）。

实际 CPU 中 L1/L2 cache 几乎全部用 PLRU 或类似方案。

## Q12. Parity Check: 16-bit Data Implementation and Gate Layers (designer interview question)

QUESTION

(a) For 16-bit data, how many parity bits are needed?

Describe simple even parity vs Hamming code (SECDED).

(b) Implement even parity for 16-bit data in hardware.

How many XOR gates are needed and how many logic layers?

(Hint: the 16-input XOR tree has 4 layers of 2-input XOR gates.)

(c) Implement Hamming code for 16-bit data (SECDED):

Where are the parity bits placed? How are they computed?

How does the decoder detect and correct a 1-bit error?

(d) What is the difference in hardware cost between simple parity

and SECDED? When would you use each?

ANSWER

```
// (a) Parity bits needed for 16-bit data
//
// Simple even/odd parity: 1 bit
//   One parity bit P = XOR of all 16 data bits.
//   Detects odd number of bit errors (1, 3, 5...).
//   Cannot correct errors. Cannot detect even-number errors.
//
// Hamming code (SECDED - Single Error Correct, Double Error Detect):
//   Minimum parity bits: 2^r >= n + r + 1 where n = data bits
//   For n=16: 2^5 = 32 >= 16 + 5 + 1 = 22 -> r = 5 check bits
//   Plus 1 extra bit for double-error detection = 6 total check bits
//   Transmitted word width: 16 + 6 = 22 bits (SECDED)
//
// Common shorthand: "SECDED for 16-bit needs 6 check bits (5+1)"
```

```
// (b) Even parity hardware for 16-bit data
//
// Parity = D0 XOR D1 XOR D2 XOR ... XOR D15
//
// XOR tree structure (binary tree, 2-input XOR gates):
//
//   Layer 1 (8 gates): pair up 16 inputs
//    X0   = D0   ^ D1
//    X1   = D2   ^ D3
//    X2   = D4   ^ D5
//    X3   = D6   ^ D7
//    X4   = D8   ^ D9
//    X5   = D10 ^ D11
//    X6   = D12 ^ D13
//    X7   = D14 ^ D15
//
//   Layer 2 (4 gates): combine pairs
//    Y0 = X0 ^ X1
//    Y1 = X2 ^ X3
//    Y2 = X4 ^ X5

//    Y3 = X6 ^ X7
//
//   Layer 3 (2 gates):
//    Z0 = Y0 ^ Y1
//    Z1 = Y2 ^ Y3
//
//   Layer 4 (1 gate):
//    Parity = Z0 ^ Z1
//
// Summary:
//   Total 2-input XOR gates: 8 + 4 + 2 + 1 = 15 gates
//   Logic layers: 4 layers (log2(16) = 4)
//   Critical path delay: 4 * tXOR
//
// Note: 16 inputs -> log2(16) = 4 layers
//        General formula: N inputs -> log2(N) layers, N-1 gates

// (c) Hamming SECDED for 16-bit data
//
// Bit positions (1-indexed): check bits at powers of 2
//   P1   at position   1 (2^0)
//   P2   at position   2 (2^1)
//   P4   at position   4 (2^2)
//   P8   at position   8 (2^3)
//   P16 at position 16 (2^4)
//   P_overall at position 0 (overall parity for SECDED)
//   Data bits fill remaining positions: 3,5,6,7,9,10,11,12,13,14,15,17,18,19,20,21
//
// Each parity bit covers positions where that bit position has a 1
// in its binary representation:
//   P1   covers: positions with bit0=1 (1,3,5,7,9,11,13,15,17,19,21)
//   P2   covers: positions with bit1=1 (2,3,6,7,10,11,14,15,18,19)
//   P4   covers: positions with bit2=1 (4,5,6,7,12,13,14,15,20,21)
//   P8   covers: positions with bit3=1 (8-15, plus 24... not applicable)
//   P16 covers: positions with bit4=1 (16-21)

 //   P_overall: XOR of ALL bits in codeword
 //
 // Error detection and correction:
 //   On read: recompute all parity bits -> form syndrome S = [s4,s3,s2,s1,s0]
 //   S = 0: no error (or undetectable even-bit error caught by P_overall)
```

```
//   S != 0, P_overall mismatch: single-bit error at position S -> correct
//   S != 0, P_overall match: double-bit error -> report uncorrectable error
//
// Hardware:
//   Check bit generation: 5 XOR trees, each covering ~10 bits
//     Each tree: ~10 inputs -> 4 layers
//   Error correction: 22-bit error vector decode + XOR to flip bad bit
//   Total overhead: ~40% more logic than simple parity

// (d) Simple Parity vs SECDED - When to Use
//
// Simple Parity (1 check bit):
//   Detects:   single-bit errors (and any odd number of errors)
//   Corrects: nothing
//   Cost:      1 extra bit, 15 XOR gates, 4 gate layers
//   Use when: error detection (not correction) is sufficient,
//             retransmit on error (UART, PCIe CRC), low-cost designs,
//             data path errors are rare (SRAM with ECC backup).
//
// SECDED (6 check bits for 16-bit data):
//   Detects:   all 1-bit and 2-bit errors
//   Corrects: all 1-bit errors automatically
//   Cost:      6 extra bits (37.5% overhead), 5 XOR trees, more area
//   Use when: automatic correction needed without retry, high-reliability
//             memory (cache, register file, DRAM ECC), cannot tolerate
//             the latency of error retransmit.
//
// Summary table:
//   Metric           | Simple parity | SECDED (Hamming)
//   -----------------|---------------|------------------

//   Check bits       | 1             | 6 (for 16-bit data)
//   Detects          | 1-bit error   | 1-bit and 2-bit errors
//   Corrects         | None          | 1-bit errors
//   Gate count       | 15 XOR gates  | ~5x15 = 75 XOR gates
//   Logic layers     | 4             | 4 (per check bit tree)
//   Overhead (bits) | 6.25%          | 37.5%
```

解释

16bit 数据的寇校：

简单奇偶校验：1 bit，15 个两输入 XOR 门，4 层逻辑。

Hamming SECDED：6 bit（5+1），占 16+6=22 bits，5 条 XOR 树。

XOR 树层数公式：log2(N) 层，16 inputs -> 4 层，这是面试的关键数据。

应该记住：

- 违背XOR门数 = N-1 = 15 个（N=16）

- 层数 = log2(N) = 4 层

综合类型选择：

数据路径错误少且允许重试：简单奇偶校验。

缓存、寄存器文件、DRAM：必须 SECDED 自动纠错。

## Q13. Digital Design: Build an Inverter Using Gates X and Y

QUESTION

You are given two black-box gates X and Y, each with two inputs

A and B and one tri-state output. Z = high-impedance (floating).

Using only gates X and Y, design a circuit whose output is the

```
logical inverse of a single input A (A=1->out=0, A=0->out=1).
```

ANSWER

```
// Gate X and Y truth tables:
  //
  // Gate X:                Gate Y:
  // +----+-----+           +----+-----+
  // | AB | out |           | AB | out |
  // +----+-----+           +----+-----+
  // | 00 |    1   |        | 00 |    Z   |
  // | 01 |    Z   |        | 01 |    Z   |
  // | 10 |    Z   |        | 10 |    Z   |
  // | 11 |    Z   |        | 11 |    0   |
  // +----+-----+           +----+-----+
  // X: output=1 ONLY when AB=00, else Z       (pull-up driver)
  // Y: output=0 ONLY when AB=11, else Z       (pull-down driver)
  //
  // Analysis of X and Y:
  // Gate X: output is 1 ONLY when A=0, B=0. Otherwise high-Z.
  //           X is essentially: out = 1 if (~A & ~B), else Z
  //           -> pulls output HIGH when both inputs are 0.
  //
  // Gate Y: output is 0 ONLY when A=1, B=1. Otherwise high-Z.
  //           Y is essentially: out = 0 if (A & B), else Z
  //           -> pulls output LOW when both inputs are 1.
  //
  // These are open-collector/open-drain style gates:
  //   X acts as a pull-up driver (drives 1, else Z)
  //   Y acts as a pull-down driver (drives 0, else Z)
  //
  // Inverter construction:
  // Connect both A inputs of X and Y to the same signal IN.
  // Connect both B inputs of X and Y to IN as well.
  // Wire-OR (connect) both outputs together -> OUT.
  //
  //           +-------+
  //   IN -->|A         |
  //           |    X   |---+

 //    IN -->|B        |    |
 //           +-------+     +---> OUT
 //           +-------+     |
 //    IN -->|A        |    |
 //           |    Y   |---+
 //    IN -->|B        |
 //           +-------+
 //
 // When IN = 0 (A=0, B=0):
 //    X output = 1    (active)
 //    Y output = Z    (floating)
 //    OUT = 1    (driven by X)    -> correct, 0 inverted = 1
 //
 // When IN = 1 (A=1, B=1):
 //    X output = Z    (floating)
 //    Y output = 0    (active)
 //    OUT = 0    (driven by Y)    -> correct, 1 inverted = 0
```

```
//
// At any given time, exactly one gate drives the output;
// the other is high-Z. No bus contention occurs.
// This is the wired-logic / tri-state bus inversion technique.
//
// Truth table verification:
//    IN=0: X(00)=1, Y(00)=Z -> OUT=1       [correct: ~0=1]
//    IN=1: X(11)=Z, Y(11)=0 -> OUT=0       [correct: ~1=0]
```

解释

这道题考察三态逻辑（tri-state）和总线驱动的理解。

X 门是高电平驱动器（pull-up driver）：只有 AB=00 时输出 1，否则浮空。

Y 门是低电平驱动器（pull-down driver）：只有 AB=11 时输出 0，否则浮空。

把两个门的输出线接在一起（wired-OR），输入端都接同一个信号 IN：

IN=0 时只有 X 驱动输出为 1；IN=1 时只有 Y 驱动输出为 0。

这是 CMOS 推挽输出的三态等价：PMOS 上拉对应 X，NMOS 下拉对应 Y。

面试重点：理解三态总线的 wired-logic 特性，以及不同时驱动避免总线冲突。

## Q14. Verilog: Implement Fibonacci Sequence

QUESTION

Implement the Fibonacci sequence in Verilog/SystemVerilog.

Output the N-th Fibonacci number, where the sequence starts:

F(0)=0, F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5, ...

Provide:

(a) A combinational (non-clocked) implementation using a function.

(b) A sequential (clocked) implementation using registers that

computes Fibonacci numbers one per clock cycle.

ANSWER

```
// (a) Combinational: recursive function (for small N, synthesis only)
module fib_comb #(parameter N = 8) (
  output logic [31:0] result
);
  function automatic logic [31:0] fib(input int n);
     if (n <= 0) return 0;
     if (n == 1) return 1;
     return fib(n-1) + fib(n-2);
  endfunction

  assign result = fib(N);
endmodule

// (b) Sequential: compute one Fibonacci number per clock cycle
//      Outputs the running sequence: F(0), F(1), F(2), ...
module fib_seq (
  input     logic        clk,
  input     logic        rst_n,
  output logic [31:0] fib_out        // current Fibonacci number
);
```

```
  logic [31:0] a, b;     // a = F(n-1), b = F(n)

  always_ff @(posedge clk or negedge rst_n) begin
     if (!rst_n) begin
        a        <= 32'd0;   // F(0) = 0
        b        <= 32'd1;   // F(1) = 1
        fib_out <= 32'd0;
     end else begin
        fib_out <= a;           // output current F(n)
        a        <= b;          // shift: a = old b
        b        <= a + b;      // b = old a + old b = F(n+1)
     end
  end
endmodule

 // Simulation output (fib_out each clock):
 //     Reset released: fib_out = 0 (F0)
 //     Cycle 1:             fib_out = 1 (F1)
 //     Cycle 2:             fib_out = 1 (F2)
 //     Cycle 3:             fib_out = 2 (F3)
 //     Cycle 4:             fib_out = 3 (F4)
 //     Cycle 5:             fib_out = 5 (F5)
 //     ...

 // (c) Compute F(N) exactly after N cycles with a counter
 module fib_nth #(parameter WIDTH = 32) (
     input     logic               clk,
     input     logic               rst_n,
     input     logic [7:0]         n,       // compute F(n)
     output logic [WIDTH-1:0] result,
     output logic                  valid    // high when result is ready
 );
     logic [WIDTH-1:0] a, b;
     logic [7:0]            cnt;

     always_ff @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
           a        <= 0;
           b        <= 1;
           cnt      <= 0;
           valid    <= 0;
           result <= 0;
      end else if (cnt < n) begin
           a       <= b;
           b       <= a + b;
           cnt     <= cnt + 1;
           valid <= 0;
      end else begin
           result <= a;
           valid    <= 1;

      end
     end
 endmodule
```

解释

斐波那契数列的 Verilog 实现考察寄存器的迭代更新写法。

关键是用两个寄存器 a 和 b 分别保存 F(n-1) 和 F(n)：

每个时钟：a <= b，b <= a + b，这样 b 就变成了 F(n+1)。

注意非阻塞赋值：a 和 b 同时更新，使用的是上一拍的值，不会出现竞争。

组合逻辑递归函数只适合小 N（综合时会展开全部递归，面积指数级增长）。

实际工程中用顺序版本（b），或加计数器精确控制输出第 N 个值（c）。

## Q15. Digital Design: NOR + Two D-FFs Circuit Waveform

QUESTION

For the following circuit, draw the waveform of signals A, B,

and the NOR_out (= FF1.D). Assume both FFs start with Q=0

(A=0, B=0), positive-edge triggered, no reset.

ANSWER

```
// Circuit diagram:
 //
 // +-------------------------------------------------------------------------+
 // |
   |
 // |              +-------+             +----------+                     +----------+
   |
 // |       +--->|             |         |       FF1        |             |   FF2        |
   |
 // |       | +->|      NOR    |-----+>| D               Q |--+-->| D                Q
   |---+--> B(out) |
 // |       | |    |           |         |                    | A|          |              |   |
   |
 // |       | |    +-------+             |       clk>       |       |     |   clk>       |   |
   |
 // |       | |                          +----------+               |     +----------+       |
   |
 // |       | |                                      ^                |          ^
   |            |
 // |       | |                                      | clk            |          | clk
   |            |
 // |       | +---- (A feedback) ----+                                |          |
   |            |
 // |       +------- (B feedback) ------------+--------+----------+
   |
 // +-------------------------------------------------------------------------+
 //
 // Signal connections:
 //     A              = FF1.Q     (output of first                flip-flop)
 //     B              = FF2.Q     (output of second flip-flop)
 //     NOR inputs = A and B
 //     NOR output = FF1.D             (NOR_out)
 //     FF2.D          = A
 //     Both FFs       : posedge clk triggered
 //
 // Initial state: A=0, B=0
 //
 // Step-by-step state analysis:
 // NOR_out = ~(A | B)
 // On each posedge clk: A_next = NOR_out,                              B_next = A
 //
 // +-------+-----+-----+------------+--------+--------+
 // | Cycle |          A   |   B   | NOR=~(A|B) | A_next | B_next |
 // +-------+-----+-----+------------+--------+--------+
 // |       0     |    0   |   0   |         1          |       1         |   0      |
```

```
// |      1     |    1   |   0   |         0          |       0         |   1     |

// |      2     |    0   |    1    |       0         |     0         |   0   |
// |      3     |    0   |    0    |       1         |     1         |   0   | <- repeat
// |      4     |    1   |    0    |       0         |     0         |   1   |
// |      5     |    0   |    1    |       0         |     0         |   0   |
// |      6     |    0   |    0    |       1         |     1         |   0   |
// +-------+-----+-----+------------+--------+--------+
//
// Waveform (each cell = 1 clock period, value shown AFTER posedge):
//
// clk        : _/~\_/~\_/~\_/~\_/~\_/~\_/~\_/~                  //           |   |     |   |
  |   |   |
// NOR=D1: _/~~~~\________/~~~~\________/~~~~\_
//              1       0        0    1       0     0        1
//              |       |        |    |       |     |        |
// A          : ______/~~~~\________/~~~~\________/~
//              0       1        0    0       1     0        0
//              |       |        |    |       |     |        |
// B          : __________/~~~~\________/~~~~\______
//              0       0        1    0       0     1        0
//              |       |        |    |       |     |        |
//              C0      C1       C2   C3      C4    C5       C6
//
// Period = 3 clock cycles.
// State sequence: (A,B) = (0,0) -> (1,0) -> (0,1) -> (0,0) -> ...
// This is a mod-3 counter / divide-by-3 circuit.
// Neither A nor B is ever HIGH simultaneously (NOR would be 0).
// NOR output is HIGH only when both A=0 and B=0.
```

解释

这道题的关键是正确追踪状态机的三个状态。

NOR = ~(A|B)：只有 A 和 B 都为 0 时输出才为 1。

每个时钟沿：A 采样 NOR_out，B 采样旧的 A（两级流水）。

初始 (A=0,B=0) -> NOR=1 -> 下一周期 (A=1,B=0) -> NOR=0

-> 下一周期 (A=0,B=1) -> NOR=0 -> 下一周期 (A=0,B=0) -> 回到初始。

周期为 3 个时钟，本质上是一个 mod-3 计数器/分频器。

面试重点：能快速列出状态转移表，然后画出对应波形。

## Q16. Debug: Multiple CPUs Reading Inconsistent Data

QUESTION

Multiple CPUs are reading data in parallel and getting inconsistent

results (different CPUs see different values for the same address).

How do you systematically debug this issue?

Consider both hardware and software perspectives.

ANSWER

```
// Root cause categories to investigate:

// 1. CACHE COHERENCE VIOLATION
//   Most common cause in multi-CPU systems.
```

```
//   Each CPU has its own L1/L2 cache; if one CPU writes without
//   properly invalidating other caches, other CPUs read stale data.
//
//   Debug steps:
//   - Check if the cache coherence protocol (MESI/MOESI) is enabled.
//   - Verify that the memory region is marked as cacheable and
//     coherent (not as Device or non-cacheable in the page table).
//   - Use hardware performance counters to check cache invalidation
//     and coherence traffic on the interconnect (snoop count, etc.).
//   - Inject known patterns (e.g. write 0xDEADBEEF from CPU0) and
//     immediately read from CPU1 -- if stale, coherence is broken.

// 2. MEMORY ORDERING / BARRIER MISSING
//   Out-of-order execution and write buffer reordering can cause
//   a write to appear in memory AFTER a subsequent read from
//   another CPU's perspective.
//
//   Debug steps:
//   - Check if memory barriers (DMB/DSB on ARM, MFENCE on x86)
//     are missing between write and read operations.
//   - Inspect compiler output: is volatile or atomic being used?
//     Without volatile, compiler may reorder or cache in registers.
//   - Reproduce with single-threaded execution -- if issue disappears,
//     strongly suggests ordering/synchronization bug.

// 3. RACE CONDITION / ATOMICITY VIOLATION
//   A read-modify-write operation is not atomic; two CPUs may
//   both read the same value, modify, and write back, losing one update.
//
//   Debug steps:
//   - Identify shared variables accessed without locks or atomics.

 //   - Add mutex/spinlock around critical section and check if
 //     issue disappears.
 //   - Use atomic instructions (LDREX/STREX on ARM, LOCK CMPXCHG
 //     on x86) for counter increments and flag updates.

 // 4. HARDWARE BUG: INTERCONNECT / BUS PROTOCOL ERROR
 //   Rare but possible: the NoC (Network-on-Chip), AXI crossbar,
 //   or DRAM controller has a bug that returns wrong data.
 //
 //   Debug steps:
 //   - Run single-CPU access to the same address -- if still wrong,
 //     the bug is in hardware, not coherence.
 //   - Use hardware bus monitors / logic analyzers to capture all
 //     transactions on the interconnect: verify addresses, data,
 //     and response ordering.
 //   - Check AXI/CHI transaction IDs: out-of-order completion should
 //     always match data to the correct ID; ID aliasing is a known bug.
 //   - Test with a simpler memory pattern (single cache line, natural
 //     alignment) to isolate address-dependent issues.

 // 5. TLB / PAGE TABLE INCONSISTENCY
 //   If CPUs have different virtual-to-physical mappings (e.g. after
 //   a page table update without TLB shootdown), they may read from
 //   different physical locations.
 //
 //   Debug steps:
 //   - Verify TLB invalidation broadcasts (TLBI on ARM) after any
 //     page table modification.
 //   - Use physical addresses directly (bypass MMU in debug mode)
 //     and check if the issue persists.
```

```
// 6. SYSTEMATIC DEBUG METHODOLOGY
//   Step 1: Isolate -- reduce to fewest CPUs that reproduce the bug.
//           Single CPU? -> hardware or ordering bug.
//           Two CPUs? -> coherence or race condition.

//   Step 2: Identify memory region -- check cacheability attributes.
//   Step 3: Check protocol -- enable coherence debug logging if available.
//   Step 4: Add instrumentation -- software counters, log timestamps,
//           compare sequences of reads and writes across CPUs.
//   Step 5: Use tools:
//           - Valgrind/Helgrind: race condition detection (simulation)
//           - Hardware performance counters: snoop hit/miss rates
//           - Logic analyzer / protocol analyzer on bus
//           - RTL simulation with full coherence checking assertions
```

解释

多 CPU 读数据不一致是经典的并行系统 debug 题，考察对整个内存系统的理解。

按可能性从高到低排查：

1. 缓存一致性（最常见）：确认 MESI/MOESI 协议正确启用，内存属性正确。
2. 内存序/屏障缺失：写操作后没有 barrier，导致其他 CPU 看到旧值。
3. 竞争条件：非原子的读-改-写操作导致更新丢失。
4. 硬件 bug：NoC/AXI 总线、DRAM 控制器的数据路由或 ID 匹配问题。
5. TLB 不一致：页表更新后没有广播 TLB 无效化。

Debug 方法论：先单 CPU 隔离（排除软件竞争），再逐步加 CPU，

配合总线监控工具和断言进行系统性定位。

## Q17. Computer Architecture: MOESI Protocol -- Two Processors Writing Same Cache Line

QUESTION

(a) Briefly explain the MOESI cache coherence protocol states:

M, O, E, S, I.

(b) Two processors P1 and P2 simultaneously try to write to the

same cache line address. Walk through the MOESI protocol

transactions step by step.

(c) When both processors issue invalidate (INVL) simultaneously,

what determines the order of invalidation?

Is it decided by the processor or by the bus/interconnect?

What happens to the "loser"?

ANSWER

```
// (a) MOESI states:
  //
  // M (Modified):      This cache has the ONLY valid copy. Data is dirty
  //                   (differs from memory). Other caches: no copy.
  //                   On eviction: must write back to memory.
  //
```

```
  // O (Owned):        This cache has a dirty copy AND other caches may
  //                   have stale (S-state) copies. Owner is responsible
  //                   for supplying data on snoop and writing back.
  //                   (MOESI extension over MESI: allows sharing dirty data)
  //
  // E (Exclusive): This cache has the ONLY valid copy. Data is clean
  //                   (matches memory). Can silently upgrade to M on write
  //                   without a bus transaction.
  //
  // S (Shared):       This cache has a valid copy. Other caches may also
  //                   have copies. Data is clean (or owner has dirty copy
  //                   in MOESI). Cannot write without invalidating others.
  //
  // I (Invalid):      Cache line is not valid. Must fetch from memory
  //                   or another cache before use.

  // (b) Two processors simultaneously writing same cache line:
  //
  // Initial state: both P1 and P2 have the line in S (Shared) state.
  // (Common starting point; or both in I state for a cold scenario.)
  //
  // Step 1: P1 issues Write (BusUpgr / RWITM - Read With Intent To Modify)
  //              P2 issues Write (BusUpgr / RWITM) simultaneously
  //
  // Step 2: Both transactions appear on the shared bus/interconnect.
  //              The BUS ARBITER grants one processor priority.
  //              Assume P1 wins arbitration.
  //
  // Step 3: P1's BusUpgr is broadcast on the bus.

//         P2 snoops P1's transaction -> P2 sees someone else
//         is invalidating -> P2 must invalidate its own copy.
//         P2's cache line: S -> I
//
// Step 4: P1 receives acknowledgment (all other caches invalidated).
//         P1 cache line: S -> M (Modified)
//         P1 completes its write locally.
//
// Step 5: P2's BusUpgr now gets to run (it was queued/retried).
//         P2 sees the line is in M state at P1.
//         P2 issues BusRd (read request) or RWITM.
//         P1 must intervene: P1 flushes its dirty line to the bus
//         (or to memory), cache line: M -> I (or M -> S/O).
//         P2 receives the data, upgrades to M, completes its write.
//
// Net result: writes are serialized by bus arbitration.
// One processor wins, completes write first; the other retries.
// Both writes DO complete, but NOT simultaneously -- they are ordered.

// (c) What determines invalidation order?
//
// The BUS (interconnect / coherence fabric) decides, NOT the processor.
//
// Reason: the processors issue invalidate requests independently and
// (from their local perspective) simultaneously. There is no way for
// a processor to know the other is doing the same thing without
// communicating through the shared bus.
//
// Mechanism:
//   1. Bus Arbitration: the bus has an arbiter (round-robin, priority,
//     or first-come-first-served) that grants one transaction at a time.
//     The winning processor's invalidate is broadcast FIRST.
//
```

```
//   2. Snooping: the losing processor's cache SNOOPS the winning
//      invalidate on the bus, sees its own pending upgrade is pre-empted,

//      and self-invalidates before retrying.
//
//   3. In directory-based coherence (scalable systems):
//      The HOME NODE (directory) receives both upgrade requests,
//      serializes them, grants one, sends NACK or retry to the other.
//      The directory is the single point of serialization.
//
// What happens to the "loser":
//   - Snooping bus: loser sees its line invalidated by winner's broadcast.
//      Loser must re-issue a Read (BusRd or RWITM) to get the updated
//      data, then retry the write. This is called a "snoop hit on upgrade"
//      or "upgrade collision".
//   - Directory: loser receives a NACK from the directory and retries
//      the upgrade request after waiting for the winner to complete.
//
// Key rule: the bus/interconnect is the SINGLE POINT OF SERIALIZATION
// for cache coherence. Processors are not aware of each other's writes
// until the coherence message propagates through the shared medium.
//
// This is the hardware implementation of the memory consistency model:
// the order in which the bus serializes competing writes defines the
// "global memory order" that all processors observe.
```

解释

MOESI 是 MESI 的扩展，增加了 Owned 状态允许 dirty 数据在缓存间直接传递，

避免了每次 dirty eviction 都写回内存。

两个处理器同时写同一地址这道题的核心考点是：

1. 写操作不是真正"同时"完成的——总线仲裁强制串行化。

2. 失败方必须重试，先自我无效化，再重新发起读-修改-写。

Invalidate 顺序由总线（或 directory）决定，而不是由处理器决定。

这是整个缓存一致性协议的根本：总线是单一序列化点（single point of serialization）。

面试进阶：能区分 snooping bus（广播，适合小系统）和 directory protocol

（点对点，适合大规模多核）的序列化机制。

## Q18. Computer Architecture: Cache Types and Write-Back Policy

QUESTION

(a) What are the different types/levels of cache in a modern

processor? Describe L1, L2, L3 and their typical characteristics.

(b) What is the difference between write-back and write-through

cache policies? What are the tradeoffs?

(c) In a write-back L1 cache:

- When does data get written to the next level (L2/DRAM)?

- What is a "dirty" cache line?

- What happens on a cache eviction of a dirty line?

- What is the risk if power is lost before a dirty line is flushed?

(d) What is write-allocate vs no-write-allocate, and how does

it interact with write-back/write-through?

ANSWER

```
// (a) Cache levels in a modern processor:
//
// L1 Cache (Level 1) -- closest to CPU core:
//   Size:     16KB - 64KB (separate I-cache and D-cache typical)
//   Latency: 1-5 cycles
//   Policy:   Almost always write-back + write-allocate
//   Private: per-core (not shared)
//   Purpose: hide the latency of almost all memory accesses;
//            must be extremely fast -> small size, high associativity
//
// L2 Cache (Level 2):
//   Size:     256KB - 4MB
//   Latency: 5-20 cycles
//   Policy:   Usually write-back
//   Private: often per-core (some designs share between 2 cores)
//   Purpose: catch misses from L1; larger working set coverage
//
// L3 Cache (Last Level Cache, LLC):
//   Size:     4MB - 64MB (server CPUs up to 256MB+)
//   Latency: 20-50 cycles
//   Policy:   Write-back; often inclusive or non-inclusive of L1/L2
//   Shared:   across all cores on the same die
//   Purpose: last on-chip defense before expensive DRAM access;
//            also serves as coherence point in many designs
//
// DRAM (main memory):
//   Size:     GBs to TBs
//   Latency: 50-200 ns (~100-400 cycles at 2GHz+)
//
// Cache organization parameters:
//   Direct-mapped: 1-way, fast but high conflict miss rate
//   N-way set-associative: N candidates per set (e.g. 8-way L1)
//   Fully associative: any line can go anywhere (TLB, victim cache)

// (b) Write-back vs Write-through:

//
// Write-through:
//   Every write to cache ALSO writes to the next level immediately.
//   Cache and memory are ALWAYS in sync.
//   Pros: simpler coherence; no dirty data lost on power failure
//   Cons: every write generates a bus transaction -> high bandwidth;
//         slow for write-intensive workloads
//   Typical use: small L1 in simple embedded MCUs, I/O device registers
//
// Write-back:
//   Writes go to cache ONLY. Memory is updated LAZILY when the
//   cache line is evicted or explicitly flushed.
//   Cache line is marked "dirty" to indicate it differs from memory.
//   Pros: absorbs burst writes locally (write coalescing);
//         dramatically reduces memory bandwidth
//   Cons: more complex; dirty data at risk on power failure;
//         cache coherence is harder (other agents may see stale memory)
//   Typical use: L1, L2, L3 in virtually all modern CPUs
//
// +------------------+--------------------+--------------------+
// | Property         | Write-through      | Write-back         |
// +------------------+--------------------+--------------------+
```

```
// | Memory updated    | Every write        | Only on eviction    |
// | Dirty lines       | None               | Yes                 |
// | Bandwidth         | High               | Low (coalesced)     |
// | Coherence         | Simple             | Complex             |
// | Power-loss risk   | None               | Dirty data lost     |
// | Performance       | Lower (writes)     | Higher              |
// +-----------------+--------------------+--------------------+

// (c) Write-back L1 cache details:
//
// When does data get written to L2/DRAM?
//   ONLY when the dirty cache line is evicted (replaced by a new line)
//   OR when software explicitly flushes (CLFLUSH on x86, DC CIVAC on ARM)

 //   OR during cache coherence (another core requests the line)
 //
 // "Dirty" cache line:
 //   A cache line with a dirty bit set means: the data in this cache
 //   line has been modified by the CPU but the modification has NOT
 //   yet been written back to the next memory level.
 //   The cache copy is the "truth"; the memory copy is stale.
 //
 // On eviction of a dirty line:
 //   1. Cache detects the line to be replaced has dirty bit = 1
 //   2. Cache controller writes the dirty data back to L2/DRAM
 //     (write-back transaction on the bus)
 //   3. The evicted slot is freed, new data is loaded into it
 //   This write-back adds latency to the eviction (not free).
 //   "Write-back buffer" or "victim cache" absorbs this latency.
 //
 // Power-loss risk:
 //   Any dirty line in cache that has NOT been flushed to persistent
 //   storage (DRAM or NVM) is LOST if power fails.
 //   Critical for: file systems (fsync needed), databases (WAL),
 //   safety-critical systems (must flush before power-down).
 //   Solution: cache flush on shutdown, battery-backed SRAM,
 //   or use write-through / non-cacheable mapping for critical data.

 // (d) Write-allocate vs No-write-allocate:
 //
 // Write-allocate (fetch-on-write):
 //   On a write MISS: first load the cache line from memory into cache,
 //   then perform the write in cache. Line is now dirty.
 //   Pro: subsequent reads/writes to the same line hit in cache
 //   Con: one extra memory read on write miss
 //   Paired with: write-back (natural combination)
 //
 // No-write-allocate (write-around):
 //   On a write MISS: write directly to memory/next level,

 //   do NOT load the line into cache.
 //   Pro: avoids polluting cache with data that may not be re-read
 //   Con: subsequent reads to the same address will still miss
 //   Paired with: write-through (natural combination)
 //
 // Standard combinations in practice:
 //   Write-back   + Write-allocate:     modern L1/L2/L3 (default)
 //   Write-through + No-write-allocate: embedded MCU L1, MMIO regions
 //
 // ARM example: memory attribute NORMAL CACHEABLE uses
 //   write-back + write-allocate by default.
 //   DEVICE memory type uses write-through + no-write-allocate
 //   (for peripheral register access -- no caching at all).
```

解释

Cache 层次结构和写策略是计算机体系结构面试的必考题。

三个层次的核心特征：

L1 = 最快最小（<64KB），每核私有，分 I-cache 和 D-cache。

L2 = 中间层（几百KB），私有或半共享，catch L1 miss。

L3 = 最大最慢（几MB-几十MB），全核共享，last resort before DRAM。

Write-back 的核心理解：

Dirty bit 是关键——有 dirty bit 的 cache line 是"缓存中的真相"，

内存里的数据是过时的。只有在 eviction 或显式 flush 时才写回内存。

这让 cache 可以合并多次写操作（write coalescing），大幅节省带宽。

Write-back 的两大风险：

1. 断电丢数据（文件系统必须显式 fsync/flush）

2. 缓存一致性复杂（DMA 和其他 CPU 可能读到内存中的旧数据）

这正是 DMA 验证中 cache flush barrier 至关重要的原因。

## Q19. Analog/Digital: CMOS Inverter with PMOS and NMOS

QUESTION

(a) Draw a CMOS inverter using one PMOS and one NMOS transistor.

Explain how it works.

(b) What happens if PMOS and NMOS are swapped?

Does it still function as an inverter?

(c) In the swapped circuit (which becomes a buffer), what if you

use a single larger PMOS/NMOS instead of multiple inverters

in series? Which is better and why?

ANSWER

```
// (a) CMOS Inverter
//
// Circuit:
//
//       VDD
//        |
//       [P]    <- PMOS   (gate = IN, source = VDD, drain = OUT)
//        |
//       OUT
//        |
//       [N]    <- NMOS   (gate = IN, drain = OUT, source = GND)
//        |
//       GND
//
// PMOS: gate connected to IN, source to VDD, drain to OUT
// NMOS: gate connected to IN, drain to OUT, source to GND
// Both gates tied together at IN.
//
```

```
// Operation:
//   IN = 0 (logic LOW):
//     PMOS: Vgs = 0 - VDD = -VDD     -> |Vgs| > |Vtp| -> PMOS ON
//     NMOS: Vgs = 0                  -> Vgs < Vtn       -> NMOS OFF
//     OUT connected to VDD through PMOS     -> OUT = HIGH (logic 1)
//
//   IN = 1 (logic HIGH = VDD):
//     PMOS: Vgs = VDD - VDD = 0      -> |Vgs| < |Vtp| -> PMOS OFF
//     NMOS: Vgs = VDD                -> Vgs > Vtn       -> NMOS ON
//     OUT connected to GND through NMOS -> OUT = LOW (logic 0)
//
// Truth table:    IN=0 -> OUT=1,   IN=1 -> OUT=0    => INVERTER

// (b) What if PMOS and NMOS are swapped?
//
// Swapped circuit:
//

//       VDD
//        |
//       [N]   <- NMOS   (gate = IN, source = VDD, drain = OUT)
//        |
//       OUT
//        |
//       [P]   <- PMOS   (gate = IN, drain = OUT, source = GND)
//        |
//       GND
//
//   IN = 0:
//     NMOS: Vgs = 0 - VDD = -VDD -> NMOS OFF (needs positive Vgs)
//     PMOS: Vgs = 0 - 0 = 0        -> PMOS OFF (needs negative Vgs)
//     Both OFF -> OUT floating    (undefined / high-Z)
//
//   IN = 1 (VDD):
//     NMOS: Vgs = VDD - VDD = 0    -> NMOS OFF
//     PMOS: Vgs = VDD - 0 = VDD    -> PMOS OFF (Vgs > 0, PMOS needs Vgs < Vtp < 0)
//     Both still OFF -> OUT floating
//
//   Actually the swapped version doesn't cleanly work as either
//   inverter or buffer. The circuit is dysfunctional in standard CMOS.
//
//   What is sometimes discussed: using NMOS as pull-up (source-follower)
//   and PMOS as pull-down -- this creates a pseudo-buffer with serious
//   issues: the NMOS pull-up loses Vtn voltage drop (output never reaches VDD),
//   and PMOS pull-down similarly cannot pull to GND properly.
//   The output is degraded (doesn't reach full rail voltages).
//   The function is a weak, degraded non-inverting path -- NOT a proper buffer.

// (c) Larger PMOS/NMOS (degraded buffer) vs. series inverter chain
//
// Option A: Single PMOS-only pull-up (or NMOS-only) made larger
//   Increasing transistor width (W) reduces resistance -> faster charge/discharge.
//   BUT: in the swapped/degraded circuit, the fundamental issue is

  //   INCOMPLETE charging/discharging of the output node.
  //   A larger transistor can charge faster but the OUTPUT VOLTAGE
  //   never cleanly reaches VDD or GND:
  //       NMOS source follower: Vout_max = VDD - Vtn         (missing a Vtn)
  //       PMOS source follower: Vout_min = |Vtp|              (above GND)
  //   This creates UNDEFINED INTERMEDIATE VOLTAGES at the output.
  //   The downstream logic that reads this output may see neither
  //   a valid '1' nor a valid '0' -- it is in the forbidden zone.
  //   Larger area only speeds up reaching the wrong (undefined) value.
```

```
//
// Option B: Proper inverter chain (buffer = even number of inverters)
//   Each inverter fully charges/discharges its output to VDD or GND.
//   The delay per stage is the time to fully switch the output rail.
//   Worst case: you sample the previous valid logic value (old data).
//   But the output is ALWAYS a valid logic level -- never undefined.
//   Multiple inverters add latency but guarantee correct logic levels.
//
// Conclusion: inverter chain buffer is ALWAYS better.
//   The degraded single-transistor "buffer" risks undefined output voltages.
//   In digital design, an unknown intermediate voltage propagating
//   through gates is catastrophic -- causes X-propagation and
//   unpredictable behavior in all downstream logic.
//   The inverter chain may be slower in some cases but is CORRECT.
```

解释

CMOS inverter 的核心是 PMOS 和 NMOS 的互补特性：

PMOS 在 gate 为低时导通（连接 VDD），NMOS 在 gate 为高时导通（连接 GND）。

两个晶体管永远不会同时导通（理想情况），因此没有静态功耗。

交换后的关键问题不是速度，而是电压不确定性（undefined logic level）：

NMOS 做上拉时最高只能拉到 VDD-Vtn，PMOS 做下拉最低只能拉到 |Vtp|，

输出电压落在禁止区间，下游逻辑无法正确采样。

加大晶体管面积只是让电路更快地到达错误的电压，并不解决根本问题。

反相器串联的 buffer 保证每一级都完整充放电到电源轨，

最坏情况是采样到旧值，而不是 undefined 值，这是本质区别。

## Q20. Analog Circuit: RC Low-Pass Filter

QUESTION

You are given a circuit with a resistor R in series with a

capacitor C. The output voltage Vout is measured across the

capacitor (from one end of C to ground).

Circuit:

Vin --[R]--+-- Vout

|

[C]

|

GND

(a) What is this circuit commonly used for?

(b) What is the transfer function H(f) = Vout/Vin?

(c) What happens to Vout as frequency increases?

(d) What is the cutoff frequency fc, and what is Vout/Vin at fc?

ANSWER

```
// (a) This is an RC Low-Pass Filter (LPF).
 //
 // Common uses:
 //   - Noise filtering: attenuate high-frequency noise on power supply
 //      or signal lines (e.g. bypass/decoupling capacitor with trace R)
 //   - Anti-aliasing filter before ADC: remove frequencies above Nyquist
 //   - Smoothing: convert PWM signal to DC analog voltage
 //   - Signal integration: approximate integrator for slow input signals
 //   - Time constant circuit: RC timer, delay element
 //
 // In chip design context:
 //   Every wire has parasitic R and C. Long interconnects form
 //   unintentional RC low-pass filters that limit signal bandwidth
 //   and cause propagation delay (Elmore delay model).

 // (b) Transfer function H(f):
 //
 // Impedance of capacitor: Zc = 1 / (j * 2*pi*f * C)
 // Voltage divider: Vout = Vin * Zc / (R + Zc)
 //
 //   H(f) = Vout/Vin = (1/jwC) / (R + 1/jwC)
 //                    = 1 / (1 + jwRC)
 //
 //   where w = 2*pi*f (angular frequency)
 //
 //   Magnitude: |H(f)| = 1 / sqrt(1 + (2*pi*f*RC)^2)
 //   Phase:         ang(H) = -arctan(2*pi*f*RC)

 // (c) Behavior as frequency increases:
 //
 //   Low frequency (f -> 0):
 //      Capacitor impedance -> infinity (open circuit)
 //      Vout ~= Vin    -> |H| ~= 1   (signal passes through)
 //
 //   High frequency (f -> infinity):

 //        Capacitor impedance -> 0 (short circuit to GND)
 //        Vout -> 0   -> |H| -> 0   (signal blocked)
 //
 //    This is why it is called a LOW-PASS filter:
 //    low frequencies pass, high frequencies are attenuated.

 // (d) Cutoff frequency fc and magnitude at fc:
 //
 //    fc = 1 / (2 * pi * R * C)      [Hz]
 //
 //    At f = fc:
 //        |H(fc)| = 1 / sqrt(1 + 1) = 1 / sqrt(2) ~= 0.707
 //        In dB: 20*log10(0.707) = -3 dB
 //
 //    fc is called the "-3dB frequency" or "corner frequency".
 //    At fc, the output power is half the input power.
 //
 // Example: R=1kOhm, C=1nF -> fc = 1/(2*pi*1e3*1e-9) ~= 159 kHz
 //    Signals below 159kHz pass through; above 159kHz are attenuated.
```

解释

RC 低通滤波器是模拟电路最基础的滤波结构。

核心直觉：电容对高频是低阻抗（短路到 GND），对低频是高阻抗（开路）。

所以高频信号被电容"旁路"到地，低频信号顺利通过到 Vout。

截止频率 fc = 1/(2πRC)：这是信号幅度降到原来 1/√2 ≈ 70.7%（-3dB）的频率。

在芯片设计中，寄生 RC 滤波效应是高速信号完整性的核心问题：

互连线的寄生 R 和 C 构成低通滤波器，限制了信号的上升时间和带宽，

这正是 Elmore delay 模型和 SI 分析要解决的问题。

## Q21. Digital Design: 2-to-1 MUX Using Only NAND Gates

QUESTION

(c) Implement a 2-to-1 multiplexer using only 2-input NAND gates.

Show the derivation from the boolean expression step by step.

Verify your circuit for both S=0 and S=1.

ANSWER

```
// 2-to-1 MUX using only NAND gates
  //
   // Boolean expression for 2-to-1 MUX:
   //      Y = A*~S + B*S      (S=0 selects A, S=1 selects B)
   //
   // Derivation using NAND (remember: NAND(x,y) = ~(x AND y))
   //
   // Step 1: Express using NAND algebra:
   //      Y = ~(~(A*~S)) * ~(~(B*S))          -- double negation
   //       = ~(~(A & ~S) & ~(B & S))           -- De Morgan
   //       = NAND( NAND(A, ~S), NAND(B, S) )
   //
   // Step 2: ~S itself needs a NAND gate (NAND(S,S) = ~S):
   //      ~S = NAND(S, S)
   //
   // Full NAND-only implementation (4 NAND gates):
   //
   //      Gate 1: N1 = NAND(S, S)        -> output = ~S
   //      Gate 2: N2 = NAND(A, N1)       -> output = ~(A & ~S)
   //      Gate 3: N3 = NAND(B, S)        -> output = ~(B & S)
   //      Gate 4: N4 = NAND(N2, N3)      -> output = ~(~(A&~S) & ~(B&S))
   //                                                = A&~S | B&S   = MUX(S,A,B)
   //
   // Circuit diagram:
   //
   //      S ---+---[NAND]--- N1(=~S)
   //           |     |

   //           |   A-+--[NAND]--- N2 --+
   //           |                       |
   //           |                     [NAND]--- Y (MUX output)
   //           |                       |
   //           +---B---[NAND]--- N3 --+
   //
   // Verification:
   //      S=0: N1=1, N2=NAND(A,1)=~A, N3=NAND(B,0)=1
   //           Y = NAND(~A, 1) = ~~A = A      -> correct (S=0 selects A)
   //      S=1: N1=0, N2=NAND(A,0)=1, N3=NAND(B,1)=~B
   //           Y = NAND(1, ~B) = ~~B = B      -> correct (S=1 selects B)
   //
   // Total: 4 NAND gates (including the inverter for ~S)
```

解释

(a) array.xor() 是 SV 的数组归约（reduction）函数，

计算所有元素的 XOR，结果为1当且仅当1的个数为奇数，

这是检测奇偶性最简洁的写法，比 $countones(array) % 2 == 1 更优雅。

(b) 动态数组 [] 和固定数组 [N-1:0] 的核心区别：

固定数组的大小是类型的一部分，不需要约束；

动态数组的大小是运行时确定的，必须通过 c_size 约束，

否则 solver 不知道该分配多少元素，randomize() 会失败。

解释

NAND 实现 MUX 的关键步骤：

先写出 MUX 的布尔表达式 Y = A·S' + B·S，

然后对整体取双重否定，再用 De Morgan 定律展开成纯 NAND 形式。

4个 NAND 门的构成：1个做 NOT(S)，2个做中间项，1个做最终 NAND。

验证时分 S=0 和 S=1 两种情况逐步追踪信号值，确认输出正确。

## Q22. Verilog: Sliding Window -- Count 1s in Last 100 Bits > 50

QUESTION

Each clock cycle, a 1-bit input 'din' arrives.

Design a circuit that checks whether the number of 1s

in the most recent 100 bits (sliding window) is greater than 50.

Output 'majority' should be HIGH when count > 50.

ANSWER

```
module sliding_window_majority (
  input  logic clk,
  input  logic rst_n,
  input  logic din,
  output logic majority    // HIGH when number of 1s in last 100 > 50
);
  // Shift register: stores last 100 bits
  logic [99:0] shift_reg;

  // Running count of 1s in the window
  // Range: 0-100, needs 7 bits
  logic [6:0] count;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      shift_reg <= '0;
      count     <= '0;
    end else begin
      // Update count: add new bit, subtract bit being shifted out
      // shift_reg[99] is the oldest bit (about to leave the window)
      count     <= count + din - shift_reg[99];
      // Shift in new bit at LSB, oldest bit exits at MSB
      shift_reg <= {shift_reg[98:0], din};
    end
  end
```

```
  // Combinational output
  assign majority = (count > 7'd50);

endmodule

// Explanation of count update:
//   New window = old window shifted left + din
//   Bit leaving: shift_reg[99] (the oldest)
//   Bit entering: din (the newest)
//   count_new = count_old - shift_reg[99] + din
//
// Example waveform:
//   Cycle 0: shift_reg = 100'b0, count = 0
//   Cycle 1: din=1, shift_reg[99]=0 -> count = 0 - 0 + 1 = 1
//   Cycle 2: din=1, shift_reg[99]=0 -> count = 1 - 0 + 1 = 2
//   ...after 51 consecutive 1s: count = 51, majority = 1
//
// Key design choices:
//   Shift register: O(100) flip-flops, simple and exact
//   Incremental count update avoids O(100) adder each cycle
//   count needs 7 bits: ceil(log2(101)) = 7
//   Output is purely combinational (no extra register needed)
```

解释

滑动窗口计数是数字设计常考题，关键在于增量更新计数器。

每个周期只有两件事发生：最老的 bit（shift_reg[99]）离开窗口，

最新的 bit（din）进入窗口。

所以 count_new = count_old - oldest + newest，只需一个加减运算。

如果每周期重新数所有 100 位，需要 100 输入加法器，面积大且时序差。

计数器位宽：0 到 100 共 101 个状态，需要 ceil(log2(101)) = 7 位。

移位寄存器用 {shift_reg[98:0], din} 一行搞定，注意 MSB 是最老的位。

## Q23. Digital Design: 2s Complement Bit Width for A-B

QUESTION

Given:

A: range [-175, +101] (signed integer)

B: range [-27, +81] (signed integer)

C = A - B

In 2s complement representation:

(a) How many bits does A need?

(b) How many bits does B need?

(c) How many bits does C need?

Show your work.

ANSWER

```
// 2s complement bit width formula:
// To represent range [min, max], need N bits where:
//   N = ceil(log2(max(|min|, max) + 1)) + 1
```

```
//         ^--- magnitude bits               ^--- sign bit
// Equivalently: N bits can represent [-2^(N-1), 2^(N-1)-1]
// So we need: 2^(N-1) > |min|  AND  2^(N-1) > max
// i.e. 2^(N-1) >= max(|min|, max) + 1
//      N-1 >= ceil(log2(max(|min|, max+1)))
//      N   >= ceil(log2(max(|min|, max+1))) + 1

// (a) Bit width of A: range [-175, +101]
//
//   |min| = |-175| = 175
//   max   = 101
//   dominant magnitude = max(175, 101) = 175
//
//   Need 2^(N-1) >= 175+1 = 176
//   2^7 = 128 < 176  -> 7 magnitude bits not enough
//   2^8 = 256 >= 176 -> 8 magnitude bits sufficient
//   N = 8 + 1 = 9 bits
//
//   Verify: 9-bit 2s complement range = [-256, +255]
//   -175 >= -256 (OK) and +101 <= +255 (OK)
//   A needs 9 bits.

// (b) Bit width of B: range [-27, +81]
//
//   |min| = |-27| = 27
//   max   = 81
//   dominant magnitude = max(27, 81) = 81
//
//   Need 2^(N-1) >= 81+1 = 82
//   2^6 = 64  < 82  -> not enough
//   2^7 = 128 >= 82 -> sufficient
//   N = 7 + 1 = 8 bits  -- wait, check: 7-bit magnitude = 8-bit signed
//   Actually: 2^(N-1) >= 82, so N-1 >= 7, N >= 8
//   Verify: 8-bit 2s complement range = [-128, +127]
//   -27 >= -128 (OK) and +81 <= +127 (OK)
//   B needs 8 bits (not 7 -- a common mistake to forget the sign bit).

// (c) Bit width of C = A - B
//
//   C_max = A_max - B_min = 101 - (-27) = 101 + 27 = 128
//   C_min = A_min - B_max = -175 - 81   = -256
//
//   C range: [-256, +128]
//   |C_min| = 256, C_max = 128
//   dominant = max(256, 128) = 256
//
//   Need 2^(N-1) >= 256+1 = 257
//   2^8 = 256 < 257  -> 8 magnitude bits not enough (just barely!)
//   2^9 = 512 >= 257 -> 9 magnitude bits sufficient
//   N = 9 + 1 = 10 bits
//
//   Verify: 10-bit range = [-512, +512-1=511]
//   -256 >= -512 (OK) and +128 <= +511 (OK)
//   C needs 10 bits.
//
// Summary:
//   A: 9 bits  (range [-256, +255], actual [-175, +101])
//   B: 8 bits  (range [-128, +127], actual [-27,  +81])
//   C: 10 bits (range [-512, +511], actual [-256, +128])
//
// Note: C_min = -256 requires 2^(N-1) >= 257, NOT >= 256.
// This is the critical trap: -256 in 2s complement needs 9 magnitude bits
// because -2^8 = -256 fits in 9 bits, but +256 would NOT fit.
```

```
// The asymmetry of 2s complement is the key insight here.
```

解释

这道题考察 2s complement 位宽计算，最常见的陷阱有两个：

1. 忘记符号位：幅度需要 N 位时，总共需要 N+1 位（含符号位）。

2. 负数的临界情况：-256 = -2^8 可以用 9 位表示，

因为 9 位 2s complement 的最小值正好是 -2^8 = -256。

但 C_min = -256 要求 2^(N-1) >= 257（不是 >= 256），

所以需要 N-1 = 9 即 N = 10 位，而不是 9 位。

计算 C 的范围：最大值 = A_max - B_min，最小值 = A_min - B_max。

记住减去最小值（最负的数）会使结果最大，减去最大值会使结果最小。

## Q24. Digital Design: Divide-by-3 Clock with 50% Duty Cycle

QUESTION

Design a divide-by-3 clock divider with exactly 50% duty cycle.

Use the inverted clock (negedge) technique.

The input clock is 'clk'. Output is 'clk_div3'.

ANSWER

```
// Problem with naive divide-by-3:
// A simple 3-state counter toggling output gives 1-HIGH / 2-LOW
// or 2-HIGH / 1-LOW per period -> duty cycle = 33% or 67%, NOT 50%.
//
// Solution: use BOTH posedge and negedge flip-flops and OR the outputs.
//
// Technique:
//   1. Create signal A: toggles on posedge clk, period = 3 clk cycles
//      A goes HIGH for 1.5 cycles (using posedge logic)
//   2. Create signal B: same counter but sampled on NEGEDGE clk
//      B is A delayed by half a clock cycle
//   3. clk_div3 = A | B  -> 50% duty cycle at 1/3 frequency
//
// Counter-based implementation:
module clk_div3_50 (
  input  logic clk,
  input  logic rst_n,
  output logic clk_div3
);
  logic [1:0] cnt_pos;    // counter on posedge
  logic [1:0] cnt_neg;    // counter on negedge (half-cycle delayed)
  logic       clk_pos;    // output from posedge FF
  logic       clk_neg;    // output from negedge FF

  // Posedge counter: 0 -> 1 -> 2 -> 0 -> ...
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      cnt_pos <= 2'd0;
      clk_pos <= 1'b0;
    end else begin
      if (cnt_pos == 2'd2) cnt_pos <= 2'd0;
      else                 cnt_pos <= cnt_pos + 1;
```

```
      // HIGH for cnt_pos == 0 only (1 out of 3 posedges)
      clk_pos <= (cnt_pos == 2'd0);
    end
  end

  // Negedge counter: same logic, triggered on falling edge
  // This gives clk_pos delayed by exactly half a clock cycle
  always_ff @(negedge clk or negedge rst_n) begin
    if (!rst_n) begin
      cnt_neg <= 2'd0;
      clk_neg <= 1'b0;
    end else begin
      if (cnt_neg == 2'd2) cnt_neg <= 2'd0;
      else                 cnt_neg <= cnt_neg + 1;
      clk_neg <= (cnt_neg == 2'd0);
    end
  end

  // OR the two half-cycle offset signals -> 50% duty cycle
  assign clk_div3 = clk_pos | clk_neg;

endmodule

// Waveform illustration (clk period = T):
//
//  clk:     _|^|_|^|_|^|_|^|_|^|_|^|_|^|_
//  clk_pos: _____|^^^|_________|^^^|___   (HIGH for 1 clk cycle)
//  clk_neg: _________|^^^|_________|^^^|   (clk_pos delayed T/2)
//  clk_div3:_____|^^^^^^^|_______|^^^^^^^  (HIGH for 1.5 T = 50% of 3T)
//
// clk_div3 period = 3T, HIGH time = 1.5T -> duty cycle = 50%
```

解释

三分频 50% 占空比是数字设计经典面试题。

核心思路：奇数分频无法用单边沿实现 50% 占空比，

因为 50% 意味着高电平持续 1.5 个时钟周期，必须利用半个周期。

解决方案：同一套计数逻辑分别在上升沿和下降沿各做一次，

两路输出相差半个周期，OR 之后高电平正好是 1.5 个周期。

这个技巧适用于所有奇数分频（3、5、7...）的 50% 占空比设计。

需要注意：negedge 触发的 FF 和 posedge 触发的 FF 同时存在，

综合工具需要正确处理，STA 也需要考虑 hold time 的问题。

## Q25. Digital Design: FIFO Full and Empty Detection

QUESTION

How do you detect FULL and EMPTY conditions in a synchronous FIFO?

(a) Using a counter (entry count) approach.

(b) Using read/write pointer comparison approach.

(c) What is the extra bit trick for pointer-based detection?

(d) For an asynchronous FIFO (different read/write clocks),

what additional technique is required?

ANSWER

```
// (a) Counter-based approach (synchronous FIFO)
//
// Maintain a count of valid entries in the FIFO.
// Increment on write (and not read), decrement on read (and not write).
//
//   logic [$clog2(DEPTH):0] count;  // one extra bit for DEPTH itself
//
//   always_ff @(posedge clk) begin
//     if (wr_en && !rd_en && !full)  count <= count + 1;
//     if (rd_en && !wr_en && !empty) count <= count - 1;
//     // simultaneous read+write: count unchanged
//   end
//
//   assign empty = (count == 0);
//   assign full  = (count == DEPTH);
//
// Pros: simple logic, easy to understand
// Cons: extra adder logic, count signal is the critical path

// (b) Read/write pointer comparison (synchronous FIFO)
//
// Use read pointer (rd_ptr) and write pointer (wr_ptr).
// Both pointers are modulo DEPTH (wrap around).
//
//   EMPTY: rd_ptr == wr_ptr
//          (read has caught up to write -- nothing to read)
//
//   FULL:  wr_ptr + 1 == rd_ptr  (next write would hit read position)
//          i.e. the FIFO has DEPTH entries, one slot "wasted" as sentinel
//
// This wastes one entry (can store DEPTH-1 entries in a DEPTH-slot FIFO)
// to avoid the ambiguity where full and empty both look like ptr equality.

// (c) Extra bit trick (pointer approach, no wasted entry)
//
// Extend pointers by ONE extra bit beyond the address width.
// Pointer width = $clog2(DEPTH) + 1
//
//   EMPTY: rd_ptr == wr_ptr
//          (all bits equal, including the extra MSB)
//
//   FULL:  rd_ptr[MSB] != wr_ptr[MSB]   -- extra bits differ
//          AND
//          rd_ptr[ADDR-1:0] == wr_ptr[ADDR-1:0]  -- lower bits equal
//          Simplified: full = (wr_ptr == {~rd_ptr[MSB], rd_ptr[ADDR-1:0]})
//
// How it works:
//   Write pointer laps the read pointer when FIFO is full.
//   After one full wrap, the MSB of wr_ptr flips while rd_ptr MSB hasn't.
//   Empty: both MSBs same + lower bits same (no lap has occurred).
//   Full:  MSBs differ + lower bits same (wr_ptr has lapped rd_ptr once).
//
// Pros: uses full DEPTH entries (no wasted slot)
//       clean FULL/EMPTY logic
// Cons: slightly more complex pointer logic

// (d) Asynchronous FIFO: Gray code pointers + synchronizers
//
// In async FIFO, rd_ptr is in the read clock domain and
// wr_ptr is in the write clock domain.
// Directly comparing them across clock domains causes metastability.
//
```

```
// Solution: convert pointers to Gray code before crossing clock domains.
//   Gray code: only ONE bit changes between consecutive values.
//   This means a metastable sample can only cause an off-by-one error
//   (the pointer appears one step behind its true value), which is safe.
//
//   Step 1: Convert wr_ptr to Gray code in write domain.
//   Step 2: Synchronize Gray-coded wr_ptr to read clock (2-FF synchronizer).
//   Step 3: Convert synchronized Gray code back to binary in read domain.
//   Step 4: Compare with rd_ptr for empty/full detection.
//
//   Binary to Gray:  gray = binary ^ (binary >> 1)
//   Gray to Binary:  binary[N-1] = gray[N-1];
//                    for i = N-2 downto 0: binary[i] = binary[i+1] ^ gray[i]
//
//   EMPTY (in read domain):  rd_ptr_gray == wr_ptr_gray_sync
//   FULL  (in write domain): wr_ptr_gray == {~rd_ptr_gray_sync[MSB:MSB-1],
//                                            rd_ptr_gray_sync[MSB-2:0]}
```

解释

FIFO 空满检测是芯片设计面试必考题，有两类主要方法：

计数器法：直观简单，但需要额外的加法器，时序压力较大。

指针比较法：面积小，时序好，但需要处理空满判断的歧义。

额外比特法（extra bit trick）：最优雅的方案，不浪费任何存储槽，

空满条件通过额外比特的异同来区分：

空 = 所有位相同，满 = MSB 不同 + 低位相同。

异步 FIFO 的核心挑战：跨时钟域的指针比较需要 Gray 码，

因为 Gray 码每次只变化 1 位，即使发生亚稳态也只会造成

"慢一拍"的误判，不会出现多位同时跳变导致的错误地址。

## Q26. Digital Design: Synchronous vs Asynchronous Reset -- When to Use Each

QUESTION

Compare synchronous reset and asynchronous reset in digital design.

(a) How does each work? Write example RTL for both.

(b) What are the pros and cons of each?

(c) When would you choose synchronous reset?

(d) When would you choose asynchronous reset?

(e) What is async-assert sync-deassert reset, and why is it preferred

in most production chips?

ANSWER

```
// (a) RTL examples:

// Synchronous reset: reset only takes effect on clock edge
always_ff @(posedge clk) begin
  if (rst_n == 1'b0)    // sampled at posedge clk
    q <= 1'b0;
  else
    q <= d;
```

```
end

// Asynchronous reset: reset takes effect immediately, independent of clk
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n)           // triggers as soon as rst_n falls
    q <= 1'b0;
  else
    q <= d;
end

// (b) Pros and cons:
//
// Synchronous Reset:
//   Pros:
//     - Reset is glitch-free: short pulses on rst shorter than one clk
//       period are filtered out (must be asserted for a full cycle)
//     - Easier static timing analysis: reset path is a normal data path
//     - No special reset recovery/removal timing constraints needed
//     - Synthesis tools handle it like any other mux
//   Cons:
//     - Requires clock to be running to reset: if clk stops or is gated,
//       reset cannot propagate
//     - Slightly more logic: a mux is needed in front of the FF D input
//     - Longer reset pulse required (must be >= 1 clock cycle)
//
// Asynchronous Reset:
//   Pros:
//     - Works even when clock is stopped or gated
//     - Can reset the chip instantly, faster response
//     - Slightly less logic on the data path (no input mux)
//     - Standard for power-on reset: clock may not be stable yet
//   Cons:
//     - Reset deassert (release) can cause metastability if it
//       occurs near a clock edge (reset removal violation)
//     - Glitches on reset signal can cause spurious resets
//     - More complex STA: requires reset recovery/removal checks

// (c) When to choose synchronous reset:
//     - Design operates in a well-defined clock environment
//     - Protection against reset glitches is important
//     - Scan/DFT: synchronous reset is easier to control during test
//     - FPGA designs: most FPGA FFs have synchronous reset support
//     - Logic that must be deterministic about reset timing relative to data
//     - Datapath logic where reset must align with protocol state machines

// (d) When to choose asynchronous reset:
//     - Power-on reset (POR): clock may not be running yet
//     - Clock domain crossing: each domain may need its own reset
//       that works independently of other clocks
//     - Safety-critical logic: must reset immediately regardless of clock
//     - Memory controllers, PLLs, analog IP where clk isn't available

// (e) Async-assert, Sync-deassert (recommended best practice):
//
//   Assert (activate) reset ASYNCHRONOUSLY: immediate effect, works
//   even without a clock (handles power-on, clock failure scenarios).
//
//   Deassert (release) reset SYNCHRONOUSLY: synchronize the reset
//   release to the clock edge to avoid metastability.
//
//   Implementation: pass the reset release through a 2-FF synchronizer.
//   rst_sync[0] and rst_sync[1] are both asynchronously set to 0,
//   but deassert (go back to 1) only on clock edges.
```

```
//
always_ff @(posedge clk or negedge rst_n_async) begin
  if (!rst_n_async) begin
    rst_sync <= 2'b00;   // async assert: both FFs go to reset immediately
  end else begin
    rst_sync <= {rst_sync[0], 1'b1};  // sync deassert: shifts in 1
  end
end
// rst_n_sync = rst_sync[1]: clean synchronized reset for the rest of the design
//
// Why this is preferred:
//   Assert is immediate (handles power-on, glitches) -- asynchronous benefit
//   Deassert is synchronous (no metastability at reset release) -- sync benefit
//   Most industry standard cell libraries and reset controllers use this scheme
```

解释

同步复位 vs 异步复位是每个数字设计面试必考的基础题。

记忆口诀：

同步复位：需要时钟才能复位，干净无毛刺，STA 友好。

异步复位：不需要时钟，即时生效，适合上电复位。

最佳实践 async-assert sync-deassert：

两者取长补短——拉低（assert）异步立即生效，

释放（deassert）同步到时钟边沿，避免亚稳态。

这几乎是所有量产芯片的标准做法。

面试加分点：能提到 reset removal/recovery timing constraint，

以及 FPGA 上同步复位更常见（Xilinx/Intel FPGA 的 FF 原生支持同步复位）。

## Q27. Digital Design: Setup/Hold Violation Analysis and Checker

QUESTION

Given:

Clock period = 200ps

First rising edge occurs at t = 100ps

Signal D falls LOW at t = 150ps

Setup time requirement (T_setup) = 150ps

Hold time requirement (T_hold) = 100ps

(a) Is there a setup violation? Is there a hold violation?

Show your work.

(b) Write a simulation checker (using always blocks and $stime)

to detect setup violations and hold violations.

ANSWER

```
// (a) Violation analysis:
//
// Clock rising edge at t = 100ps.
// D falls at t = 150ps.
```

```
//
// Setup violation check:
//   Setup window: D must be stable BEFORE the rising edge.
//   Setup window = [rising_edge - T_setup, rising_edge]
//                = [100 - 150, 100] = [-50ps, 100ps]
//   D changes at 150ps, which is AFTER the rising edge (100ps).
//   D was stable before and at the clock edge -> NO setup violation.
//
// Hold violation check:
//   Hold window: D must stay stable AFTER the rising edge.
//   Hold window = [rising_edge, rising_edge + T_hold]
//               = [100ps, 100 + 100] = [100ps, 200ps]
//   D changes at 150ps, which is INSIDE [100ps, 200ps].
//   150ps - 100ps = 50ps < T_hold = 100ps
//   -> HOLD VIOLATION: D changed only 50ps after the rising edge,
//      but it must hold stable for at least 100ps.
//
// Summary: NO setup violation, YES hold violation.

// (b) Simulation checkers using $stime:
//
// Setup violation checker:
// D must not change within T_setup before a rising clock edge.
// Equivalently: at each rising edge, the last D transition must have
// occurred at least T_setup time units ago.

// Setup checker
longint time_D_last_change = 0;  // timestamp of last D transition

always @(D) begin
  time_D_last_change = $stime();  // record when D last changed
end

always @(posedge clk) begin
  // At rising edge: check that D was stable for at least T_setup
  if (($stime() - time_D_last_change) < T_setup) begin
    `uvm_error("TIMING", $sformatf(
      "Setup violation at t=%0t: D changed %0t ago, need %0t",
      $stime(), $stime()-time_D_last_change, longint'(T_setup)))
  end
end

// Hold violation checker:
// After each rising clock edge, D must not change for T_hold time.

longint time_clk_rise = 0;  // timestamp of last rising edge

always @(posedge clk) begin
  time_clk_rise = $stime();  // record rising edge time
end

always @(D) begin
  // D changed: check that it has been at least T_hold since last rising edge
  if (($stime() - time_clk_rise) < T_hold) begin
    `uvm_error("TIMING", $sformatf(
      "Hold violation at t=%0t: D changed %0t after rising edge, need %0t",
      $stime(), $stime()-time_clk_rise, longint'(T_hold)))
  end
end

// SVA-based alternative (cleaner for cycle-accurate checks):
// For setup: signal must be stable 1 cycle before capture
property p_setup;
```

```
  @(posedge clk) $stable(D);  // D must not have changed this cycle
endproperty
// (SVA $stable checks no transition in the current time step -- approximate)
//
// For precise ps-level checks, the $stime-based always block approach
// above is more accurate than cycle-level SVA.
```

解释

Setup/Hold 违例分析的关键是时序窗口的计算：

Setup 窗口：时钟上升沿之前 T_setup 时间内，D 不能变化。

Hold 窗口：时钟上升沿之后 T_hold 时间内，D 不能变化。

本题：D 在上升沿（100ps）之后 50ps 才变化，50ps < T_hold(100ps)，

因此发生了 hold violation。D 在上升沿之前没有变化，所以没有 setup violation。

Checker 设计思路：

Setup checker：记录 D 最后一次变化的时间，在每个上升沿检查间隔是否 >= T_setup。

Hold checker：记录每个上升沿的时间，在 D 每次变化时检查间隔是否 >= T_hold。

两者互为"镜像"：setup 在时钟边沿检查，hold 在 D 变化时检查。

## Q28. Digital Design: FSM for a Digital Lock with RTL and Assertions

QUESTION

Design a digital combination lock with:

- 10 digit inputs (0-9), one-hot encoded: digit[9:0]

- A reset button (active-high async reset)

- A lock button: pressing LOCK when the correct 4-digit sequence

has been entered unlocks the lock; any error resets to IDLE

Assume the correct combination is 4-7-2-9.

(a) Draw the FSM state diagram and list the states.

(b) Write the RTL in SystemVerilog (Moore machine).

(c) Write SVA assertions to verify:

i. Wrong digit always returns to IDLE

ii. UNLOCK state is only reachable via the correct sequence

iii.UNLOCK is maintained until reset

ANSWER

```
// (a) FSM States:
//
//   IDLE  -> (digit==4) -> S1    : waiting for first digit
//   S1    -> (digit==7) -> S2    : got '4', waiting for '7'
//   S1    -> (digit!=7) -> IDLE  : wrong digit, go back
//   S2    -> (digit==2) -> S3    : got '4','7', waiting for '2'
//   S2    -> (digit!=2) -> IDLE
//   S3    -> (digit==9) -> S4    : got '4','7','2', waiting for '9'
//   S3    -> (digit!=9) -> IDLE
//   S4    -> (lock_btn) -> UNLOCK: correct! unlock
```

```
//   S4    -> (!lock_btn)-> S4   : wait for lock press
//   S4    -> (digit==any)-> IDLE: new digit invalidates entry
//   UNLOCK -> (reset)   -> IDLE : locked again on reset only

// (b) RTL Implementation:
module digital_lock (
  input  logic       clk,
  input  logic       rst,        // async reset, active-high
  input  logic [9:0] digit,      // one-hot: digit[i] = key i pressed
  input  logic       digit_valid,// pulse: new digit entered this cycle
  input  logic       lock_btn,   // pulse: lock button pressed
  output logic       unlocked    // 1 = lock is open
);
  typedef enum logic [2:0] {
    IDLE   = 3'd0,
    S1     = 3'd1,  // received '4'
    S2     = 3'd2,  // received '4','7'
    S3     = 3'd3,  // received '4','7','2'
    S4     = 3'd4,  // received '4','7','2','9' -- awaiting lock_btn
    UNLOCK = 3'd5   // unlocked
  } state_t;

  state_t state, next_state;

  // Correct combination: 4-7-2-9
  localparam bit [9:0] D4 = 10'b0000010000;  // digit[4]
  localparam bit [9:0] D7 = 10'b0010000000;  // digit[7]
  localparam bit [9:0] D2 = 10'b0000000100;  // digit[2]
  localparam bit [9:0] D9 = 10'b1000000000;  // digit[9]

  // State register
  always_ff @(posedge clk or posedge rst) begin
    if (rst) state <= IDLE;
    else     state <= next_state;
  end

  // Next state logic (combinational)
  always_comb begin
    next_state = state;  // default: stay
    unique case (state)
      IDLE: begin
        if (digit_valid)
          next_state = (digit == D4) ? S1 : IDLE;
      end
      S1: begin
        if (digit_valid)
          next_state = (digit == D7) ? S2 : IDLE;
      end
      S2: begin
        if (digit_valid)
          next_state = (digit == D2) ? S3 : IDLE;
      end
      S3: begin
        if (digit_valid)
          next_state = (digit == D9) ? S4 : IDLE;
      end
      S4: begin
        if (lock_btn)   next_state = UNLOCK;
        else if (digit_valid) next_state = IDLE;  // new digit invalidates
      end
      UNLOCK: begin
        // stays UNLOCK until external reset
        next_state = UNLOCK;
```

```
      end
      default: next_state = IDLE;
    endcase
  end

  // Output logic (Moore: output depends only on state)
  assign unlocked = (state == UNLOCK);

endmodule

// (c) SVA Assertions:

// i. Wrong digit always returns to IDLE
property p_wrong_digit_to_idle;
  @(posedge clk) disable iff (rst)
  // In any non-UNLOCK state, if a digit is entered and it is wrong,
  // the next state must be IDLE.
  // Test case: in S1, pressing anything other than '7' -> IDLE
  (state == S1 && digit_valid && digit != D7) |=>
    (state == IDLE);
endproperty
assert property (p_wrong_digit_to_idle)
  else `uvm_error("LOCK", "Wrong digit did not return to IDLE")

// ii. UNLOCK is only reachable via the correct sequence
// (i.e. must have passed through S1->S2->S3->S4->UNLOCK in order)
property p_unlock_requires_sequence;
  @(posedge clk) disable iff (rst)
  // UNLOCK can only be entered from S4
  $rose(unlocked) |-> $past(state, 1) == S4;
endproperty
assert property (p_unlock_requires_sequence)
  else `uvm_error("LOCK", "UNLOCK reached without correct prior state S4")

// iii. UNLOCK maintained until reset
property p_unlock_stable;
  @(posedge clk) disable iff (rst)
  // Once unlocked, it stays unlocked (reset is async, not captured here)
  unlocked |=> unlocked;
endproperty
assert property (p_unlock_stable)
  else `uvm_error("LOCK", "UNLOCK state was exited without reset")

// Bonus: sequence-based property -- full correct sequence leads to UNLOCK
sequence seq_correct;
  // Exact sequence: 4 -> 7 -> 2 -> 9 -> lock_btn, each on digit_valid pulses
  (digit_valid && digit == D4) ##1
  (digit_valid && digit == D7) ##1
  (digit_valid && digit == D2) ##1
  (digit_valid && digit == D9) ##1
  lock_btn [->1];         // eventually lock_btn is pressed
endsequence

property p_correct_sequence_unlocks;
  @(posedge clk) disable iff (rst)
  seq_correct |-> ##1 unlocked;
endproperty
assert property (p_correct_sequence_unlocks)
  else `uvm_error("LOCK", "Correct sequence did not unlock")
```

解释

密码锁 FSM 是经典的 Moore 状态机设计题，面试要能快速画出状态图再转成 RTL。

RTL 设计要点：

状态编码：6个状态用 3-bit，用 typedef enum 提高可读性。

状态转移：always_comb + unique case，综合工具会优化并检查完备性。

输出：Moore machine，输出只取决于当前状态（不依赖输入）。

安全性设计：default: IDLE 防止进入非法状态。

新数字使 S4 失效：在 S4 状态收到任意新数字（不管对错）都回到 IDLE，

防止暴力猜测 lock_btn 时机。

SVA assertions 三个层次：

局部正确性：wrong digit -> IDLE（每个状态的转移是否正确）。

路径约束：UNLOCK 只能从 S4 到达（不能跳过任何步骤）。

全局稳定性：UNLOCK 状态保持直到 reset。

bonus sequence：端到端验证完整密码序列能够开锁。

## Q29. Computer Architecture: ROB and CFC -- Out-of-Order Execution with In-Order Commit

QUESTION

How do the Reorder Buffer (ROB) and Common Data Bus / Completion and Forwarding Circuit (CFC) work together to allow out-of-order execution while guaranteeing in-order commitment?

ANSWER

```
// Out-of-order execution requires two seemingly contradictory goals:
//   1. Execute instructions as soon as their operands are ready (OOO)
//   2. Commit results in program order (in-order) for correct semantics
//
// ROB (Reorder Buffer) -- the in-order commitment mechanism:
//   The ROB is a circular FIFO buffer. Every instruction is allocated
//   an entry in the ROB in program order (at dispatch/issue time).
//   Each ROB entry tracks:
//     - Instruction type and destination register / memory address
//     - Computed result value (filled when execution completes)
//     - Completion bit (0 = still executing, 1 = result ready)
//     - Exception / branch misprediction flag
//
//   The HEAD of the ROB always points to the oldest instruction.
//   COMMIT (retirement) only happens from the HEAD.
//   An instruction can commit ONLY when:
//     (a) Its completion bit is 1 (result computed)
//     (b) No older instruction has a pending exception
//   This enforces in-order commitment: even if instruction N+5
//   finishes execution before instruction N, it cannot commit
//   until instructions N through N+4 have all committed first.
//
// CFC (Completion and Forwarding Circuit) / Common Data Bus (CDB):
//   When a functional unit finishes computing a result, it broadcasts
//   the result on the CDB along with the ROB entry tag (destination).
//   ALL reservation stations and the ROB listen on the CDB.
//   Any waiting instruction whose source operand matches the broadcast
//   tag captures the value immediately -- this is register forwarding.
//   The ROB entry is marked complete (completion bit set to 1).
//
```

```
// How they work together:
//   1. Dispatch: instruction enters ROB (tail), gets ROB tag.
//      Register Alias Table (RAT) maps destination register -> ROB tag.
//      Source operands: if already in ROB and complete -> read value;
//      if pending -> record ROB tag as operand source.
//
//   2. Issue: when all operands available, instruction issues to
//      functional unit. Multiple instructions can execute simultaneously.
//
//   3. Writeback (CDB broadcast): result + ROB tag broadcast.
//      Waiting reservation stations capture operands.
//      ROB entry marked complete.
//
//   4. Commit: HEAD of ROB checks completion bit each cycle.
//      If complete AND no exception: write result to architectural
//      register file (ARF) and advance HEAD.
//      This is the ONLY point where architectural state changes.
//
// Key properties:
//   - Speculative results live in ROB only (not yet in ARF)
//   - On branch misprediction: flush ROB tail back to mispredicted
//     branch; architectural state (ARF) remains correct
//   - On exception: flush everything after the excepting instruction;
//     handle exception precisely (architectural state is consistent)
//   - The ROB acts as a "speculation buffer" between execution and
//     commitment
```

解释

ROB 解决的核心问题：乱序执行时如何保证异常精确处理和分支预测恢复。

没有 ROB 时，乱序执行的结果直接写回寄存器文件，一旦发生异常就无法回滚。

有 ROB 时，所有推测执行的结果先放在 ROB 里，

只有到达 ROB HEAD（最老的指令）且没有异常，才提交到架构寄存器文件。

CDB/CFC 解决的问题：让正在等待的指令能够立即获得刚算完的值，

不需要通过寄存器文件中转，大幅减少等待时间。

两者分工：CDB 负责把值传递给等待的指令（执行层面），

ROB 负责控制何时把值永久写入架构状态（提交层面）。

## Q30. Computer Architecture: Two Kinds of RAT and What They Solve

QUESTION

Why do we need two kinds of Register Alias Table (RAT)?

What specific problem does each one solve?

ANSWER

```
// Background: Register renaming eliminates false dependencies
// (WAR and WAW hazards) by mapping architectural registers to
// physical registers or ROB entries.
//
// There are two RATs maintained simultaneously:
//
// RAT 1: Front-End RAT (Speculative RAT / Rename RAT)
//   Also called: "in-flight RAT", "dispatch-time RAT"
//
```

```
//   Purpose: tracks the LATEST mapping of each architectural register
//   including all speculative (not yet committed) instructions.
//   Updated every cycle at rename/dispatch stage.
//   Used to: look up operand sources for newly dispatched instructions.
//
//   Problem it solves: WAR and WAW hazards among in-flight instructions.
//   Example: if R1 is written by instruction 5 (not yet committed),
//   instruction 10 reading R1 must use the ROB entry of instruction 5,
//   not the stale value in the architectural register file.
//   The front-end RAT provides this up-to-date speculative mapping.
//
//   Risk: if a branch misprediction occurs, this RAT has speculative
//   state that must be rolled back -- it cannot be the "truth".
//
// RAT 2: Retirement RAT (Architectural RAT / Committed RAT)
//   Also called: "committed RAT", "ARF state RAT"
//
//   Purpose: tracks the COMMITTED (architecturally correct) mapping.
//   Updated only when an instruction commits from the ROB HEAD.
//   Always reflects the exact architectural register state --
//   no speculation.
//
//   Problem it solves: fast recovery on misprediction or exception.
//   On a flush: simply copy the retirement RAT into the front-end RAT.
//   The pipeline restarts from the correct architectural state.
//
//   Without a retirement RAT: recovery requires walking the ROB
//   backwards to undo all speculative renames (expensive, slow).
//   With a retirement RAT: O(1) recovery by copying one table.
//
// Summary:
//   Front-end RAT: enables correct OOO execution for in-flight instr.
//                  Solves: WAR/WAW hazards among speculative instructions.
//   Retirement RAT: enables fast, correct recovery from speculation.
//                   Solves: expensive rollback cost on misprediction.
//
//   Together they allow: rename on the way in (front-end RAT),
//   commit cleanly and recover cheaply (retirement RAT).
```

解释

两种 RAT 的核心区别：推测状态 vs 已提交状态。

前端 RAT（推测 RAT）：始终是"最新"的映射，包含所有还没提交的指令的结果，

用于在派发新指令时找到正确的操作数来源，解决 WAR/WAW 假相关。

退休 RAT（架构 RAT）：只反映已经提交的指令，是"真实"的架构状态，

用于预测失败时快速恢复——直接把退休 RAT 复制到前端 RAT 即可，O(1) 恢复。

类比：前端 RAT 是草稿纸，退休 RAT 是最终答案纸。

## Q31. Computer Architecture: Purpose of CFC in Tomasulo Machine

QUESTION

What is the purpose of the CFC (Common Data Bus / Completion and

Forwarding Circuit) in the Tomasulo machine?

What specific problem does it solve?

ANSWER

```
// The Tomasulo algorithm introduces the Common Data Bus (CDB),
// which is the forwarding mechanism that makes OOO execution practical.
//
// Problem it solves: WAR and WAW hazards through register forwarding,
// and eliminating the need to stall waiting for slow register file access.
//
// Without CDB:
//   An instruction waiting for a result must poll the register file
//   each cycle to see if the value has been written. This either
//   requires centralized register file writes (serialized, slow)
//   or complex point-to-point forwarding between every pair of units.
//
// The CDB solution:
//   When a functional unit completes, it broadcasts:
//     {tag (ROB/RS identifier), result value}
//   on a shared bus (the CDB).
//   ALL reservation stations simultaneously listen to the CDB.
//   Any RS holding a source operand tag that matches the broadcast
//   captures the value and marks that operand as "ready".
//   When ALL operands of an instruction are ready, it can issue.
//
// What CDB enables:
//   1. Value forwarding WITHOUT going through the register file:
//      result flows directly from producer to all waiting consumers.
//   2. Tag-based dependency tracking:
//      instructions don't wait for a specific register name --
//      they wait for a specific ROB/RS tag. This eliminates
//      WAR and WAW hazards (register renaming via tags).
//   3. Broadcast semantics: one broadcast serves all waiting consumers
//      simultaneously -- no point-to-point wiring needed.
//
// Limitations:
//   Classic Tomasulo has ONE CDB -- only one result can be
//   broadcast per cycle -> structural hazard if multiple units
//   finish simultaneously.
//   Modern processors use MULTIPLE CDBs (write ports) to handle
//   multiple completions per cycle.
```

解释

CDB 是 Tomasulo 算法中最关键的创新，它把"值传递"从集中式改为广播式。

核心思想：当一个结果计算完成，不是写入寄存器文件再等其他指令去读，

而是直接广播给所有等待的预约站（Reservation Station），

等待者通过 tag 匹配自动捕获，不需要显式的寄存器文件访问。

解决的问题：WAR（写后读）和 WAW（写后写）假相关——

因为操作数用 tag 追踪而不是寄存器名，

同一个逻辑寄存器的多次写操作互不干扰。

CDB 是乱序执行中"数据转发"的基础设施，

ROB 用它来标记指令完成，RS 用它来获取操作数。

## Q32. Computer Architecture: MOESI vs MESI -- The O State and Performance

QUESTION

(a) Describe the key difference between MOESI and the simpler

MESI protocol. What problem does the O (Owned) state solve?

(b) How does the O state improve performance?

(c) What is the difference between a cache line in Exclusive (E) state versus Modified (M) state?

ANSWER

```
// Cache coherence protocol states:
//
// MESI states:
//   M (Modified):  line is dirty, only copy, owner must write back
//                  before another cache can read it.
//   E (Exclusive): line is clean, only copy, matches main memory.
//                  Can upgrade to M silently on write (no bus transaction).
//   S (Shared):    line is clean, may be in other caches, read-only.
//   I (Invalid):   line not present or stale.
//
// MOESI adds one state:
//   O (Owned):     line is DIRTY, but SHARED with other caches.
//                  The Owned cache is responsible for supplying the
//                  line to other caches AND eventually writing back
//                  to memory. Other caches hold it in S state.
//
// (a) Problem the O state solves -- the MESI write-back bottleneck:
//
//   In MESI: when Core 0 has line X in M state and Core 1 wants to read X:
//     Step 1: Core 0 must WRITE BACK X to main memory.
//     Step 2: Main memory supplies X to Core 1.
//     Core 0 transitions: M -> S (or I).
//   Problem: this forces a WRITE-BACK to main memory even though
//   Core 0 still wants to use the line. The write-back wastes:
//     - Memory bandwidth (writing a line that's still needed)
//     - Latency (write-back must complete before Core 1 gets data)
//
//   In MOESI: when Core 0 has X in M and Core 1 reads X:
//     Core 0 supplies X directly to Core 1 (cache-to-cache transfer).
//     Core 0 transitions: M -> O (still holds dirty line, is "owner").
//     Core 1 transitions: I -> S.
//     NO write-back to main memory at this point.
//   The O cache keeps the dirty copy and supplies it on future reads.
//   Write-back only happens when the O cache itself evicts the line.
//
// (b) Performance improvement from O state:
//   - Eliminates unnecessary write-backs to memory during cache-to-cache
//     transfers -> saves memory bandwidth.
//   - Reduces latency: data goes directly from owner cache to requester,
//     faster than going through main memory.
//   - Reduces memory traffic in multi-core workloads where shared dirty
//     data is common (producer-consumer patterns).
//
// (c) Exclusive (E) vs Modified (M):
//
//   E (Exclusive):
//     - Cache line matches the copy in main memory (CLEAN).
//     - Only this cache has the line (no other sharers).
//     - Can be written silently: E -> M without any bus transaction.
//       No need to tell other caches (there are none).
//     - On eviction: no write-back needed (memory already has correct copy).
//
//   M (Modified):
//     - Cache line DIFFERS from main memory (DIRTY).
//     - Only this cache has the line (no other sharers).
//     - Already written; main memory copy is stale/invalid.
```

```
//     - On eviction OR when another cache requests the line:
//       MUST write back to memory (or supply via cache-to-cache in MOESI).
//     - Other caches cannot read this line without first getting the
//       updated value from the M cache.
//
// Summary table:
//   State | Dirty? | Only copy? | Write-back on evict?
//   E     |  No    |   Yes      |  No  (memory is up to date)
//   M     |  Yes   |   Yes      |  Yes (memory is stale)
//   O     |  Yes   |   No       |  Yes (owner responsible)
//   S     |  No    |   No       |  No  (memory is up to date)
//   I     |  N/A   |   N/A      |  N/A (line not present)
```

解释

MOESI vs MESI 的核心区别：O 状态避免了不必要的内存写回。

在 MESI 中，Dirty 数据（M state）一旦被其他 cache 请求，

必须先写回内存再由内存供给请求者，产生额外的内存带宽消耗和延迟。

MOESI 的 O 状态允许 cache-to-cache 直接传输 dirty 数据，

原持有者变成 Owner（保持脏副本的责任人），只有在最终驱逐时才写回内存。

E vs M 的关键区别：E 是干净的独占（内存有效），M 是脏的独占（内存过期）。

E 可以无总线事务地升级到 M（静默升级），而 M 在被驱逐时必须写回内存。

## Q33. Computer Architecture: MOESI State Transition Table for 4-Core CMP

QUESTION

Consider a 4-core CMP with private L1 caches connected by a

snooping bus, all using MOESI (then compare with MESI).

Walk through state transitions for cache line X across all four

cores for this access sequence:

1. Core 0 reads X (initial: not in any cache)
2. Core 0 writes X
3. Core 1 reads X
4. Core 2 reads X
5. Core 3 writes X
6. Core 0 reads X

For each step: show the state of X in each core's cache,

identify the bus transaction, and explain.

Follow-up: which steps differ between MOESI and MESI?

Specifically: which step requires a write-back to main memory

in MESI but NOT in MOESI?

ANSWER

```
// State legend: M=Modified, O=Owned, E=Exclusive, S=Shared, I=Invalid
// '-' = line not present (same as I for our purposes)
//
```

```
// ■■ MOESI Protocol ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
//
// Step 1: Core 0 reads X (cold miss)
//   Bus transaction: BusRd (no other sharer, line clean from memory)
//   C0: E  (only copy, clean -- Exclusive)
//   C1: -  C2: -  C3: -
//
// Step 2: Core 0 writes X (upgrade, C0 already has E)
//   Bus transaction: NONE -- silent upgrade E -> M
//   (No other sharers, so no need to broadcast invalidate)
//   C0: M  C1: -  C2: -  C3: -
//
// Step 3: Core 1 reads X (C0 has dirty copy in M)
//   Bus transaction: BusRd -- C0 snoops, supplies data via
//                   cache-to-cache transfer, does NOT write back to memory
//   C0: O  (was M, now Owned -- still dirty, responsible for line)
//   C1: S  (received clean copy from C0)
//   C2: -  C3: -
//
// Step 4: Core 2 reads X (C0=O, C1=S)
//   Bus transaction: BusRd -- C0 (owner) supplies data directly
//   C0: O  C1: S  C2: S  C3: -
//
// Step 5: Core 3 writes X (C0=O, C1=S, C2=S)
//   Bus transaction: BusRdX / Invalidate
//   All sharers (C1, C2) and owner (C0) invalidate their copies.
//   Data sourced from C0 (owner) since it has the dirty copy.
//   C0: I  C1: I  C2: I  C3: M
//
// Step 6: Core 0 reads X (C3=M, all others I)
//   Bus transaction: BusRd
//   C3 snoops: transitions M -> O, supplies data (cache-to-cache)
//   C0: S  C1: -  C2: -  C3: O
//
// MOESI final summary:
//   Step | Bus Transaction          | C0 | C1 | C2 | C3
//   -----+--------------------------+----+----+----+----
//   1    | BusRd (from memory)      | E  | -  | -  | -
//   2    | None (silent E->M)       | M  | -  | -  | -
//   3    | BusRd (C0 C2C, no WB)    | O  | S  | -  | -
//   4    | BusRd (C0 owner, C2C)    | O  | S  | S  | -
//   5    | BusRdX + Invalidate all  | I  | I  | I  | M
//   6    | BusRd (C3 C2C, no WB)    | S  | -  | -  | O
//
// ■■ MESI Protocol (comparison) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
//
// Steps 1 and 2 are identical to MOESI.
//
// Step 3 (KEY DIFFERENCE): Core 1 reads X (C0 has M)
//   MESI: C0 must WRITE BACK X to main memory first,
//         then memory supplies X to C1.
//         C0 transitions: M -> S  (line is now clean, shared)
//         C1 transitions: I -> S
//   Extra cost: one write-back to memory (wasted bandwidth).
//   C0: S  C1: S  C2: -  C3: -
//
// Step 4 (MESI): Core 2 reads X (C0=S, C1=S)
//   BusRd: memory supplies (C0 and C1 remain S).
//   C0: S  C1: S  C2: S  C3: -
//
// Step 5 (MESI): Core 3 writes X
//   BusRdX / Invalidate all sharers.
//   Memory supplies data (since all copies are clean/S in MESI).
```

```
//   C0: I  C1: I  C2: I  C3: M
//
// Step 6 (MESI): Core 0 reads X (C3 has M)
//   MESI: C3 must WRITE BACK to memory first,
//         memory supplies to C0.
//         C3: M -> S  C0: I -> S
//   Another write-back to memory.
//
// MESI final summary:
//   Step | Bus Transaction             | C0 | C1 | C2 | C3
//   -----+-----------------------------+----+----+----+----
//   1    | BusRd (from memory)         | E  | -  | -  | -
//   2    | None (silent E->M)          | M  | -  | -  | -
//   3    | BusRd + C0 WRITE-BACK       | S  | S  | -  | -
//   4    | BusRd (memory supplies)     | S  | S  | S  | -
//   5    | BusRdX + Invalidate all     | I  | I  | I  | M
//   6    | BusRd + C3 WRITE-BACK       | S  | -  | -  | S
//
// Follow-up answer -- steps requiring write-back in MESI but NOT MOESI:
//   Step 3: MESI writes back (M->S), MOESI does cache-to-cache (M->O)
//   Step 6: MESI writes back (M->S), MOESI does cache-to-cache (M->O)
//   MOESI saves 2 write-backs to main memory in this 6-step sequence.
```

解释

这道题综合考察 MOESI 协议的完整理解，以及与 MESI 的对比。

最关键的两个步骤是 Step 3 和 Step 6：

MESI：脏数据（M state）被请求时，必须先写回内存，再由内存供给请求者。

MOESI：脏数据被请求时，直接通过 cache-to-cache 传输，原持有者变 O state，

不需要写回内存，节省了内存带宽和时间。

O state 的本质：我还有最新的数据，我来负责供给其他人，内存暂时不用管。

只有当 O state 的 cache line 被驱逐时，才需要写回内存。

面试考点：能清楚说出 Step 3 和 Step 6 的区别，以及 O state 的作用。

## SystemVerilog — Constraints & Randomization

### Q34. Constraint: Address Alignment with CMD Signal

QUESTION

Randomize a 12-bit address 'addr' constrained to be within

[start_addr, end_addr]. Additionally, a 2-bit cmd signal controls

the alignment requirement:

cmd=0 -> 4B aligned (addr[1:0] == 0)

cmd=1 -> 8B aligned (addr[2:0] == 0)

cmd=2 -> 16B aligned (addr[3:0] == 0)

cmd=3 -> 32B aligned (addr[4:0] == 0)

ANSWER

```
class addr_txn;
     rand bit [11:0] addr;
     rand bit [1:0]   cmd;
     bit [11:0] start_addr;
     bit [11:0] end_addr;

     constraint c_range {
         addr inside {[start_addr : end_addr]};
     }

     constraint c_align {
         (cmd == 2'b00) -> (addr[1:0] == 2'b00);    // 4B   aligned
         (cmd == 2'b01) -> (addr[2:0] == 3'b000);   // 8B   aligned
         (cmd == 2'b10) -> (addr[3:0] == 4'b0000); // 16B aligned
         (cmd == 2'b11) -> (addr[4:0] == 5'b00000);// 32B aligned
     }
 endclass

 // Alternative: use solve...before to ensure cmd is chosen first,
 // then addr is randomized to satisfy alignment:
 constraint c_solve { solve cmd before addr; }

 // How alignment works:
 // 4B     aligned: lower 2 bits = 0   (addr is multiple of 4)
 // 8B     aligned: lower 3 bits = 0   (addr is multiple of 8)
 // 16B aligned: lower 4 bits = 0      (addr is multiple of 16)
 // 32B aligned: lower 5 bits = 0      (addr is multiple of 32)
```

解释

地址对齐约束是芯片验证面试的高频题，本质上是约束地址的低位 bit。

N-byte 对齐等价于地址的低 log2(N) 位全为 0。

用 implication constraint (cmd==X) -> (addr[...]==0) 来表达条件对齐。

加 solve cmd before addr 可以让求解器先决定 cmd，再根据 cmd 随机 addr，

避免因为 addr 先被随机化导致对齐约束冲突的概率过高。

## Q35. Constraint: Dynamic Array Sum and 2D Array Row Constraint

QUESTION

(a) Randomize a dynamic bit array of size 10 where exactly 6

elements are 1.

(b) Randomize a 3x3 2D bit array where each row has exactly 2 ones.

How many possible solutions are there?

ANSWER

```
// (a) 1D array: size 10, sum == 6
  class arr_q1;
     rand bit arr[];
     constraint c_size { arr.size() == 10; }
     // Must use int'(item) to promote each bit element before summing.
     // Without it, sum() accumulates in 1-bit width -> always 0 or 1.
     constraint c_sum    { arr.sum() with (int'(item)) == 6; }
  endclass
```

```
// (b) 2D array: each row sums to 2
class arr_q2;
   rand bit arr2[3][3];
   constraint c_row {
       foreach (arr2[i])
         // arr2[i] is a 3-bit packed array; promote each bit to int
         // before summing to avoid 1-bit accumulation overflow.
         arr2[i].sum() with (int'(item)) == 2;
   }
endclass

// How many solutions?
// Each row independently needs exactly 2 ones in 3 bits.
// Combinations per row: C(3,2) = 3
//      -> {110, 101, 011}
// 3 rows, each with 3 choices, independent:
// Total = 3 * 3 * 3 = 27 solutions
```

解释

(a) rand bit 的 sum() 必须加 with (int'(item)) 做类型提升。

不加的话，bit 类型的 sum() 在 1-bit 宽度内累加，

10 个 bit 相加最终只保留最低1位，结果永远是 0 或 1，

约束 == 6 永远无法满足，randomize() 会失败。

(b) foreach 遍历行下标，arr2[i] 是一个 3-bit 的 packed 数组行，

对它调用 .sum() with (int'(item)) 得到该行的1的个数（提升后可达到3）。

排列组合：每行从 3 个 bit 中选 2 个为1，有 C(3,2)=3 种，

三行独立，所以总方案数 = $3^3$ = 27。

## Q36. Constraint: Various Array Constraints

QUESTION

Write constraints for the following:

(a) int array of size 10 with all unique elements.

(b) 4x4 int 2D array where each row sums to 10.

(c) 8-bit value that is a power of 2 (exactly one bit set).

(d) Two int arrays a[] and b[] of size 10 where a is in

ascending order, b has all unique elements in range [100,200],

and a is in range [0,100].

ANSWER

```
// (a) unique 1D array
 class q_a;
     rand int arr[];
     constraint c { arr.size() == 10; unique {arr}; }
 endclass

 // (b) 2D array each row sums to 10
 class q_b;
     rand int arr2[4][4];
```

```
    constraint c {
        // int elements: no bit-width overflow, but use with(int'(item))
        // for clarity and safety. Fixed 2D arrays support .sum() per row.
        arr2[0].sum() with (int'(item)) == 10;
        arr2[1].sum() with (int'(item)) == 10;
        arr2[2].sum() with (int'(item)) == 10;
        arr2[3].sum() with (int'(item)) == 10;
    }
endclass

// (c) power of 2: exactly one bit set
class q_c;
    rand bit [7:0] value;
    constraint c { $onehot(value); }
    // Equivalently: $countones(value) == 1;
endclass

// (d) two arrays with range, ordering, and uniqueness constraints
class q_d;
    rand int a[];
    rand int b[];
    constraint c_size {
        a.size() == 10;
        b.size() == 10;
    }
    constraint c_range {

        foreach (a[i]) a[i] inside {[0:100]};
        foreach (b[i]) b[i] inside {[100:200]};
    }
    constraint c_a_asc {
        foreach (a[i])
          if (i > 0) a[i] > a[i-1];
    }
    constraint c_b_unique { unique {b}; }
endclass
```

解释

(a) unique 关键字直接约束整个数组所有元素互不相同，是最简洁的写法。

(b) 对 2D 数组每行用 .sum() with (int'(item)) == 目标值来约束，不能省略每一行。

虽然 arr2 是 int 类型不会溢出，加 int'(item) 是良好习惯，

也能避免将来改成 bit/byte 类型时忘记修改约束。

(c) $onehot(value) 是系统函数，直接约束只有一个 bit 为1，

比手写 $countones==1 更语义明确，两者效果相同。

(d) 升序约束用 foreach + if(i>0) a[i]>a[i-1]，

注意这里用严格大于（>）排除相等，如果允许相等用 >= 即可。

## Q37. SV Constraint: 1D/2D Array Size, Value, and Ordering

QUESTION

Write constraints for all of the following. Each part builds on

the previous -- the interviewer may ask them progressively.

(a) Randomize a dynamic int array of size between 5 and 10,

where all elements are in range [0, 100] and the array
is sorted in strictly ascending order.

(b) Extend (a): add a constraint that no two adjacent elements
differ by more than 20.
i.e. for all i: arr[i+1] - arr[i] <= 20

(c) Randomize a 4x4 2D int array where:
- Each row is sorted in ascending order
- Each column is also sorted in ascending order
- All values are in range [0, 255]
- No value appears more than once in the entire array

(d) Given the following class, write a constraint so that
whenever type == READ, addr must be 64-byte aligned;
whenever type == WRITE, data must be non-zero and
addr must be within [base_addr, base_addr + 4096).
Also, if burst_len > 1, all addresses in the burst
(addr, addr+4, addr+8, ..., addr+4*(burst_len-1)) must
stay within the same 4KB page (no page crossing).

```
typedef enum {READ, WRITE} op_t;
class txn;
    rand op_t        type;
    rand bit [31:0] addr;
    rand bit [31:0] data;
    rand bit [3:0]   burst_len;   // 1 to 15
    bit [31:0]       base_addr;
endclass
```

ANSWER

```
// (a) Dynamic array: size in [5,10], values in [0,100], ascending
class arr_a;
  rand int arr[];
  constraint c_size {
      arr.size() inside {[5:10]};
  }
  constraint c_range {
      foreach (arr[i]) arr[i] inside {[0:100]};
  }
  constraint c_ascending {
      foreach (arr[i])
        if (i > 0) arr[i] > arr[i-1];   // strictly ascending
  }
endclass

// (b) Add max-step constraint: adjacent diff <= 20
class arr_b extends arr_a;
  constraint c_step {
      foreach (arr[i])
        if (i > 0) arr[i] - arr[i-1] <= 20;
  }
  // Combined with c_ascending: 0 < diff <= 20 between neighbors
```

```
  // Note: c_ascending (diff > 0) + c_step (diff <= 20) together
  // constrain the step to be in [1, 20].
  // With size up to 10 and step up to 20, max reachable value
  // = arr[0] + 9*20 = arr[0] + 180. Since arr[0] >= 0 and
  // arr[9] <= 100, effective step is further constrained.
  // The solver will find a valid solution automatically.
endclass

// (c) 4x4 2D array: row-sorted, col-sorted, unique, in [0,255]
class arr_c;
  rand int arr[4][4];
  constraint c_range {
      foreach (arr[i,j]) arr[i][j] inside {[0:255]};

    }
    constraint c_row_asc {
        foreach (arr[i,j])
          if (j > 0) arr[i][j] > arr[i][j-1];
    }
    constraint c_col_asc {
        foreach (arr[i,j])
          if (i > 0) arr[i][j] > arr[i-1][j];
    }
    // Global uniqueness: row+col ascending already prevents
    // duplicates within a row or column, but not across.
    // Add explicit cross-element uniqueness:
    constraint c_unique {
        foreach (arr[i,j])
          foreach (arr[k,l])
           if (!(i==k && j==l)) arr[i][j] != arr[k][l];
    }
    // Note: c_row_asc + c_col_asc imply a "staircase" structure.
    // The solver will find a valid Young tableau arrangement.
 endclass

 // (d) Conditional alignment, range, and page-crossing constraint
 class txn;
    rand op_t        type;
    rand bit [31:0] addr;
    rand bit [31:0] data;
    rand bit [3:0]   burst_len;
    bit [31:0]       base_addr;

    constraint c_burst_len {
        burst_len inside {[1:15]};
    }

    constraint c_read {
        // READ: addr must be 64-byte aligned

        (type == READ) -> (addr[5:0] == 6'b0);
    }

    constraint c_write {
        // WRITE: data non-zero, addr in [base_addr, base_addr+4096)
        (type == WRITE) -> (data != 0);
        (type == WRITE) -> (addr >= base_addr &&
                            addr <   base_addr + 32'h1000);
    }

    constraint c_no_page_cross {
        // All burst beats must stay within the same 4KB page.
        // Last beat addr = addr + 4*(burst_len-1)
```

```
        // Same page means addr[31:12] == last_addr[31:12]
        // i.e. (addr + 4*(burst_len-1)) >> 12 == addr >> 12
        // Simplified: addr[11:0] + 4*(burst_len-1) < 4096
        (burst_len > 1) ->
          ({20'b0, addr[11:0]} + (32'(burst_len) - 1) * 4 < 32'h1000);
    }
endclass
```

解释

这道题是 Google 第一道题的完整版本，逐步加难，考查层次很分明。

(a)(b) 动态数组升序 + 步长限制：

foreach 约束升序是最基础的写法， if(i>0) arr[i] > arr[i-1]

加步长限制时用继承扩展类（extends arr_a）而不是重写，

这样原有约束和新约束同时生效。

(c) 2D 数组行列升序是 Young tableau 结构，

行内升序 + 列内升序已经隐含了行内和列内的唯一性，

但不能防止跨行跨列的重复元素，需要额外的全局唯一性约束。

(d) 页面不跨越约束是最具挑战性的部分：

计算最后一拍 addr = addr + 4*(burst_len-1)，

判断和起始 addr 是否在同一个 4KB 页面内，

即 addr[11:0] + 4*(burst_len-1) < 4096。

这种写法比判断页号 addr[31:12] 更直接，也不会有溢出问题。

## Q38. SV Constraint: Dynamic Array with Sum Equal to 20

QUESTION

Write a constraint class where a dynamic integer array has:

- Size between 4 and 8

- All elements in range [1, 10]

- The sum of all elements equals exactly 20

Explain why int'(item) is needed in the sum constraint.

ANSWER

```
class sum_array;
    rand int unsigned arr[];

    constraint c_size {
        arr.size() inside {[4:8]};
    }

    constraint c_range {
        foreach (arr[i]) arr[i] inside {[1:10]};
    }

    constraint c_sum {
        // int'(item) promotes each element to 32-bit signed int
```

```
        // before accumulating in the sum.
        // Without it, sum() accumulates in the native element width.
        // For int unsigned this is fine here, but for bit or smaller
        // types it would overflow (e.g. 10 x bit[3:0] max=15 -> wraps).
        // int'(item) is best practice for ALL sum() constraints.
        arr.sum() with (int'(item)) == 20;
    }
endclass

// Why int'(item) matters -- concrete overflow example:
// rand bit [3:0] small_arr[];
// small_arr.size() == 5; each element max = 15
// small_arr.sum() == 20:
//       Without int'(item): accumulates in 4-bit; 20 % 16 = 4 -> wrong!
//       With   int'(item): accumulates in 32-bit; 20 fits -> correct.

// Feasibility check: size in [4,8], elements in [1,10], sum=20
//       Min possible sum: 4 elements * 1 = 4     (feasible, 4 <= 20)
//       Max possible sum: 8 elements * 10 = 80     (feasible, 80 >= 20)
//       Average needed: 20 / size (2.5 to 5) -- always within [1,10].
//       Solver can find solutions easily.

// Bonus: count elements equal to a specific value
// arr.sum() with (int'(item == 5)) == 3;
// -> exactly 3 elements equal 5
```

解释

sum() 约束是高频考题，int'(item) 是必须写对的细节。

本质：sum() 默认在元素的原始类型宽度内累加，窄类型会溢出。

int'(item) 先把每个元素提升到 32-bit 有符号整数，再累加，避免溢出。

可行性检查：面试时能快速验证约束是否有解（最小和 <= 目标 <= 最大和）。

延伸写法：sum() with (int'(item == VAL)) 统计满足条件的元素个数。

## Q39. SV Constraint: Non-Overlapping Rectangle Regions in 2D Array

QUESTION

Given a 2D grid of size ROWS x COLS, repeatedly randomize a
rectangular region (defined by top-left corner and dimensions)
such that:

- Each selected rectangle lies fully within the grid boundary
- No two selected rectangles overlap (even partially)
- Keep selecting until no valid non-overlapping rectangle can
be found (the remaining free area cannot fit even 1x1)

Requirements:

(a) Define the rectangle class with proper constraints.

(b) Track previously selected rectangles and add the
non-overlap constraint dynamically for each new call.

(c) Handle the termination condition correctly.

(d) Explain why a purely declarative constraint approach has limitations here and how you work around them.

ANSWER

```
// Rectangle overlap detection:
  // Two rectangles R1 and R2 DO NOT overlap if any of these hold:
  //     R1 is entirely to the LEFT of R2:        R1.col + R1.w <= R2.col
  //     R1 is entirely to the RIGHT of R2:       R2.col + R2.w <= R1.col
  //     R1 is entirely ABOVE R2:                 R1.row + R1.h <= R2.row
  //     R1 is entirely BELOW R2:                 R2.row + R2.h <= R1.row
  // They OVERLAP if NONE of the above hold (De Morgan's law).

  // -- Data type ------------------------------------------------
  typedef struct {
    int row, col;      // top-left corner (0-indexed)
    int h,    w;       // height, width (>= 1)
  } rect_t;

  // -- Rectangle class with dynamic non-overlap constraint -----
  class rect_picker #(
    parameter ROWS = 8,
    parameter COLS = 8
  );
    // Randomizable fields for the current rectangle
    rand int row, col, h, w;

    // Accumulated list of already-chosen rectangles
    // (cannot be rand; updated externally after each successful pick)
    rect_t chosen[$];

    // -- Boundary constraint ---------------------------------
    constraint c_bounds {
        row >= 0; col >= 0;
        h   >= 1; w    >= 1;
        row + h <= ROWS;
        col + w <= COLS;
    }

    // -- Non-overlap constraint (dynamically generated) --------

// For every previously chosen rectangle R, the new rectangle
// must satisfy at least ONE of the four separation conditions.
// Because the number of chosen rectangles grows each round,
// we use constraint_mode() + a hand-built constraint expression
// encoded via a helper.
//
// SV does not allow foreach over a dynamic queue inside a
// declarative constraint block with runtime-variable bounds,
// so we use soft constraints + post_randomize rejection,
// OR we programmatically build the constraint via rand_mode.
//
// Cleanest SV approach: encode the constraint via a packed
// bit array that flags each cell as occupied, then constrain
// that the new rectangle covers NO occupied cell.

// Occupied map: 1 = cell already taken by a chosen rectangle
bit occupied[ROWS][COLS];

// The constraint: for every cell (r,c) in the new rectangle,
// occupied[r][c] must be 0.
// We express this as: the sum of occupied cells in the
// candidate region must be 0.
```

```
//
// Direct cell-level constraint (correct but solver-heavy):
constraint c_no_overlap {
  foreach (occupied[r, c]) {
      // If cell (r,c) is occupied, the new rectangle must not cover it.
      // Cell (r,c) is covered by new rect iff:
      //    row <= r < row+h   AND   col <= c < col+w
      // Equivalently: NOT (row<=r && r<row+h && col<=c && c<col+w)
      if (occupied[r][c]) {
          !(row <= r && r < row + h &&
            col <= c && c < col + w);
      }
  }

  }

  // Update occupied map after a rectangle is chosen
  function void mark_occupied(rect_t r);
      for (int i = r.row; i < r.row + r.h; i++)
        for (int j = r.col; j < r.col + r.w; j++)
         occupied[i][j] = 1'b1;
      chosen.push_back(r);
  endfunction

  // Check if ANY valid placement still exists (termination condition)
  // A 1x1 rectangle can always be placed in any free cell.
  // So we just need at least one unoccupied cell.
  function bit any_free_cell();
      foreach (occupied[r, c])
        if (!occupied[r][c]) return 1'b1;
      return 1'b0;
  endfunction

  // Reset for a fresh run
  function void reset();
      foreach (occupied[r, c]) occupied[r][c] = 1'b0;
      chosen.delete();
  endfunction
endclass

// -- Controller: repeatedly pick non-overlapping rectangles --
class rect_controller #(
  parameter ROWS = 8,
  parameter COLS = 8
);
  rect_picker #(ROWS, COLS) picker;
  rect_t                    all_rects[$];

  function new();

 picker = new();
endfunction

task run();
 rect_t cur;
 int      attempt;
 int      MAX_ATTEMPTS = 1000;

 picker.reset();
 all_rects.delete();

 // Keep picking until no free cell remains
 while (picker.any_free_cell()) begin
```

```
    bit success = 0;

    // Try up to MAX_ATTEMPTS times to find a valid rectangle
    for (attempt = 0; attempt < MAX_ATTEMPTS; attempt++) begin
       if (picker.randomize()) begin
          // Double-check: constraint solver may not be perfect
          // for complex conditional constraints; verify manually
          if (no_overlap_check(picker.row, picker.col,
                               picker.h,   picker.w)) begin
             cur = '{picker.row, picker.col, picker.h, picker.w};
             picker.mark_occupied(cur);
             all_rects.push_back(cur);
             success = 1;
             break;
          end
       end
    end

    // If we exhausted attempts and still failed, stop
    // (remaining free cells may be isolated / too small)
    if (!success) break;
 end

     $display("Selected %0d non-overlapping rectangles:", all_rects.size());
     foreach (all_rects[i])
       $display("   [%0d] row=%0d col=%0d h=%0d w=%0d",
         i, all_rects[i].row, all_rects[i].col,
            all_rects[i].h,     all_rects[i].w);
  endtask

  // Manual overlap verification (used as fallback check)
  function bit no_overlap_check(int row, int col, int h, int w);
     for (int r = row; r < row + h; r++)
       for (int c = col; c < col + w; c++)
         if (picker.occupied[r][c]) return 0;
     return 1;
  endfunction
endclass

// -- Example usage -----------------------------------------
// rect_controller #(.ROWS(8), .COLS(8)) ctrl = new();
// ctrl.run();
//
// Possible output on an 8x8 grid:
//    [0] row=1 col=2 h=3 w=4    (covers rows 1-3, cols 2-5)
//    [1] row=0 col=0 h=1 w=2    (covers row 0,    cols 0-1)
//    [2] row=5 col=5 h=2 w=3    (covers rows 5-6, cols 5-7)
//    ...

// -- Why pure declarative constraints have limitations --------
//
// Problem 1: Dynamic queue size in constraint block
//    SV constraint blocks are elaborated before simulation;
//    a foreach over a runtime-growing queue (chosen[$]) is not
//    well-supported in all solvers. The occupied[][] map approach
//    sidesteps this by pre-computing the "forbidden zone" before
//    each randomize() call.

  //
  // Problem 2: Conditional cell constraint is solver-heavy
  //   For an 8x8 grid, the constraint has up to 64 conditional
  //   terms (one per cell). For a 64x64 grid this is 4096 terms.
  //   Most commercial solvers handle this but performance degrades.
```

```
//   Alternative: replace with implication constraints per chosen
//   rectangle (4 separation conditions per rect), which scales
//   as O(N_chosen) rather than O(ROWS*COLS).
//
// Problem 3: Termination detection
//   "No valid rectangle exists" is hard to express as a constraint.
//   We use any_free_cell() + MAX_ATTEMPTS to detect it: if the
//   solver consistently fails to find a non-overlapping placement
//   after many tries, we assume we are stuck and stop.
//   A more rigorous approach: enumerate all possible 1x1..NxM rects
//   and check if any is free -- but this is O(ROWS^2 * COLS^2).
//
// Alternative O(N_chosen) constraint formulation:
//   Instead of the occupied map, add one constraint per chosen rect:
//   constraint c_sep_0 {   // for chosen[0]
//       (col + w <= chosen[0].col) ||   // new is left of chosen[0]
//       (chosen[0].col + chosen[0].w <= col) ||    // right
//       (row + h <= chosen[0].row) ||   // above
//       (chosen[0].row + chosen[0].h <= row);      // below
//   }
//   Each new round adds one more such named constraint.
//   Use constraint_mode() to enable/disable them if needed.
//   This is cleaner for small N_chosen but requires dynamic
//   constraint generation (often done via macros or code-gen).
```

解释

这道题是非常有深度的 constraint 开放题，有两种主要思路。

思路一：Occupied Map（本题采用）

用一个二维 bit 数组记录已占用的单元格，

每次 randomize() 之前更新占用图，

约束：新矩形覆盖的每个单元格都必须是 0。

优点：直观，易于手动验证；缺点：约束项数 = ROWS*COLS。

思路二：每个已选矩形加一个分离条件约束

管吗分离的四个条件（左/右/上/下）用 OR 连接，

每个已选矩形对应一个这样的约束块，动态添加。

优点：约束项数 = O(N_chosen)；缺点：需要动态生成约束名称。

终止条件：检查是否还有至少一个空闲单元格（any_free_cell）是最实用的方法。

加上最大尝试次数守卫（MAX_ATTEMPTS）防止求解器卡死在无解情况下。

## Q40. SV Constraint: Exactly One Consecutive Run of 1s in 16-bit Value

QUESTION

(a) Write a constraint class for a 16-bit value where the bits
contain EXACTLY ONE consecutive run of 1s (and all other
bits are 0). For example: 16'b0001_1110_0000_0000 is valid
(one run of four 1s), but 16'b0001_0001_0000_0000 is not
(two separate runs).

(b) The interviewer's intended approach: randomize a starting

index and a size, where index + size <= 16, then construct the target value in post_randomize by building a mask of 'size' consecutive 1s starting at 'index'.

(c) Also write an alternative purely declarative constraint solution without using post_randomize.

ANSWER

```
// (b) Interviewer's intended approach: index + size -> build value
 class one_run_b;
   rand bit [3:0] index;      // starting bit position (0 = LSB)
   rand bit [4:0] size;       // number of consecutive 1s (1 to 16)
   bit [15:0] value;          // result -- NOT rand, built in post_randomize

   constraint c_bounds {
       size   >= 1;
       index + size <= 16;
   }

   function void post_randomize();
       // Build a mask of 'size' ones starting at bit 'index'
       // Example: index=3, size=4 -> 0000_0000_0111_1000
       value = 16'b0;
       for (int i = index; i < index + size; i++)
         value[i] = 1'b1;
       // Equivalently in one expression:
       // value = ((16'b1 << size) - 1) << index;
   endfunction
 endclass

 // Verification: quick check
 // index=2, size=3 -> value = 16'b0000_0000_0000_1110      (bits 2,3,4)
 // index=0, size=1 -> value = 16'b0000_0000_0000_0001      (bit 0 only)
 // index=12,size=4 -> value = 16'b1111_0000_0000_0000      (bits 12-15)

 // (c) Alternative: purely declarative constraint
 // Observation: a single run of 1s means:
 //     value != 0    (at least one 1)
 //     (value & (value + 1)) == 0 is NOT sufficient -- that only checks
 //     if value is of the form 0...01...1 (run at LSB).
 //     We need to check that (value >> ctz(value)) has no 0 before a 1.
 //
 // Cleaner declarative approach:

 // A value has exactly one run of 1s iff:
 //       Let shifted = value >> (first set bit position)
 //       Then shifted must be of the form 0...01...1 (all-ones run at LSB)
 //       i.e., shifted & (shifted + 1) == 0
 //
 // In SV constraints, we can use the two-variable approach:
 // rand bit [15:0] mask_low;       // mask of lower part: 0...01...1
 // value = mask_low << lsb;        -- but this needs post_randomize too
 //
 // Most elegant declarative-only constraint:
 // "No 0 appears between two 1s" <=>
 // "value & ~(value | (value-1)) shifted variants == 0"
 // This is complex. The practical alternative is:
 //
 class one_run_declarative;
     rand bit [15:0] value;
```

```
    constraint c_not_zero { value != 0; }

    // Key property: adding 1 to a "pure run" value fills all lower bits
    // and then carries past the run.
    // (value | (value - 1)) is all-ones from LSB up to the top of the run.
    // XOR with (value | (value-1)) + 1 gives a power of two at the top+1.
    // We can express: value has one run iff ~value & (~value + 1) is
    // a power of two from above the run... but constraint solvers
    // struggle with this.
    //
    // Simpler declarative trick using three constraints:
    rand bit [15:0] lo_mask;     // 0...01...1 -- ones from bit 0 upward
    rand bit [15:0] hi_mask;     // 1...10...0 -- ones from bit N upward

    constraint c_lo_mask {
        // lo_mask must be of the form 0...01...1 (run at LSB or zero)
        lo_mask == 0 || (lo_mask & (lo_mask + 1)) == 0;
    }

    constraint c_hi_mask {
        // hi_mask must be of the form 1...10...0 (complement of lo_mask form)
        // i.e., ~hi_mask is of the form 0...01...1
        hi_mask == 16'hFFFF || ((~hi_mask) & ((~hi_mask) + 1)) == 0;
    }
    constraint c_value {
        // value = lo_mask & ~hi_mask
        // This gives exactly the bits that are in lo but not hi:
        // a run of 1s somewhere in the middle
        value == (lo_mask & ~hi_mask);
        value != 0;
    }
endclass

// --- Simplest correct approach for interviews ---
// Use approach (b): randomize index+size, build in post_randomize.
// This is the most readable, most efficient, and solver-friendly.
// For the interview, present (b) first, then mention (c) as alternative.
```

解释

这道题的关键洞察是：直接约束 value 的比特模式很难，

但约束"起始位置和长度"然后在 post_randomize 里构造就非常简单。

post_randomize() 是 randomize() 完成后自动调用的钩子函数，

在其中可以用 rand 字段计算出 non-rand 字段的值。

((1 << size) - 1) << index 是生成连续 1s mask 的标准位运算写法：

(1 << size) - 1 生成低 size 位全为1的掩码，再左移 index 位定位。

纯声明式约束（方案c）虽然可行但对求解器压力大，

实际工程中 index+size 方案更常用且更易理解。

## Q41. SV Constraint: rand bit Array with Odd Number of 1s

QUESTION

(a) Write a constraint for `rand bit array[N-1:0]` (fixed-size

packed array) such that the number of 1s in the array is odd.

What SV built-in function makes this elegant?

(b) What does `rand bit array[]` (with empty brackets) mean?

How does the constraint change compared to (a)?

(c) Implement a 2-to-1

ANSWER

```
// (a) Odd number of 1s in a fixed-size packed bit array
//
// Key insight: XOR of all bits = 1 iff the number of 1s is ODD.
// SV provides array.xor() which computes XOR reduction of all elements.

class odd_ones_fixed #(parameter N = 8);
  rand bit [N-1:0] array;

  constraint c_odd {
      // array.xor() returns 1 if number of 1s is odd, 0 if even
      array.xor() == 1'b1;
  }
endclass

// Examples:
// array = 8'b0000_0001 -> xor = 1 -> 1 one      -> ODD   (valid)
// array = 8'b0000_0011 -> xor = 0 -> 2 ones -> EVEN (invalid)
// array = 8'b0000_0111 -> xor = 1 -> 3 ones -> ODD       (valid)
// array = 8'b1111_1111 -> xor = 0 -> 8 ones -> EVEN (invalid)

// (b) rand bit array[] -- dynamic array
//
// Empty brackets [] declare a DYNAMIC array (size unknown at compile time).
// The size must be constrained before or alongside other constraints.
//
class odd_ones_dynamic;
  rand bit array[];          // dynamic array -- size not fixed

  constraint c_size {
      array.size() == 8;     // must constrain size explicitly
  }
  constraint c_odd {
      // For dynamic bit array, sum() with int promotion avoids overflow
      // .xor() also works on dynamic arrays
      array.xor() == 1'b1;

     }
 endclass
 //
 // Key differences from fixed-size:
 // Fixed    [N-1:0]: size fixed at compile time, no c_size needed
 // Dynamic []:        size randomized, MUST add size constraint,
 //                    can use inside{[min:max]} for variable sizes
```

解释

(a) array.xor() 是 SV 的数组归约（reduction）函数，

计算所有元素的 XOR，结果为1当且仅当1的个数为奇数，

这是检测奇偶性最简洁的写法，比 $countones(array) % 2 == 1 更优雅。

(b) 动态数组 [] 和固定数组 [N-1:0] 的核心区别：

固定数组的大小是类型的一部分，不需要约束；

动态数组的大小是运行时确定的，必须通过 c_size 约束，

否则 solver 不知道该分配多少元素，randomize() 会失败。

## Q42. SV Constraint: 300-Element Array, Values 0-5, Each >= 40 Times, No Consecutive 0s

QUESTION

Generate a 300-element integer array where:

- All values are in range [0, 5]

- Each value (0, 1, 2, 3, 4, 5) appears at least 40 times

- No two consecutive elements are both 0

(i.e. arr[i]==0 -> arr[i-1] != 0)

ANSWER

```
class no_consec_zeros #(parameter N = 300);
  rand int arr[N];

  // All elements in range [0:5]
  constraint c_range {
    foreach (arr[i]) arr[i] inside {[0:5]};
  }

  // Each value must appear at least 40 times
  // arr.sum() with (int'(item == V)) counts how many elements equal V
  constraint c_min_count {
    arr.sum() with (int'(item == 0)) >= 40;
    arr.sum() with (int'(item == 1)) >= 40;
    arr.sum() with (int'(item == 2)) >= 40;
    arr.sum() with (int'(item == 3)) >= 40;
    arr.sum() with (int'(item == 4)) >= 40;
    arr.sum() with (int'(item == 5)) >= 40;
  }

  // No two consecutive 0s:
  // If arr[i] == 0, then arr[i-1] must not be 0
  constraint c_no_consec_zeros {
    foreach (arr[i]) {
      if (i > 0)
        (arr[i] == 0) -> (arr[i-1] != 0);
    }
  }
endclass

// Notes:
// - 6 values * 40 minimum = 240 entries guaranteed; remaining 60 are free
// - The implication constraint (arr[i]==0) -> (arr[i-1]!=0) is checked
//   for every adjacent pair starting from index 1
// - int'(item == V) returns 1 if true, 0 if false, then sum() counts matches
// - Feasibility: N=300, 6 values each >=40, no consecutive 0s -- solver
//   can find solutions since 0s are at most 150 of 300 (every other slot)
//   and 40 zeros out of 300 is easily achievable with non-zero neighbors
```

解释

这道题综合考察三类约束的写法：范围约束、计数约束、相邻元素约束。

计数约束的关键语法：arr.sum() with (int'(item == V)) >= 40

这个写法把"数组中等于 V 的元素个数"转换成 sum，用 int' 避免位宽溢出。

相邻元素约束：foreach + if(i>0) + implication (->)，

(arr[i]==0) -> (arr[i-1]!=0) 表示"如果当前是0，前一个不能是0"。

注意 foreach 的 if(i>0) 是必须的，否则 i=0 时 arr[-1] 会越界。

可行性检查：6个值各至少40次共240个，剩余60个自由分配，

同时0最多占150个位置（每隔一个），40个0完全可行。

## Q43. SV Constraint: 300-Element Array, 0 May Repeat, Non-Zero Values Must Not Be Consecutive

QUESTION

(a) Same setup as the previous question (300 elements, values 0-5,

each at least 40 times), but the rule is reversed:

- 0 may appear consecutively (no restriction on 0s)

- Non-zero values (1-5) must NOT appear consecutively

(arr[i] != 0 && arr[i-1] != 0 is forbidden)

(b) Extension: expand the value range to [0, 100].

Rewrite the minimum-count constraint without writing

100 individual lines (use a loop or smarter approach).

ANSWER

```
// (a) Values 0-5, non-zero values must not be consecutive
class no_consec_nonzero #(parameter N = 300);
  rand int arr[N];

  constraint c_range {
    foreach (arr[i]) arr[i] inside {[0:5]};
  }

  constraint c_min_count {
    arr.sum() with (int'(item == 0)) >= 40;
    arr.sum() with (int'(item == 1)) >= 40;
    arr.sum() with (int'(item == 2)) >= 40;
    arr.sum() with (int'(item == 3)) >= 40;
    arr.sum() with (int'(item == 4)) >= 40;
    arr.sum() with (int'(item == 5)) >= 40;
  }

  // No two consecutive non-zero values:
  // If arr[i] != 0 AND arr[i-1] != 0, that is forbidden.
  // Equivalently: NOT (arr[i] != 0 && arr[i-1] != 0)
  // Which is:     arr[i] == 0 || arr[i-1] == 0
  constraint c_no_consec_nonzero {
    foreach (arr[i]) {
      if (i > 0)
        (arr[i] != 0) -> (arr[i-1] == 0);
    }
  }
endclass

// (b) Extension: values [0, 100], each value >= 1 occurrence
//     (requiring each of 101 values >= 40 times in 300 elements
//      is infeasible: 101 * 40 = 4040 > 300)
```

```
//     Realistic version: each value in [0,100] appears >= 1 time,
//     non-zero values (1-100) must not be consecutive.
//
// Key insight for the loop approach:
// Instead of 100 individual sum() lines, use a local variable
// and a foreach loop INSIDE the constraint block, OR use
// a helper array that tracks counts.
//
// Method 1: constraint with foreach loop over value range
class no_consec_nonzero_wide #(
  parameter N    = 300,
  parameter VMAX = 100
);
  rand int arr[N];

  constraint c_range {
    foreach (arr[i]) arr[i] inside {[0:VMAX]};
  }

  // Each value 0..VMAX must appear at least once
  // Use foreach over value space, not array space
  constraint c_min_count {
    foreach (arr[i]) {
      // This foreach iterates array indices -- use sum() per value
      // We cannot foreach over value space directly in constraints,
      // so use the "count via sum" approach in a loop at class level
      // with soft constraints generated in post_randomize validation,
      // OR use a smarter formulation:
      1; // placeholder -- see Method 2 below
    }
  }

  // Method 2: use a coverage-style approach with soft constraints
  // For large value ranges, generate a rand helper array of counts
  // and use it as a proxy -- but this is complex for a constraint.
  //
  // Practical solution for interview: use post_randomize to verify
  // and re-randomize if count constraint violated, OR reduce requirement
  // to "all values present" using a set-coverage approach:

  // Simpler version: just constrain range + no consecutive non-zeros
  // The "each value appears" constraint for large ranges is
  // typically handled outside the constraint solver (post_randomize check):
  constraint c_range_only {
    foreach (arr[i]) arr[i] inside {[0:VMAX]};
  }

  constraint c_no_consec_nonzero {
    foreach (arr[i]) {
      if (i > 0)
        (arr[i] != 0) -> (arr[i-1] == 0);
    }
  }

  // Post-randomize: verify each value in [1:VMAX] appears at least once
  // For large VMAX, this is more practical than constraint-solver approach
  function void post_randomize();
    int count[int];
    foreach (arr[i]) count[arr[i]]++;
    for (int v = 0; v <= VMAX; v++) begin
      if (!count.exists(v) || count[v] == 0) begin
        // Force at least one occurrence by replacing a safe position
        // (a position where arr[i-1]!=0 and arr[i+1]!=0 is not needed
```

```
        //  since v might be 0, or if v!=0, neighbors must be 0)
        // Simple approach: mark for re-randomize
        `uvm_warning("CONSTRAINT",
          $sformatf("Value %0d missing, consider re-randomizing", v))
      end
    end
  endfunction

  // --- Clean alternative: use a separate rand "used_values" bitvector ---
  // rand bit used[VMAX+1];   // used[v] = 1 means value v must appear
  // constraint c_all_used {
  //   foreach (used[v]) used[v] == 1;  // all values must be used
  //   foreach (used[v]) if (used[v])
  //     arr.sum() with (int'(item == v)) >= 1;
  // }
  // This is still O(VMAX) constraints but avoids writing them by hand.
endclass

// Summary of the key constraint for non-consecutive non-zeros (both parts):
//
// No consecutive 0s:
//   (arr[i] == 0) -> (arr[i-1] != 0)
//
// No consecutive non-zeros:
//   (arr[i] != 0) -> (arr[i-1] == 0)
//
// The two are duals of each other -- just flip the == 0 / != 0 condition.
```

解释

(a) 非零值不能连续的约束是 Q28 的对称版本。

(arr[i] != 0) -> (arr[i-1] == 0) 表示"如果当前不是0，前一个必须是0"。

换句话说，两个非零值之间必须至少有一个0作为分隔。

(b) 范围扩展到 0-100 的关键问题：

101个值各至少40次 = 4040，远超数组长度300，所以"各至少40次"不可行，

改为"各至少出现1次"。

写法上不能直接用 foreach 遍历值域（约束块不支持以值为循环变量），

实际工程解决方案：

1. 用 post_randomize 检查并标记缺失值（最实用）

2. 用 rand bit used[] 辅助数组 + sum() 约束（可以，但仍然 O(VMAX)）

3. 对于面试：说明大范围时 constraint solver 的局限性，

提出 post_randomize 验证 + 重新随机化的混合方案。

## Q44. SV Constraint: Memory Access with Excluded Range

QUESTION

Randomize a memory access where:

- address and size are random

- address + size <= 4096 (stay within 4KB space)

- The access must NOT overlap with an excluded range

[range_lower_bound, range_upper_bound]

- All variables are 32-bit

Given answer to review and correct:

```
rand bit [31:0] address;
rand int        size;
constraint c {
  address + size <= 4096;
  address > range_upper_bound || address + size <= range_lower_bound;
}
```

ANSWER

```
// The given answer is logically correct in structure but has issues:
// 1. 'size' should be rand int unsigned (size must be > 0)
// 2. Need to ensure size >= 1 (zero-size access is meaningless)
// 3. range_lower_bound and range_upper_bound need to be non-rand inputs
// 4. The non-overlap condition is correct:
//    "new region does NOT overlap excluded region" iff:
//    new_end <= excl_start  (new region is entirely before excluded)
//    OR
//    new_start >= excl_end  (new region is entirely after excluded)

class mem_access;
  // Inputs: excluded range (set before randomize)
  bit [31:0] range_lower_bound;  // start of excluded range (inclusive)
  bit [31:0] range_upper_bound;  // end   of excluded range (exclusive)

  rand bit [31:0] address;
  rand int unsigned size;        // unsigned: size must be >= 1

  constraint c_bounds {
    size >= 1;
    address + size <= 4096;      // stay within 4KB
  }

  constraint c_exclude {
    // Access [address, address+size) must not overlap [lower, upper)
    // No overlap iff: address >= upper  OR  address+size <= lower
    address >= range_upper_bound ||
    address + size <= range_lower_bound;
  }
endclass

// Overlap detection reminder:
// Two ranges [a, a+s) and [L, U) OVERLAP iff:
//   a < U  AND  a+s > L
// They DO NOT overlap iff:
//   a >= U  (new starts at or after excluded ends)
//   OR a+s <= L  (new ends at or before excluded starts)
// The constraint uses the non-overlap condition directly.

// Example:
//   range_lower = 0x100, range_upper = 0x200
//   Valid:   address=0x000, size=0x100  (ends exactly at lower bound)
//   Valid:   address=0x200, size=0x100  (starts exactly at upper bound)
//   Invalid: address=0x080, size=0x100  (overlaps [0x100, 0x180))
//   Invalid: address=0x1C0, size=0x100  (overlaps [0x1C0, 0x200))
```

解释

内存访问约束的核心是区间重叠判断。

两个区间不重叠的条件（取反即重叠）：

新区间完全在排除区间之前：address + size <= range_lower_bound

新区间完全在排除区间之后：address >= range_upper_bound

用 OR 连接两个不重叠条件即可。

注意 size 要用 unsigned 且 >= 1，避免零长度或负长度访问。

面试考点：能快速推导出不重叠条件，不要死记硬背。

## Q45. SV Constraint: Repeatedly Randomize Non-Overlapping Memory Regions Until Full

QUESTION

Follow-up from the previous question: remove the excluded range.

Now repeatedly randomize address+size within [0, 4095], each new

region must not overlap any previously selected region.

Keep going until no valid non-overlapping region can fit.

Use a queue to track used regions and a helper function to check

for overlaps in post_randomize.

ANSWER

```
typedef struct {
  bit [31:0] addr;
  int unsigned size;
} region_t;

class mem_region_picker;
  rand bit [31:0] address;
  rand int unsigned size;

  // Queue of already-allocated regions
  region_t used_regions[$];

  constraint c_bounds {
    size >= 1;
    address + size <= 4096;
  }

  // Constraint: no overlap with any already-used region
  // For each used region, the new [address, address+size) must not overlap
  // [used.addr, used.addr+used.size).
  constraint c_no_overlap {
    foreach (used_regions[i]) {
      address >= (used_regions[i].addr + used_regions[i].size) ||
      (address + size) <= used_regions[i].addr;
    }
  }

  // post_randomize: record the new region in the queue
  function void post_randomize();
    region_t r;
    r.addr = address;
    r.size = size;
    used_regions.push_back(r);
  endfunction

  // Helper: check if a candidate region overlaps any used region
  function bit has_overlap(bit [31:0] addr, int unsigned sz);
```

```
    foreach (used_regions[i]) begin
      // Overlap: not (entirely before OR entirely after)
      if (!(addr >= used_regions[i].addr + used_regions[i].size ||
            addr + sz <= used_regions[i].addr))
        return 1;  // overlap found
    end
    return 0;
  endfunction

  // Helper: check if ANY valid 1-byte region can still be placed
  // (termination condition: is there at least one free address?)
  function bit any_free_address();
    for (int a = 0; a < 4096; a++) begin
      if (!has_overlap(a, 1)) return 1;
    end
    return 0;
  endfunction

  function void reset();
    used_regions.delete();
  endfunction
endclass

// Controller: keep randomizing until no space left
class mem_allocator;
  mem_region_picker picker;
  int attempt_limit = 1000;

  function new();
    picker = new();
  endfunction

  task run();
    int attempts;
    picker.reset();

    while (picker.any_free_address()) begin
      attempts = 0;
      while (attempts < attempt_limit) begin
        if (picker.randomize()) begin
          // post_randomize already added to queue
          $display("Allocated: addr=0x%0h size=%0d",
                   picker.address, picker.size);
          break;
        end
        attempts++;
      end
      if (attempts == attempt_limit) begin
        $display("Could not find non-overlapping region, stopping.");
        break;
      end
    end

    $display("Total regions allocated: %0d", picker.used_regions.size());
  endtask
endclass

// Key design decisions:
// 1. foreach in constraint grows dynamically as used_regions grows.
//    Each new randomize() call adds one more constraint term.
// 2. post_randomize adds to the queue AFTER successful randomization.
// 3. any_free_address() scans 0-4095 for any uncovered 1-byte slot.
//    This is O(4096) but space is small -- acceptable.
```

```
// 4. attempt_limit guards against solver being stuck on infeasible
//    constraints (when remaining free space exists but is fragmented).
```

解释

这道题是 Q3 的扩展，核心难点是约束随着 queue 增长而动态扩展。

foreach 在 constraint 块中可以遍历动态 queue——每次 randomize() 前，

used_regions 有多少元素，就有多少条不重叠约束。

post_randomize 是在约束求解成功后自动调用的，是记录结果的标准位置。

终止条件：any_free_address() 扫描整个空间，如果连1字节都放不下就停止。

attempt_limit 防止碎片化导致 solver 反复失败卡死。

## Q46. SV Constraint: Symmetric Array Using Helper Function

QUESTION

Randomize a dynamic integer array such that it is symmetric

(palindrome): arr[i] == arr[size-1-i] for all i.

The interviewer's approach uses a helper function called in the

constraint. Review and fix the given code:

function bit helper(int my_array[]);

```
int left = 0;
int right = my_array.size() - 1;
while (left <= right) begin
  if (my_array[left] != my_array[right]) return 0;
  // BUG: left and right never advance -- infinite loop!
end
return 1;
```

endfunction

constraint c {

my_array.size() > 0;

helper(my_array) == 1;

}

ANSWER

```
// Bug in given code: the while loop never advances left/right,
// causing an infinite loop. Fix: add left++ and right-- inside the loop.

// Fixed helper function:
function automatic bit helper(int my_array[]);
  int left  = 0;
  int right = my_array.size() - 1;
  while (left <= right) begin
    if (my_array[left] != my_array[right]) return 0;
    left++;   // advance inward
    right--;  // advance inward
  end
  return 1;
endfunction
```

```
// Class with constraint:
class sym_array;
  rand int my_array[];

  constraint c_size {
    my_array.size() inside {[1:16]};  // reasonable size bound
  }

  constraint c_sym {
    // Constraint solver calls helper() to verify symmetry.
    // IMPORTANT: calling a function in a constraint limits the solver.
    // The solver must find values that make helper() return 1,
    // which is only possible because the solver can try many combinations.
    // For large arrays this becomes very slow; post_randomize is better.
    helper(my_array) == 1;
  }
endclass

// Better approach: use post_randomize to enforce symmetry directly.
// This is faster because we generate free values and then mirror them,
// rather than relying on the solver to guess symmetric arrays.
class sym_array_fast;
  rand int my_array[];
  rand int half[];       // randomize only the first half

  constraint c_size {
    half.size() inside {[1:8]};
  }

  function void post_randomize();
    int n = half.size();
    my_array = new[2 * n];
    for (int i = 0; i < n; i++) begin
      my_array[i]         = half[i];
      my_array[2*n-1-i]   = half[i];   // mirror
    end
    // Result: [a, b, c, c, b, a] for n=3
  endfunction
endclass

// For odd-length palindrome (e.g. [a, b, c, b, a]):
class sym_array_odd;
  rand int half[];       // first half
  rand int middle;       // center element
  rand int my_array[];

  constraint c_size { half.size() inside {[1:7]}; }

  function void post_randomize();
    int n = half.size();
    my_array = new[2*n + 1];
    for (int i = 0; i < n; i++) begin
      my_array[i]       = half[i];
      my_array[2*n-i]   = half[i];
    end
    my_array[n] = middle;  // center
  endfunction
endclass
```

解释

原始代码有一个经典 bug：while 循环里忘记移动 left 和 right 指针，

导致无限循环（left=0, right=n-1 永远不变，条件永远成立）。

修复只需在循环体末尾加 left++ 和 right--。

在 constraint 里调用 helper 函数的注意事项：

constraint solver 必须找到让 helper() 返回 1 的值，这对于大数组非常低效，

因为 solver 需要随机猜测直到满足对称条件。

更好的方案是 post_randomize 里直接构造对称数组：

只随机化一半元素，然后镜像到另一半，简单且高效。

这是"用 rand 生成一半，post_randomize 推导另一半"的标准模式。

## Q47. SV Constraint: Constraint Inheritance and constraint_mode()

QUESTION

Given:

```
class base;
  rand int a;
  constraint c { (a >= 0 && a <= 99) || (a >= 201 && a <= 299); }
endclass

class child extends base;
  constraint c_child { a >= 100 && a <= 199; }
endclass
```

(a) What happens when you randomize a child object? Why?

(b) If you remove (comment out) c_child from child, how do you

make 'a' completely unconstrained (ignoring the base class

```
constraint c)?
```

(c) Write the correct code to disable the base class constraint.

ANSWER

```
// (a) Randomizing child: FAILS (constraint contradiction)
//
// Both constraints are active simultaneously on child objects:
//   From base: a in [0,99] OR a in [201,299]
//   From child: a in [100,199]
//
// The intersection of these two sets is EMPTY:
//   ([0,99] OR [201,299]) AND [100,199] = {} (no valid value)
//
// Result: randomize() returns 0 (failure), 'a' is not updated.
// The constraint solver cannot find any value satisfying both.
//
// child c = new();
// if (!c.randomize())
//   $display("Randomize failed -- constraint contradiction");

// (b) How to disable the base class constraint 'c':
//
// After removing c_child, child has no own constraints,
// but base class constraint 'c' is still inherited and active.
//
```

```
// To disable it, use constraint_mode(0) on the specific constraint:
//   object.c.constraint_mode(0);
//
// constraint_mode() is called as a method ON the constraint BY NAME.
// The syntax is: <object_handle>.<constraint_name>.constraint_mode(0);
// 0 = disable, 1 = enable (default)

// (c) Correct code:
class child_free extends base;
  // No additional constraints -- inherits base::c
  // c_child is removed / not present
endclass

// Usage:
// child_free cf = new();
//
// Disable the inherited base constraint by name:
// cf.c.constraint_mode(0);   // 'c' is the name defined in base class
//
// Now randomize: 'a' is fully unconstrained (any 32-bit int value)
// assert(cf.randomize());
// $display("a = %0d", cf.a);  // any value, no restriction
//
// To re-enable the base constraint:
// cf.c.constraint_mode(1);

// Common mistakes:
//   WRONG: cf.constraint_mode(0);
//          -- This disables ALL constraints on cf (correct syntax but
//             would also disable any other constraints you want to keep)
//   WRONG: cf.c_child.constraint_mode(0);
//          -- c_child doesn't exist in child_free; compile error
//   CORRECT: cf.c.constraint_mode(0);
//          -- disables only the named constraint 'c' from base class

// Alternative: use rand_mode(0) to disable randomization of 'a' entirely
// cf.a.rand_mode(0);   // 'a' is no longer randomized, keeps its current value
// This is different from constraint_mode -- rand_mode affects the variable,
// constraint_mode affects the constraint block.
```

解释

这道题考察 SV 约束继承和 constraint_mode() 的正确用法。

约束冲突：子类和父类的约束同时有效，条件域互斥 -> randomize() 失败返回 0。

constraint_mode(0)：按名字关闭指定约束块。

语法：object.constraint_name.constraint_mode(0)

其中 constraint_name 是在 class 中用 constraint 关键字声明的名字（本题中为 c）。

注意区分：

constraint_mode(0)：关闭约束块，变量仍然随机化，只是不受这个约束限制。

rand_mode(0)：关闭变量的随机化，变量保持当前值不变。

两者效果不同，面试时要能清楚区分。

## SystemVerilog — Assertions (SVA)

## Q48. SVA: Analyze a Complex Property

QUESTION

Explain what the following SVA property checks:

property p1;

@(posedge clk) disable iff (!rst_n)

(req == rsp) |=> (req != rsp) |-> ##[1:1000] (req == rsp);

```
endproperty
```

ANSWER

```
// Step-by-step breakdown:
  //
  // Antecedent: (req == rsp)
  //    -> Trigger when req equals rsp in the current cycle.
  //
  // |=> (non-overlapping implication, advance 1 cycle)
  //    -> In the NEXT cycle, check the consequent.
  //
  // (req != rsp) |-> ##[1:1000] (req == rsp)
  //    -> In that next cycle, IF req != rsp (overlapping implication),
  //       THEN within 1 to 1000 cycles after that, req must equal rsp again.
  //
  // Full semantics:
  // "Whenever req equals rsp, then in the very next cycle req must differ
  //   from rsp; and once they differ, they must become equal again within
  //   1 to 1000 clock cycles."
  //
  // Practical interpretation:
  // This checks a handshake/toggling protocol where:
  //    - req==rsp is the "idle" or "acknowledged" state
  //    - After acknowledgment, the signals must diverge (req!=rsp)
  //       in the next cycle (e.g. a new request is issued)
  //    - The new request must be acknowledged (req==rsp) within 1000 cycles

  // Potential issue with this property:
  // If req==rsp in cycle N, and req==rsp again in cycle N+1,
  // the inner |-> antecedent (req!=rsp) is FALSE -> vacuous pass.
  // The property only enforces the 1000-cycle response when req!=rsp
  // is actually observed in cycle N+1.
```

解释

这道题考查 SVA 的嵌套 implication 和时序运算符的组合解读。

|=> 是非重叠蕴含（等价于 |-> ##1），把检查推迟一拍。

|-> 是重叠蕴含，在同一拍开始检查 consequent。

##[1:1000] 是在 1 到 1000 拍内必须满足。

整体描述了一个"确认->分离->再确认"的握手协议，

面试时能逐层拆解每个算符的含义是关键。

## Q49. SVA: Glitch Detection -- Signal HIGH for Less Than 2.5ns

QUESTION

Write an SVA assertion to detect a glitch on signal 'sig'.

A glitch is defined as: sig rises to HIGH and then falls back

LOW within 2.5ns (i.e. the HIGH pulse width is less than 2.5ns).

The assertion should fire when such a glitch is detected.

Assume the design uses a 1ns time unit.

ANSWER

```
// Approach: use $rose to detect the rising edge, then check
// if sig falls LOW within 2.5ns (i.e. within 2 time units
// when time unit = 1ns, sig goes low at +1ns or +2ns).
//
// A glitch means: sig rises, then within < 2.5ns it is LOW again.
// With 1ns time unit, "less than 2.5ns" means at most 2 cycles later.
//
// Method 1: time-based assertion using ##[1:2] (for integer cycles)
property p_no_glitch;
  @(posedge clk) disable iff (!rst_n)
  // If sig rises, it must stay HIGH for at least 3 cycles
  // (i.e. NOT fall within 1 or 2 cycles after rising)
  $rose(sig) |-> sig[*3];
endproperty
assert property (p_no_glitch)
  else `uvm_error("GLITCH", "Glitch detected: sig HIGH < 3 cycles")

// Method 2: explicitly detect the glitch condition
// Glitch = sig rises and is LOW within 2 cycles (< 2.5ns at 1ns/unit)
property p_glitch_detect;
  @(posedge clk) disable iff (!rst_n)
  $rose(sig) |-> ##[1:2] $fell(sig);
endproperty
// This property is TRUE when a glitch OCCURS -- use assert to flag it
// as an error, or cover to check it is exercised in directed tests.
assert property (!p_glitch_detect)
  else `uvm_error("GLITCH", "Glitch: sig pulsed HIGH for < 2.5ns")

// Method 3: for non-clocked (pure-time) assertion with $time
// Use an event-based checker outside clock domain:
always @(posedge sig) begin : glitch_check
  automatic time rise_time = $time;
  @(negedge sig);
  if (($time - rise_time) < 2500ps)  // 2.5ns = 2500ps
    `uvm_error("GLITCH", $sformatf(
      "Glitch at t=%0t: HIGH duration=%0t", rise_time, $time-rise_time))
end

// Which method to use:
//   Clock-domain designs: Method 1 or 2 (cycle-based, portable)
//   Analog/mixed-signal or gate-level: Method 3 ($time, ps resolution)
//
// Method 2 explanation:
//   $rose(sig) |-> ##[1:2] $fell(sig)
//   "After sig rises, if it falls within 1 or 2 clock cycles,
//    that is a glitch (HIGH pulse < 2.5ns assuming 1ns/cycle)"
//   The outer !p_glitch_detect assertion fires when this pattern occurs.
```

解释

毛刺（glitch）检测是数字验证中常见的 SVA 题，考察时间相关的断言写法。

核心思路：$rose(sig) 检测上升沿，然后用 ##[1:2] 检查是否在 2 个周期内就下降。

如果下降了（$fell(sig)），说明高电平持续时间 < 2.5ns，是毛刺。

三种方法的选择取决于设计层次：

RTL 仿真用基于时钟的 Method 1/2（##N cycle-based），简洁可靠；

门级或混合信号仿真用 Method 3（$time 精确到 ps），能处理非整数周期的毛刺。

面试要点：能说出 $rose/$fell 的用法，以及 |-> 和 ##[1:2] 的语义。

## Q50. SVA: valid Must Stay High Until ready

QUESTION

Write an SVA assertion: after valid rises, valid must remain

HIGH until ready is also HIGH (inclusive).

ANSWER

```
property valid_until_ready;
     @(posedge clk) disable iff (!rst_n)
     $rose(valid) |-> valid until_with ready;
  endproperty
  assert property (valid_until_ready)
     else $error("valid de-asserted before ready!");

  // 'until_with' (strong until):
  // valid must hold TRUE on every cycle from the trigger cycle
  // until AND INCLUDING the cycle where ready is TRUE.
  // The property also requires that ready eventually becomes TRUE.
  //
  // Compare with 'until' (weak until):
  // valid must hold while ready is not yet TRUE, but ready is NOT
  // required to eventually become TRUE (vacuously passes if ready
  // never asserts).
  //
  // Timeline example:
  // clk    : __|--|__|--|__|--|__|--|
  // valid : 0    1   1   1   1   0     <- valid de-asserts too early!
  // ready : 0    0   0   1   0         <- ready seen at cycle 4
  // Assertion passes if valid stays high through cycle 4 (when ready=1)
```

解释

until_with 是 SVA 中的"强直到"运算符，要求：

1. 左侧表达式（valid）在每一拍都为真，直到右侧（ready）为真的那一拍（含）。

2. 右侧最终必须变为真（否则 assertion 失败）。

与 until（弱直到）的区别：until 不要求右侧最终发生。

这是 AXI/APB 等总线协议验证中非常常见的 assertion 模式。

## Q51. SVA: Sequence Assertion -- cat Followed by 3 or 4 dogs

QUESTION

Write an SVA assertion: when signal 'cat' is HIGH in cycle N,

signal 'dog' must be HIGH for exactly 3 or 4 consecutive cycles

starting from cycle N+1, and then 'dog' must go LOW.

ANSWER

```
// Requirement: cat -> ##1 (dog for exactly 3 or 4 cycles) ##1 !dog

 property p_cat_3or4_dogs;
     @(posedge clk) disable iff (!rst_n)
     $rose(cat) |-> ##1 (dog [*3:4]) ##1 !dog;
 endproperty

 assert property (p_cat_3or4_dogs)
     else $error("cat not followed by exactly 3 or 4 dogs!");

 // Syntax breakdown:
 // $rose(cat)      : trigger when cat rises (0->1 transition)
 //                   Use cat instead of $rose(cat) if cat is level-sensitive
 // |->             : overlapping implication (check starts same cycle as trigger)
 // ##1             : advance 1 clock cycle
 // dog [*3:4]      : consecutive repetition -- dog HIGH for 3 or 4 cycles
 //                   [*3] = exactly 3,    [*3:4] = 3 or 4 consecutive
 // ##1 !dog        : one cycle after the dog sequence, dog must be LOW
 //                   (this enforces the "exactly" -- without it, [*3:4]
 //                      would allow more than 4 dogs to follow)

 // Timeline (3-dog case):
 // clk : __|--|__|--|__|--|__|--|__|--|
 // cat : 0     1    0     0   0   0   0
 // dog : 0     0    1     1   1   0   0    <- 3 consecutive after cat
 //                                  ^ !dog asserted here (cycle +5)

 // Timeline (4-dog case):
 // cat : 0     1    0     0   0   0   0   0
 // dog : 0     0    1     1   1   1   0   0    <- 4 consecutive
 //                                      ^ !dog here (cycle +6)

 // Common mistake: omitting ##1 !dog
 // dog [*3:4] alone allows 3, 4, 5, 6... dogs (no upper bound)
 // ##1 !dog after [*3:4] forces termination after at most 4.

 // Cover property to verify the sequence is exercised:
 cover property (@(posedge clk) $rose(cat) |-> ##1 dog[*3:4] ##1 !dog);
```

解释

连续重复 [*N:M] 是 SVA 常考语法，面试中要能快速写出。

[*3] = 恰好 3 个连续周期为高。

[*3:4] = 3 或 4 个连续周期为高。

##1 !dog 是"最多 4 个"的关键：没有它 [*3:4] 只保证至少 3 个，

实际可以有更多，不满足"恰好"的语义。

$rose(cat) vs cat：$rose 只在上升沿触发，cat 每个高电平周期都触发。

根据题意选择——这里用 $rose 更合理（cat 拉高事件触发一次）。

## Q52. SVA: Clock Period Verification -- Within 5% of Expected Period

QUESTION

Write an SVA assertion to verify that the clock period stays

within 5% of an expected_clock_period value.

The clock is NOT guaranteed to be stable (jitter is present).

The interviewer's answer uses $stime to measure the actual period:

```
int last_rising_edge = -1;
always @(posedge clk) begin
  last_rising_edge = $stime;
end

property my_property;
  @(posedge clk)
  last_rising_edge != -1 |->
    (100 * ($stime - last_rising_edge) / expected_period) < 5;
endproperty
assert property (my_property);
```

(a) Identify and fix the bugs in the given code.

(b) Explain how the corrected assertion works.

ANSWER

```
// (a) Bugs in the given code:
//
// Bug 1: Sampling race condition
//   'last_rising_edge' is updated by an always block at posedge clk.
//   The SVA property also samples at posedge clk.
//   In the same time step, the always block WRITES last_rising_edge
//   while the SVA property READS it -- this is a race condition.
//   The SVA sampler may read the OLD value (before this rising edge)
//   or the NEW value (current rising edge), giving non-deterministic results.
//   Fix: use $past() to explicitly read the PREVIOUS rising edge time,
//   or store the previous edge time in a separate variable.
//
// Bug 2: Integer arithmetic truncation
//   (100 * ($stime - last_rising_edge) / expected_period) < 5
//   With integer division, 4.9% error rounds to 4 -> passes.
//   But 5.0% exactly = 5 -> assertion fires (correct).
//   The intent is "less than 5%", so the formula should check
//   that the deviation is within 5%, meaning the actual period
//   must be in [95%, 105%] of expected. The formula only checks
//   ONE direction (period too short or the difference is <5%).
//   Fix: check both upper and lower bounds.
//
// Bug 3: The formula checks relative error incorrectly
//   (actual - expected) / expected * 100 < 5
//   gives the percentage deviation, but $stime - last_rising_edge
//   is the CURRENT period, not the difference from expected.
//   The correct formula:
//   deviation = |actual_period - expected_period|
//   percentage = 100 * deviation / expected_period
//   must be < 5.

// Corrected implementation:
longint last_rising_edge = -1;  // use longint for $stime (64-bit)
longint prev_rising_edge = -1;  // previous edge (to avoid race)

always @(posedge clk) begin
  prev_rising_edge = last_rising_edge;  // save previous BEFORE updating
  last_rising_edge = $stime;
end
```

```
// Helper function for the check (avoids inline absolute value)
function automatic bit period_ok(
  longint actual_period,
  longint exp_period
);
  longint deviation;
  if (actual_period > exp_period)
    deviation = actual_period - exp_period;
  else
    deviation = exp_period - actual_period;
  // Check: deviation / exp_period < 5% => 100*deviation < 5*exp_period
  return (100 * deviation < 5 * exp_period);
endfunction

property p_clock_jitter;
  @(posedge clk)
  // Only check after we have seen at least 2 rising edges
  (prev_rising_edge != -1) |->
    period_ok($stime - prev_rising_edge, expected_period);
endproperty
assert property (p_clock_jitter)
  else `uvm_error("CLK", $sformatf(
    "Clock period %0t is outside 5%% of expected %0t",
    $stime - prev_rising_edge, expected_period))

// How it works:
// At each posedge clk:
//   prev_rising_edge holds the time of the PREVIOUS posedge (no race)
//   $stime is the current posedge time
//   actual_period = $stime - prev_rising_edge
//   Check: |actual - expected| / expected < 5%
//   Multiply through by 100*expected to avoid floating point:
//   100 * |actual - expected| < 5 * expected
```

解释

这道题考察 SVA 与时间相关的高级用法。

原始代码有三个关键问题：

1. 竞争条件：always @(posedge clk) 和 SVA @(posedge clk) 在同一时间步

同时读写 last_rising_edge，结果不确定。

修复：用两个变量分离当前边沿和前一个边沿，always 里先保存再更新。

2. 整数截断：/ expected_period 在整数域做会丢失精度。

修复：把 < 5% 转换成 100*deviation < 5*expected，避免除法。

3. 单向检查：原始代码可能只检查了一个方向。

修复：用绝对值（分支或函数）同时检查 period 偏大和偏小两种情况。

面试要点：$stime 返回 64-bit longint（不是 int），要注意类型。

## Q53. SVA: Clock Period Check When expected_clock_period is Async

QUESTION

Follow-up: expected_clock_period is driven asynchronously and

may change at any time, NOT synchronized to clk.

How do you ensure the assertion uses the correct expected period

for each clock cycle measurement?

ANSWER

```
// Problem: if expected_clock_period changes between two rising edges,
// which value should we compare against?
// Answer: the value of expected_clock_period that was valid AT THE
// RISING EDGE when we started measuring (i.e. at prev_rising_edge).
//
// Solution: sample (latch) expected_clock_period at each rising edge.
// This creates a registered copy that is stable throughout the
// measurement window between two consecutive rising edges.

longint last_rising_edge     = -1;
longint prev_rising_edge     = -1;
longint sampled_exp_period   = -1;  // expected period sampled at prev edge

always @(posedge clk) begin
  // Step 1: save previous edge time and its sampled expected period
  prev_rising_edge   = last_rising_edge;
  // Step 2: update current edge time
  last_rising_edge   = $stime;
  // Step 3: sample expected_clock_period NOW (at this rising edge)
  // This value will be used to check the NEXT clock period
  sampled_exp_period = expected_clock_period;
end

// The assertion now uses sampled_exp_period (captured at prev edge)
// instead of the live expected_clock_period signal.
property p_clock_jitter_async;
  @(posedge clk)
  (prev_rising_edge != -1 && sampled_exp_period > 0) |->
    period_ok($stime - prev_rising_edge, sampled_exp_period);
endproperty
assert property (p_clock_jitter_async);

// Why this works:
//   At each posedge clk, we snapshot expected_clock_period.
//   The NEXT rising edge measures the actual period that elapsed
//   and compares it against the snapshot from the PREVIOUS edge.
//   Even if expected_clock_period changes between the two edges,
//   the comparison uses the value that was valid when the period started.
//
// Alternative: use $past() in the SVA property
property p_using_past;
  @(posedge clk)
  (prev_rising_edge != -1) |->
    period_ok(
      $stime - prev_rising_edge,
      $past(expected_clock_period, 1, , @(posedge clk))
    );
endproperty
// $past(expr, n, ...) returns the value of expr n clock cycles ago.
// $past(expected_clock_period, 1) = value at the previous posedge clk.
// This is cleaner than the manual latching approach for simple cases.
```

解释

异步信号同步到时钟域的问题在 SVA 里的解决思路和 RTL 一样：

在时钟边沿采样一拍，用采样后的寄存器值代替实时信号。

两种方案：

1. 手动采样：always @(posedge clk) 里显式保存 expected_clock_period

到 sampled_exp_period，下一拍用这个采样值比较，简单直观。

2. $past()：在 SVA property 里用 $past(expected_clock_period, 1)

获取上一个时钟沿的值，更简洁，是 SVA 标准写法。

面试要点：能说出"异步信号先在时钟边沿采样，再在同时钟域里使用"

这个核心原则，和 CDC 同步器的思想是一样的。

## Q54. SVA: APB Protocol Assertion -- PSEL to PENABLE to PREADY

QUESTION

Review and explain the following SVA assertion for an APB

(Advanced Peripheral Bus) handshake:

@(posedge clk)

$rose(psel) |-> ##1 $rose(penable) ##0 (psel & penable)[*0:$] ##0 $rose(pready);

(a) What does this assertion check?

(b) Is it correct for the APB protocol? Identify any issues.

(c) Write a corrected and complete APB assertion.

ANSWER

```
// APB protocol handshake (simplified):
// Phase 1 (Setup):   PSEL rises, PENABLE=0. Address/control is presented.
// Phase 2 (Access):  Next cycle: PENABLE rises (while PSEL stays HIGH).
//                    Slave asserts PREADY when ready.
// Transfer done:     When PSEL & PENABLE & PREADY are all HIGH.
//
// (a) What the given assertion checks:
//   After PSEL rises:
//   ##1 $rose(penable):           1 cycle later, PENABLE rises
//   ##0 (psel & penable)[*0:$]:   then both PSEL and PENABLE stay HIGH
//                                 for 0 or more cycles (wait states)
//   ##0 $rose(pready):            then PREADY rises
//
// (b) Issues with the given assertion:
//
// Issue 1: $rose(pready) at the end
//   PREADY may already be HIGH before the transaction (some slaves
//   have PREADY tied HIGH by default). $rose() fires only on 0->1.
//   If PREADY is already HIGH when PENABLE rises, the assertion fails.
//   Fix: use 'pready' instead of '$rose(pready)' to check PREADY is HIGH.
//
// Issue 2: (psel & penable)[*0:$] allows 0 wait cycles
//   This is correct for APB -- 0 wait states means PREADY is HIGH
//   the same cycle PENABLE rises. The [*0:$] is intentional.
//
// Issue 3: The assertion only checks SETUP to ACCESS transition.
//   It does not verify that PSEL stays HIGH during the entire transfer,
//   or that PENABLE drops after PREADY.
//
// Issue 4: Missing disable iff for reset

// (c) Corrected APB assertion:
//
// Rule 1: After PSEL rises, PENABLE must rise exactly 1 cycle later
property p_apb_setup_to_access;
```

```
  @(posedge clk) disable iff (!rst_n)
  $rose(psel) |-> ##1 penable;
endproperty
assert property (p_apb_setup_to_access)
  else `uvm_error("APB", "PENABLE did not rise 1 cycle after PSEL")

// Rule 2: PSEL must stay HIGH while PENABLE is HIGH (no early deassertion)
property p_apb_psel_stable;
  @(posedge clk) disable iff (!rst_n)
  (psel && penable) |-> psel;
endproperty
assert property (p_apb_psel_stable)
  else `uvm_error("APB", "PSEL deasserted while PENABLE still HIGH")

// Rule 3: Once PENABLE rises, hold until PREADY (allow wait states)
//         When PREADY is HIGH, transfer completes this cycle.
property p_apb_transfer_complete;
  @(posedge clk) disable iff (!rst_n)
  $rose(penable) |-> (psel && penable)[*1:$] ##0 (psel && penable && pready);
endproperty
assert property (p_apb_transfer_complete)
  else `uvm_error("APB", "APB transfer did not complete correctly")

// Rule 4: After PREADY, PENABLE drops next cycle
property p_apb_penable_drop;
  @(posedge clk) disable iff (!rst_n)
  (psel && penable && pready) |=> !penable;
endproperty
assert property (p_apb_penable_drop)
  else `uvm_error("APB", "PENABLE did not drop after PREADY")

// Combined: the key fix to the original assertion
// Replace $rose(pready) with pready:
property p_apb_corrected;
  @(posedge clk) disable iff (!rst_n)
  $rose(psel) |->
    ##1 $rose(penable)            // PENABLE rises 1 cycle after PSEL
    ##0 (psel && penable)[*1:$]   // both hold for 1+ cycles
    ##0 pready;                   // until PREADY is HIGH (not just rising)
endproperty
assert property (p_apb_corrected);
```

解释

APB 协议是面试最常考的总线协议之一，三个阶段要熟记：

1. Setup phase：PSEL=1, PENABLE=0，地址/控制信号稳定。

2. Access phase：PENABLE=1（PSEL 保持1），等待 PREADY。

3. Transfer done：PSEL & PENABLE & PREADY 同时为1，传输完成。

原始断言的关键错误：$rose(pready) 只检测上升沿，

但很多 APB slave 的 PREADY 默认为高（no-wait-state），

在 PENABLE 拉高的同一个周期 PREADY 已经是高了，不会有上升沿。

应该用 pready（电平）而不是 $rose(pready)（边沿）。

另外，disable iff (!rst_n) 是必须的，否则复位期间信号的随机状态

可能触发误报。面试时要主动提到 disable iff。

## Q55. SVA: B Toggles 2 Cycles After A Toggles

QUESTION

Write an SVA assertion to verify:

"Whenever signal A toggles (either 0->1 or 1->0),

signal B must toggle exactly 2 clock cycles later."

Note: "toggle" means any transition (rising OR falling edge).

B must show the OPPOSITE value from its previous value 2 cycles

after A shows the OPPOSITE value from its previous value.

ANSWER

```
// Key insight: "toggle" = signal changes value = $changed()
// $changed(sig) is true when sig != its value one cycle ago.
// Alternatively: detect any edge using (sig ^ $past(sig)) == 1
//
// Method 1: using $changed() -- most readable
property p_a_toggle_then_b_toggle;
  @(posedge clk) disable iff (!rst_n)
  $changed(A) |-> ##2 $changed(B);
endproperty
assert property (p_a_toggle_then_b_toggle)
  else `uvm_error("SVA", "B did not toggle 2 cycles after A toggled")

// Method 2: using XOR to detect toggle explicitly
// A toggles when: A ^ $past(A) == 1  (one bit changed)
// B toggles when: B ^ $past(B) == 1
property p_toggle_xor;
  @(posedge clk) disable iff (!rst_n)
  (A ^ $past(A)) |-> ##2 (B ^ $past(B));
endproperty
assert property (p_toggle_xor)
  else `uvm_error("SVA", "B did not toggle 2 cycles after A toggled")

// Method 3: separate rising and falling edge cases
// (equivalent to Method 1 but more explicit)
property p_a_rise_b_toggle;
  @(posedge clk) disable iff (!rst_n)
  $rose(A) |-> ##2 $changed(B);
endproperty
property p_a_fall_b_toggle;
  @(posedge clk) disable iff (!rst_n)
  $fell(A) |-> ##2 $changed(B);
endproperty
assert property (p_a_rise_b_toggle);
assert property (p_a_fall_b_toggle);

// Timing diagram example:
//
//  clk:  _|^|_|^|_|^|_|^|_|^|_
//  A:    0  0  1  1  1  0  0  0   (toggles at cycle 2, then cycle 5)
//  B:    0  0  0  0  1  1  1  0   (must toggle at cycle 4, then cycle 7)
//        |        |->2cy->|        |->2cy->|
//        A toggles        B toggles
//
// $changed(A) fires at cycle 2 (A: 0->1) -> ##2 -> B must toggle at cycle 4
// $changed(A) fires at cycle 5 (A: 1->0) -> ##2 -> B must toggle at cycle 7
//
// Note: $changed(B) is evaluated at cycle 4 -- it checks whether
// B at cycle 4 differs from B at cycle 3.
```

```
// If B was 0 at cycle 3 and becomes 1 at cycle 4 -> $changed = true. OK.

// Important edge case: what if A toggles again before 2 cycles have passed?
// Example: A toggles at cycle 1 AND cycle 2.
// Then B must toggle at cycle 3 (from the first toggle)
//       AND B must toggle at cycle 4 (from the second toggle).
// The |-> operator launches a new check for EACH trigger occurrence.
// Both checks run independently -- SVA handles this correctly.

// Cover property: ensure the toggle scenario is exercised in simulation
cover property (
  @(posedge clk) $changed(A) ##2 $changed(B)
);
```

解释

这道题的核心是理解"toggle"（翻转）的 SVA 写法。

$changed(sig)：当前周期的 sig 值与上一个周期不同时为真，

即检测到任意边沿（上升沿或下降沿），是最简洁的翻转检测方式。

等价写法：(sig ^ $past(sig)) 也可以检测翻转，但 $changed() 更简洁。

|-> ##2 的语义："A 翻转"发生在当前周期，2个周期后（##2）检查"B 翻转"。

注意：##2 是严格的"恰好2个周期后"，不是"至少2个周期后"。

如果要"至少2个周期后"，改用 ##[2:$]。

disable iff (!rst_n) 是必须的，防止复位期间的信号变化触发误报。

## SystemVerilog — OOP

### Q56. OOP: Inheritance, Polymorphism, and Singleton Pattern

QUESTION

(a) What is a virtual function? How does super work in SV/C++?

(b) Write an example showing inheritance and polymorphism with virtual functions.

(c) Implement a Singleton class in SystemVerilog (only one instance can ever be created).

ANSWER

```
// (a) virtual function:
  // Declares that a method can be overridden by subclasses.
  // Without virtual: static dispatch (resolved at compile time
  //    based on handle type).
  // With virtual: dynamic dispatch (resolved at runtime based
  //    on actual object type).
  //
  // super: calls the parent class version of a method.
  // Used in overriding methods to invoke parent behavior first.

  // (b) Polymorphism example
```

```
 class Animal;
   virtual function string speak();
      return "...";
   endfunction
 endclass

 class Dog extends Animal;
   virtual function string speak();
      return "Woof";
   endfunction
 endclass

 class Cat extends Animal;
   virtual function string speak();
      return "Meow";
   endfunction
 endclass

 // Dynamic dispatch demo:
 Animal a;
 Dog d = new();
 Cat c = new();
 a = d; $display(a.speak()); // prints "Woof"      (virtual dispatch)
 a = c; $display(a.speak()); // prints "Meow"      (virtual dispatch)

// Without 'virtual', both would print "..." (static dispatch on Animal)

// (c) Singleton pattern in SystemVerilog
class Singleton;
    // Private static instance handle
    static local Singleton _instance = null;

    // Private constructor (local prevents external new())
    local function new();
    endfunction

    // Public static accessor
    static function Singleton get_instance();
     if (_instance == null)
        _instance = new();
     return _instance;
    endfunction

    // Example method
    function void do_something();
     $display("Singleton instance doing work");
    endfunction
endclass

// Usage:
// Singleton s1 = Singleton::get_instance();
// Singleton s2 = Singleton::get_instance();
// s1 == s2 -> always the same object
```

解释

virtual function 是多态的核心。没有 virtual 时，调用哪个函数由句柄的声明类型决定；

有 virtual 时，由运行时实际对象的类型决定。

super 关键字用于在子类中调用父类的同名方法，常见于 super.new() 和

super.build_phase() 等 UVM 场景。

Singleton 模式用 static local 限制只能通过 get_instance() 获取实例，

local 关键字让 new() 只能在类内部调用，外部无法直接 new()。

## Q57. OOP: UVM Sequence Bug Analysis

QUESTION

Is there a bug in the following UVM sequence? If so, what is it

and what is the impact?

```
class my_seq extends uvm_sequence #(apb_txn);
    task body();
     apb_txn txn = apb_txn::type_id::create("txn");
     forever begin
        // apb_txn txn = apb_txn::type_id::create("txn");
        start_item(txn);
        if (!txn.randomize()) `uvm_error("SEQ", "rand failed")
        finish_item(txn);
     end
    endtask
endclass
```

ANSWER

```
// YES, there is a critical bug.
 //
 // Bug: txn is created ONCE outside the forever loop, then the
 // SAME object handle is reused for every iteration.
 //
 // Impact:
 // 1. Aliasing: start_item / finish_item pass the same handle to
 //        the driver each iteration. The driver holds a reference to
 //        the object. When randomize() runs in the NEXT iteration,
 //        it modifies the same object the driver may still be using
 //        -> race condition / data corruption.
 //
 // 2. Scoreboard: if the monitor captures the txn handle for
 //        reference checking, it will see the overwritten (randomized)
 //        values from subsequent iterations, not the original values
 //        -> false failures in scoreboard.
 //
 // Fix: create a new txn object inside the loop each iteration:
 task body();
     forever begin
      apb_txn txn = apb_txn::type_id::create("txn"); // inside loop
      start_item(txn);
      if (!txn.randomize()) `uvm_error("SEQ", "rand failed")
      finish_item(txn);
     end
 endtask

 // Why this matters:
 // Each call to type_id::create() allocates a NEW object on the heap.
 // The driver and scoreboard each get a unique snapshot of the txn.
 // No aliasing, no race condition.
```

解释

这是 UVM sequence 中非常经典且容易犯的 bug。

txn 在循环外创建只有一个对象，每次 randomize() 都在原地修改这个对象的字段。

Driver 调用 get_next_item() 得到的是句柄，不是值的拷贝，

所以当下一次 randomize() 跑完，driver 正在处理的数据就被篡改了。

Scoreboard 存的也是同一个句柄，参考值同样会被后续迭代覆盖。

修复方法：把 txn 的创建移到 forever 循环内部，每次迭代 new 一个新对象。

## Q58. OOP: Singleton Class and Abstract Class

QUESTION

(a) What is a Singleton class? Why is it useful in a UVM
testbench context? Implement one in SystemVerilog.

(b) What is an Abstract class (virtual class) in SystemVerilog?
How does it differ from a regular class and an interface class?
Give a concrete UVM example of where abstract class is used.

(c) Can a Singleton also be an Abstract class? Why or why not?

(d) In a UVM testbench, you want a global configuration object
that is accessible from any component without passing handles.
Show two ways to implement this -- one using Singleton,
one using uvm_config_db -- and compare the tradeoffs.

ANSWER

```
// (a) Singleton class
 // Definition: a class that can only ever have ONE instance.
 // The single instance is created lazily on first access.
 //
 // UVM use cases:
 //    - Global scoreboard accessible from any component
 //    - Shared counter/statistics collector
 //    - Global configuration store
 //    - Resource manager (e.g. memory map, register model)
 //
 class tb_config_singleton;
   // 1. Private static handle (only one ever exists)
   static local tb_config_singleton _inst = null;

   // 2. Public config fields
   int   num_transactions = 100;
   bit   enable_checks     = 1;
   int   timeout_cycles    = 1000;

   // 3. Private constructor -- no one can call new() externally
   local function new();
   endfunction

   // 4. Static accessor -- the ONLY way to get the instance
   static function tb_config_singleton get();
      if (_inst == null)
         _inst = new();
      return _inst;
   endfunction
 endclass
```

```
 // Usage from anywhere in the TB (no handle passing needed):
 // tb_config_singleton cfg = tb_config_singleton::get();
 // cfg.num_transactions = 500;
 // if (tb_config_singleton::get().enable_checks) ...

// (b) Abstract class (virtual class)
//
// virtual class: cannot be instantiated directly.
//    Defines a contract (interface) that subclasses must honor.
//    May contain pure virtual methods (no implementation) and
//    also concrete methods with implementation.
//
// Regular class: can be instantiated. All methods have bodies.
//
// Interface class (SV 2012+): can ONLY contain pure virtual
//    methods. No data members, no implementation at all.
//    A class can implement multiple interface classes
//    (vs single inheritance for virtual class).
//
// UVM example: uvm_object is effectively abstract
//    do_copy(), do_compare(), do_print() are virtual methods
//    that subclasses override. The base uvm_object cannot
//    represent a useful transaction on its own.
//
virtual class base_checker;
  // Concrete shared method
  function void report_error(string msg);
     `uvm_error("CHK", msg)
  endfunction

  // Pure virtual: each checker must define its own check logic
  pure virtual function bit check(uvm_sequence_item item);
endclass

class protocol_checker extends base_checker;
  virtual function bit check(uvm_sequence_item item);
     // protocol-specific check implementation
     return 1;
  endfunction

 endclass

 // (c) Can Singleton be Abstract?
 // NO. An abstract (virtual) class cannot be instantiated.
 // A Singleton requires exactly ONE instance to be created
 // via new() inside get().
 // If the class is abstract, new() can never be called ->
 // the single instance can never be created -> contradiction.
 //
 // The Singleton pattern fundamentally requires instantiation;
 // abstract classes fundamentally prohibit it.
 // They are mutually exclusive design patterns.

 // (d) Global config: Singleton vs uvm_config_db
 //
 // Method 1: Singleton
 class global_cfg_singleton;
     static local global_cfg_singleton _inst = null;
     int   max_txn   = 100;
     bit   en_cov    = 1;
     local function new(); endfunction
     static function global_cfg_singleton get();
      if (_inst == null) _inst = new();
```

```
      return _inst;
    endfunction
endclass
// Access: global_cfg_singleton::get().max_txn = 200;

// Method 2: uvm_config_db
// In test:
// uvm_config_db #(int)::set(this, "*", "max_txn", 200);
// In any component:
// int max_txn;
// uvm_config_db #(int)::get(this, "", "max_txn", max_txn);

// Tradeoffs:
// Singleton:
//     + Accessible anywhere with zero setup (no handle passing)
//     + Type-safe (compile-time check)
//     + Simple API: just call ::get()
//     - Hard to override per-test (global state is sticky)
//     - Not compatible with UVM factory override mechanism
//     - Harder to use in multi-threaded / multi-env scenarios
//
// uvm_config_db:
//     + Integrates with UVM hierarchy (context-aware)
//     + Easy to override per-component or per-test
//     + Supports wildcard paths ("env.agent*")
//     - Requires knowing the key string (runtime string matching)
//     - get() can silently fail if key mismatches (debug pain)
//     - Slightly verbose syntax
```

解释

Singleton 和 Abstract class 是 OOP 面试的高频点，且常常被放在一起考。

Singleton 的实现要点：

1. static local 句柄保存唯一实例

2. local function new() 防止外部创建

3. static function get() 提供全局访问入口

Abstract class 的记忆技巧：

virtual class + pure virtual function = 必须由子类实现的接口合同。

UVM 里大量使用这个模式（do_copy, do_compare, do_print 等）。

(c) 为什么 Singleton 不能是 Abstract：

这是一个常见面试考题，答案很明确：

Abstract class 禁止实例化，Singleton 一定要实例化一次，两者逻辑矛盾。

(d) Singleton vs config_db 的选择：

简单全局配置用 Singleton 更简洁；

需要按组件层次覆盖配置用 config_db 更灵活。

## Q59. SV OOP: Class Handle -- Pass by Value vs Pass by ref

QUESTION

What does the following code display? Explain why each function

produces the result it does.

```
class packet;
   int data;
endclass


function void modify_1(packet p);
   p.data = 100;
endfunction


function void modify_2(ref packet p);
   p.data = 100;
endfunction


function void modify_3(packet p);
   p = new();
   p.data = 200;
endfunction


initial begin
   packet pkt = new();
   modify_1(pkt); $display(pkt.data);
   modify_2(pkt); $display(pkt.data);
   modify_3(pkt); $display(pkt.data);
end
```

ANSWER

```
// Output:
 //    100    <- modify_1
 //    100    <- modify_2
 //    100    <- modify_3   (NOT 200 -- this surprises most people!)

 // modify_1(packet p) -- handle passed by VALUE
 //    A COPY of the handle is made.
 //    Both pkt and local p point to the SAME heap object.
 //    p.data = 100 modifies the shared object -> pkt.data = 100.
 //    Display: 100

 // modify_2(ref packet p) -- handle passed by REFERENCE
 //    p IS pkt (same handle variable, no copy at all).
 //    p.data = 100 modifies the same object pkt.data was already 100.
 //    Display: 100
 //    (Difference from modify_1: if p = new() were done here,
 //      pkt would also change -- that's the power of ref.)

 // modify_3(packet p) -- handle passed by VALUE, then local reassign
 //    A COPY of the handle is made (p initially points to same object).
 //    p = new() -> local p now points to a NEW heap object.
 //    pkt still points to the ORIGINAL object (unchanged!).
 //    p.data = 200 modifies the NEW object, not the original.
 //    -> pkt.data is still 100.
 //    Display: 100   (NOT 200)

 // Key insight:
 //    SV class arguments are HANDLES (pointers), not object values.
 //    Pass by value copies the POINTER, not the object.
 //    Modifying FIELDS (p.data=X) through a copy DOES affect original.
 //    Reassigning the HANDLE (p = new()) only changes local copy;
```

```
//    caller's handle is unaffected unless ref is used.

// To get 200 from modify_3, use ref:
function void modify_3_fix(ref packet p);

    p = new();       // now changes caller's handle too
    p.data = 200;
endfunction
// After modify_3_fix(pkt): pkt.data == 200
```

解释

这是 SV OOP 最经典的面试题，几乎每家公司都会考。

三种情况的核心区别：

modify_1: 传句柄副本，两个句柄指向同一对象，修改字段有效 -> 100。

modify_2: 传句柄引用，p 就是 pkt，效果与 modify_1 相同（此例中）。

modify_3: 传句柄副本，p = new() 让本地句柄指向新对象，

但 pkt 仍指向原来的对象，修改的是新对象 -> pkt.data 还是 100。

面试关键：能清楚区分"修改字段"和"重新分配句柄"两种操作的影响。

## SystemVerilog — Coding

### Q60. SV Coding: Edge Detector (All Edges and Rising Edge)

QUESTION

Write a synchronous edge detector in SystemVerilog:

(a) Detect ALL edges (both rising and falling).

(b) Detect only RISING edges.

Output a single-cycle pulse for each detected edge.

ANSWER

```
module edge_detector (
    input     logic clk,
    input     logic rst_n,
    input     logic din,
    output logic rising_edge_det,
    output logic any_edge_det
  );
    logic din_d1;

    always_ff @(posedge clk or negedge rst_n) begin
       if (!rst_n) din_d1 <= 1'b0;
       else        din_d1 <= din;
    end

    // (a) Any edge: XOR of current and previous value
    assign any_edge_det       = din ^ din_d1;

    // (b) Rising edge: current HIGH, previous LOW
```

```
  assign rising_edge_det = din & ~din_d1;

  // Falling edge (for reference): current LOW, previous HIGH
  // assign falling_edge_det = ~din & din_d1;
endmodule

// Fully registered version (glitch-free outputs, 1 cycle later):
module edge_detector_reg (
  input     logic clk,
  input     logic rst_n,
  input     logic din,
  output logic rising_edge_det,
  output logic any_edge_det
);
  logic din_d1;

  always_ff @(posedge clk or negedge rst_n) begin

    if (!rst_n) begin
         din_d1           <= 1'b0;
         rising_edge_det <= 1'b0;
         any_edge_det     <= 1'b0;
    end else begin
         din_d1           <= din;
         rising_edge_det <= din & ~din_d1;
         any_edge_det     <= din ^ din_d1;
    end
   end
endmodule
```

解释

边沿检测原理：把输入打一拍，与当前值做逻辑运算。

任意边沿：XOR（任何变化都输出1）。

上升沿：AND NOT（当前为高，上一拍为低）。

下降沿：NOT AND（当前为低，上一拍为高）。

两种版本的区别：combinational assign 输出更快但可能有毛刺；

registered 版本多一拍延迟但输出完全同步无毛刺，实际设计中更常用。

## Q61. SV Coding: Monitor and Testbench for Interrupt Protocol

QUESTION

There are two signals: 'interrupt' and 'task_started'.

Write monitor/test code that:

- Waits for task_started to go HIGH

- Then waits for an interrupt while task_started is HIGH

- Prints "Interrupt seen during task" if interrupt occurs

before task_started goes LOW

- Prints "No interrupt seen during task" if task_started goes

LOW without an interrupt

ANSWER

```
// Assumption: signals are synchronous to posedge clk.
  // Assumption: task_started and interrupt are level signals.

  task automatic monitor_interrupt_during_task(
     virtual my_if vif
  );
     // Step 1: wait for task_started to go high
     @(posedge vif.clk);
     wait (vif.task_started == 1'b1);
     $display("[MON] task_started asserted");

     // Step 2: now race between interrupt and task_started falling
     fork
       begin : check_interrupt
         // Wait for interrupt while task is active
         @(posedge vif.clk iff vif.interrupt);
         $display("[MON] Interrupt seen during task");
         disable check_task_end;
       end

       begin : check_task_end
         // Wait for task_started to go low
         @(negedge vif.task_started);
         // Check if interrupt also happened this cycle
         if (!vif.interrupt) begin
             $display("[MON] No interrupt seen during task");
             disable check_interrupt;
         end
       end
     join_any
     disable fork;
  endtask

  // Alternative using |-> SVA style check:
  // Check that if task_started goes low, interrupt must have been seen

  property p_interrupt_during_task;
     @(posedge clk) disable iff (!rst_n)
     $rose(task_started) |->
       (interrupt throughout task_started) or
       (!task_started ##0 1'b1);    // task ended
  endproperty
```

解释

这道题考察 fork/join_any + disable fork 的并发等待模式。

两个线程同时跑：一个等 interrupt，一个等 task_started 下降沿。

先发生的那个赢，然后 disable 掉另一个线程。

这是 UVM/SV testbench 中处理"两个事件谁先发生"问题的标准写法。

join_any 等第一个线程结束，disable fork 杀掉所有剩余线程。

## Q62. SV Coding: Covergroup for 8-bit Signal

QUESTION

Write coverpoints for an 8-bit signal 'data':

(a) Create bins only for values 4 and 7.

(b) Create bins for all values from 14 to 19.

ANSWER

```
bit [7:0] data;

  covergroup data_cg @(posedge clk);

     // (a) Bins only for specific values 4 and 7
     cp_specific: coverpoint data {
         bins val_4 = {4};
         bins val_7 = {7};
         // All other values are ignored (no 'default' bin)
     }

     // (b) Bins for range 14 to 19
     cp_range: coverpoint data {
         bins range_14_19[] = {[14:19]};
         // [] creates one bin per value: bin for 14, 15, 16, 17, 18, 19
     }

     // Alternative: single bin covering the whole range
     cp_range_single: coverpoint data {
         bins range_14_19 = {[14:19]};
         // Single bin: hit if data is any value in [14,19]
     }

  endgroup

  // Common bin syntax reference:
  // bins name      = {val}          -> single value bin
  // bins name      = {[lo:hi]}      -> one bin for entire range
  // bins name[]    = {[lo:hi]}      -> one bin PER value in range
  // bins name[N] = {[lo:hi]}        -> N bins, range split evenly
  // default bins name               -> catches all uncovered values
```

解释

SV covergroup 的 bins 语法是面试高频考点。

关键区别：

bins name = {[14:19]} 是一个 bin，覆盖14到19中任意一个值就算命中。

bins name[] = {[14:19]} 是6个独立的 bin，每个值需要单独被覆盖。

如果只写 {4} 和 {7}，其他值不会被统计（没有 default bin）。

面试时经常追问"如果要单独追踪每个值的覆盖，应该用哪种写法"——答案是 []。

## Q63. SV Coding: AHB Interface with Clocking Blocks

QUESTION

Write a SystemVerilog AHB interface with separate clocking blocks

```
for driver and monitor.
```

ANSWER

```
interface ahb_if (input logic clk);

     logic         reset_n;
     logic [31:0] HADDR;
     logic [31:0] HWDATA;
```

```
    logic [31:0] HRDATA;
    logic        HWRITE;
    logic        HREADY;
    logic        HVALID;
    logic [2:0]  HSIZE;
    logic [1:0]  HTRANS;    // IDLE=0, NONSEQ=2, SEQ=3

    // Driver clocking block: drive on posedge, sample 1ns before edge
    clocking drv_cb @(posedge clk);
     default input #1ns output #1ns;
     output HADDR, HWDATA, HWRITE, HVALID, HSIZE, HTRANS;
     input  HRDATA, HREADY;
    endclocking

    // Monitor clocking block: observe all signals
    clocking mon_cb @(posedge clk);
     default input #1ns output #1ns;
     input HADDR, HWDATA, HRDATA, HWRITE,
           HREADY, HVALID, HSIZE, HTRANS;
    endclocking

    // Modport for driver (uses drv_cb)
    modport drv_mp   (clocking drv_cb, input reset_n);

    // Modport for monitor (uses mon_cb)
    modport mon_mp   (clocking mon_cb, input reset_n);

endinterface

// Key design decisions:

// 1. clk passed as port (not declared inside) for proper connection
// 2. Driver output = bus inputs (what driver drives onto bus)
// 3. Monitor inputs only (passive observation, no driving)
// 4. default input/output #1ns: sample 1ns before edge,
//      drive 1ns after edge -> avoids setup/hold issues in simulation
```

解释

Clocking block 是 SV 避免仿真竞争的核心机制。

不用 clocking block 直接在 always @(posedge clk) 里驱动信号，

可能产生时序竞争（信号在时钟边沿同时被 monitor 采样和 driver 更新）。

default input #1ns output #1ns 意思是：

采样信号在时钟沿前 1ns（避免 setup 问题），

驱动信号在时钟沿后 1ns（避免 hold 问题）。

Driver 的 clocking block 只有 output（它要驱动的）和 input（它要读的），

Monitor 的 clocking block 全部是 input（只观察不驱动）。

## Q64. SV Coding: fork/join_none Loop Variable Capture

QUESTION

What does the following code display? Explain why, and how to fix it.

Follow-up: What if join_none is replaced with join or join_any?

ANSWER

```
  // --- Problem code (what does this display?) ---
  for (int i = 0; i < 3; i++) begin
      fork
        $display("%0d", i);
      join_none
  end

  // Output with join_none: 3   3   3   (most common result)
  //
  // Why:
  //   join_none fires the thread and returns IMMEDIATELY.
  //   The loop finishes (i becomes 3) before any thread executes.
  //   All 3 threads share the SAME loop variable 'i'.
  //   When they eventually run, i == 3.
  //
  // Fix: capture 'i' in an automatic local variable inside the fork:
  for (int i = 0; i < 3; i++) begin
      fork
        begin
          automatic int j = i;  // each thread gets its own copy of i
          $display("%0d", j);
        end
      join_none
  end
  // Output: 0   1   2  (in some order, but each thread has correct value)

  // Follow-up -- join vs join_any vs join_none:
  //
  // join:      waits for ALL forked threads to finish before continuing.
  //            Loop blocks at each iteration -> only 1 thread active at a time.
  //            Output: 0 1 2 (in order, no race condition with i).
  //
  // join_any:  waits for ANY ONE forked thread to finish, then continues.
  //            Intermediate behavior: may still see variable capture issues
  //            if threads complete out of order.
  //
  // join_none: fires thread and continues immediately (non-blocking).
  //            All threads start "simultaneously", sharing loop variable.
  //            Must use automatic variable to capture per-iteration value.
  //
  // Rule of thumb:
  //   Always use 'automatic int j = i' when forking inside a loop
  //   with join_none. This is the most common fork/join bug in interviews.

Q58. SV Coding: Interface Declaration and config_db set/get
```

QUESTION

(a) What is a SystemVerilog interface? What problem does it solve?

(b) Declare a simple valid-ready interface with clocking blocks.

(c) Show the complete config_db set/get pattern to pass a virtual

interface from testbench top to a UVM driver.

Explain every parameter of the set and get calls.

ANSWER

```
// (a) Interface
  // Bundles related signals into a named group.
  // Problem solved: with individual ports, adding a signal requires
  // changing every module in the hierarchy (brittle, tedious).
```

```
// Interface: add the signal once; all connected modules see it.
// Also enables: clocking blocks (no race), modports (role-based
// access restriction), assertions directly on the interface.

// (b) Valid-ready interface
interface vld_rdy_if #(parameter DATA_W = 32) (input logic clk);
  logic                valid;
  logic                ready;
  logic [DATA_W-1:0] data;

  clocking drv_cb @(posedge clk);
    default input #1ns output #1ns;
    output valid, data;
    input   ready;
  endclocking

  clocking mon_cb @(posedge clk);
    default input #1ns;
    input valid, ready, data;
  endclocking

  modport drv_mp (clocking drv_cb, input clk);
  modport mon_mp (clocking mon_cb, input clk);
endinterface

// (c) config_db set (in tb_top module):
module tb_top;
  logic clk;
  vld_rdy_if vif (.clk(clk));

  initial begin

    // set(CONTEXT, PATH, NAME, VALUE)
    //      CONTEXT : null   = resolve from uvm_test_top root
    //      PATH     : path to target component; "*" = wildcard match all
    //      NAME     : string key -- must match get() exactly
    //      VALUE    : the virtual interface handle to store
    uvm_config_db #(virtual vld_rdy_if)::set(
         null,
         "uvm_test_top.env.agent.drv",
         "vif",
         vif
    );
    run_test("my_test");
   end
endmodule

// config_db get (in driver build_phase):
class my_driver extends uvm_driver #(my_seq_item);
   virtual vld_rdy_if vif;

   function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // get(CONTEXT, PATH, NAME, VARIABLE)
    //      CONTEXT : this   = current component (path resolved relative to it)
    //      PATH     : ""    = look at this component's exact path
    //      NAME     : "vif" = must match set() NAME exactly
    //      VARIABLE: vif    = output variable to receive the handle
    //      returns : 1 if found, 0 if not found
    if (!uvm_config_db #(virtual vld_rdy_if)::get(
               this, "", "vif", vif))
         `uvm_fatal("NO_VIF", "Virtual interface not found!")
   endfunction
```

```
  endclass

  // Most common mistake: NAME string mismatch between set and get.
  // e.g. set uses "vif" but get uses "VIF" -> get returns 0 -> fatal.
```

解释

Interface + config_db 是 UVM 中最基础的配置传递机制，面试必考。

Interface 的核心价值：模块化、减少端口耦合、clocking block 消除竞争。

config_db 四参数记忆法：

CONTEXT: 从哪里出发（null=顶层，this=当前组件）

PATH: 到哪里去（组件层次路径，支持*通配符）

NAME: 叫什么（字符串键，set和get必须完全一致）

VALUE/VAR: 传什么（接口句柄、整数、对象等）

get 返回 0 最常见原因：NAME 字符串不匹配，或 PATH 指向错误。

## Q65. SV Coding: fork/join Variations and wait fork

QUESTION

(a) Explain the difference between fork/join, fork/join_any,

and fork/join_none. Predict the output of Scenario A.

(b) What does wait fork do? Predict the output of Scenario B.

```
// Scenario A
initial begin
  fork
      begin #10; $display("Thread 1 done"); end
      begin #20; $display("Thread 2 done"); end
      begin #5;    $display("Thread 3 done"); end
  join            // change to join_any or join_none for each case
  $display("After fork");
end


// Scenario B
initial begin
  for (int i = 0; i < 3; i++) begin
      fork
        begin automatic int k = i; #(k*10); $display("k=%0d",k); end
      join_none
  end
  wait fork;
  $display("All done");
end
```

ANSWER

```
// Scenario A outputs:

 // fork/join (wait for ALL threads):
 //   T=5:    "Thread 3 done"
 //   T=10: "Thread 1 done"
 //   T=20: "Thread 2 done"
```

```
//   T=20: "After fork"          <- unblocked only when ALL done (T=20)

// fork/join_any (wait for FIRST thread):
//   T=5:    "Thread 3 done"
//   T=5:    "After fork"        <- unblocked when first done (T=5)
//   T=10: "Thread 1 done"       <- threads 1 and 2 keep running in background
//   T=20: "Thread 2 done"

// fork/join_none (no wait):
//   T=0:    "After fork"        <- immediate, no blocking at all
//   T=5:    "Thread 3 done"
//   T=10: "Thread 1 done"
//   T=20: "Thread 2 done"

// Scenario B output (with automatic + wait fork):
//   T=0:    (loop fires 3 threads, all join_none -- returns immediately)
//   T=0:    "k=0"   (k=0, #0 delay)
//   T=10: "k=1"
//   T=20: "k=2"
//   T=20: "All done"       <- wait fork blocks until all 3 threads complete

// Why 'automatic int k = i' is necessary in Scenario B:
//   Without it: all threads share 'i'. Loop ends with i=3.
//   All threads print k=3 (wrong).
//   With automatic: each thread captures its own k at fork time.

// wait fork vs disable fork:
//   wait fork:      block until ALL previously spawned threads finish.
//   disable fork: KILL all currently running child threads immediately.

// Summary table:
// Keyword      | Blocks until          | After-fork threads?
// join         | ALL done              | N/A (none remaining)
// join_any     | FIRST done            | Others keep running
// join_none    | Never (immediate)     | All run in background
// wait fork    | ALL done (explicit)| Explicit barrier after join_none
```

解释

fork/join 三种变体是面试必考，且经常和 Scenario B 配合考 wait fork。

记忆口诀：

join = 等所有（全员到齐才走）

join_any = 等最快（第一名到就走，其他继续）

join_none = 不等（开枪就跑，所有人在后台跑）

wait fork = 补丁（join_none 用完后显式等待所有后台线程）

Scenario B 的经典考点是 automatic + wait fork 的组合：

join_none 循环启动多个并发任务，wait fork 确保全部完成后再继续。

## Q66. SV Coding: fork/join -- Call Function After Any Two of Three Tasks Complete

QUESTION

Start task1, task2, and task3 in parallel using fork/join_none.

Call call_after_any_two_finishes() as soon as ANY TWO of the

three tasks complete, without waiting for the third.

Important: your completion counter will be incremented by multiple threads concurrently. How do you prevent a race condition where two threads simultaneously read the same counter value, both see count < 2, both increment, and the callback fires twice (or the second increment is lost)?

Show the correct implementation and explain the race fix.

ANSWER

```
// WRONG approach -- race condition on shared counter:
  // int done_count = 0;
  // task1(); done_count++;    // thread A reads 0, adds 1, writes 1
  // task2(); done_count++;    // thread B reads 0 simultaneously!
  //                           // thread B also writes 1 -> count stays 1
  //                           // callback never fires, or fires once instead of
  //                           // firing at exactly count==2

  // ROOT CAUSE:
  // In SystemVerilog, ++ on a shared variable is NOT atomic.
  // It expands to: read -> add 1 -> write back.
  // Two threads at the same simulation time step can both read
  // the old value, both compute old+1, and both write the same
  // incremented value -> one increment is silently lost.

  // CORRECT approach 1: semaphore (mutex + counting)
  // Use a semaphore initialized to 0.
  // Each task puts 1 key when done.
  // The waiter tries to get 2 keys -- blocks until 2 are available.
  // semaphore.put() is atomic in SV (no race).

  semaphore done_sem;

  task run_with_semaphore();
    done_sem = new(0);   // start with 0 keys

    fork
       begin task1(); done_sem.put(1); end   // atomic put
       begin task2(); done_sem.put(1); end   // atomic put
       begin task3(); done_sem.put(1); end   // atomic put
    join_none

    // Block until 2 keys are available (any 2 tasks finished)
    done_sem.get(2);       // atomic: waits until 2 puts have occurred
    call_after_any_two_finishes();

  // Third task may still be running -- that's fine
endtask

// CORRECT approach 2: event + atomic counter via semaphore guard
// Use a 1-key semaphore as a mutex to protect the counter,
// then use an event to signal the waiter.

semaphore count_mutex;
int         done_count;
event       two_done;

task run_with_mutex();
  count_mutex = new(1);   // mutex: 1 key = unlocked
  done_count   = 0;
```

```
  fork
      begin task1(); signal_done(); end
      begin task2(); signal_done(); end
      begin task3(); signal_done(); end
  join_none

  @two_done;   // wait for event
  call_after_any_two_finishes();
endtask

task automatic signal_done();
  count_mutex.get(1);      // lock: only one thread enters at a time
  done_count++;
  if (done_count == 2)
      ->two_done;          // trigger event exactly once
  count_mutex.put(1);      // unlock
endtask

// CORRECT approach 3: uvm_event (preferred in UVM testbench)
// uvm_event internally handles thread safety.

  uvm_event two_done_ev;

  task run_uvm();
     int unsigned cnt = 0;
     semaphore     mtx = new(1);
     two_done_ev = new("two_done_ev");

     fork
       begin task1(); on_done(cnt, mtx, two_done_ev); end
       begin task2(); on_done(cnt, mtx, two_done_ev); end
       begin task3(); on_done(cnt, mtx, two_done_ev); end
     join_none

     two_done_ev.wait_trigger();
     call_after_any_two_finishes();
  endtask

  task automatic on_done(
     ref int unsigned cnt,
     ref semaphore     mtx,
     input uvm_event   ev
  );
     mtx.get(1);
     cnt++;
     if (cnt == 2) ev.trigger();
     mtx.put(1);
  endtask

  // Why approach 1 (semaphore.get(2)) is cleanest:
  // - No manual counter needed
  // - put() is guaranteed atomic by SV LRM
  // - get(2) blocks until exactly 2 puts have completed
  // - Minimal code, no explicit mutex management
  // - Works correctly regardless of task completion order
```

解释

这道题考察两个核心知识点的结合：fork/join 并发控制 和 共享变量的竞争条件。

竞争的根本原因：SV 中的 ++ 不是原子操作，展开为读-加-写三步，

两个线程在同一时间步同时执行这三步，其中一次写入会覆盖另一次，

导致计数器只增加了 1 而不是 2。

最简洁的正确方案：semaphore.get(2)。

SV 的 semaphore.put() 是原子操作（LRM 保证），

get(2) 阻塞直到累计有 2 次 put 完成，天然解决了竞争问题，

且不需要手动管理计数器或互斥锁。

Mutex 方案（方案2/3）也正确，但代码更复杂。

面试中能指出"++ 不是原子操作"并给出 semaphore 方案是最强的回答。

## Q67. Verilog: Falling Edge Detector and Its Synthesized Circuit

QUESTION

(a) Write a Verilog/SystemVerilog module that detects the falling

edge of an input signal 'din' and outputs a single-cycle

pulse 'falling_edge' on each falling edge.

(b) Draw (describe in ASCII) the synthesized gate-level circuit.

What logic elements does synthesis produce?

ANSWER

```
// (a) Falling edge detector
 module falling_edge_det (
   input     logic clk,
   input     logic rst_n,
   input     logic din,
   output logic falling_edge_det
 );
   logic din_d1;       // din delayed by 1 cycle

   // Register to store previous cycle value
   always_ff @(posedge clk or negedge rst_n) begin
      if (!rst_n) din_d1 <= 1'b0;
      else            din_d1 <= din;
   end

   // Falling edge: din was HIGH last cycle, LOW this cycle
   // din_d1=1 AND din=0         <=>   din_d1 AND (NOT din)
   assign falling_edge_det = din_d1 & ~din;
 endmodule

 // Truth table:
 // din_d1 | din | falling_edge_det
 //    0      |   0   |         0   (staying low, no edge)
 //    0      |   1   |         0   (rising edge -- NOT what we detect)
 //    1      |   0   |         1   ** FALLING EDGE DETECTED **
 //    1      |   1   |         0   (staying high, no edge)

 // (b) Synthesized circuit (ASCII):
 //
 //                          +-------+
 //   clk    ------------>|           |
 //                        |    D FF |
 //   rst_n ------------>|        (Q)   |--din_d1--+---[AND]---> falling_edge_det
 //                        |          |          |       ^
 //   din    ------------>|      (D)   |          |       |

 //                       +-------+            |    [NOT]
```

```
//                                          |      |
//   din   ----------------------------------+------+
//
// Components after synthesis:
//    1 x D flip-flop (posedge clk, negedge async reset)
//          D input   = din
//          Q output = din_d1     (din delayed 1 cycle)
//    1 x Inverter (NOT gate)
//          input     = din
//          output    = ~din
//    1 x AND gate (2-input)
//          inputs    = din_d1,   ~din
//          output    = falling_edge_det
//
// Signal flow:
//    Each posedge clk: FF captures current din -> din_d1
//    Combinational:      AND( din_d1, NOT(din) )
//    Pulse is HIGH for exactly 1 clock cycle on each falling edge of din.
//
// Note: for a rising edge detector, swap the inputs:
//    rising_edge_det = ~din_d1 & din
//    (was LOW last cycle, HIGH this cycle)
```

解释

边沿检测电路是数字设计基础，综合结果固定：1个FF + 1个NOT + 1个AND。

下降沿：当前为低（~din）且上一拍为高（din_d1），两者 AND。

上升沿：当前为高（din）且上一拍为低（~din_d1），两者 AND。

FF 的作用：把输入"记忆"一个时钟周期，用于与当前值对比。

输出脉冲宽度：恰好 1 个时钟周期（组合逻辑，FF 输出变化后立即有效）。

注意：如果需要无毛刺输出，可以在 always_ff 里对输出也打一拍（寄存器输出版本）。

## Q68. UVM: Architecture Overview

QUESTION

Describe the overall UVM testbench architecture. What are the

key components and how do they interact?

ANSWER

```
// UVM Testbench Hierarchy:
//
//   uvm_test
//     |
//     +-- uvm_env
//           |
//           +-- uvm_agent (x N)
//           |      |
//           |      +-- uvm_sequencer
//           |      +-- uvm_driver    <-----> DUT interface
//           |      +-- uvm_monitor <-----> DUT interface
//           |
//           +-- uvm_scoreboard
//           +-- uvm_coverage_collector
//
// Data flow:
```

```
// Test -> creates Sequences -> Sequencer arbitrates -> Driver drives DUT
//                                  Monitor observes DUT -> Analysis port
//                                  Analysis port -> Scoreboard / Coverage
//
// Key component responsibilities:
// uvm_test        : top-level test, configures env, starts sequences
// uvm_env         : groups agents and shared components
// uvm_agent       : encapsulates a single interface's driver+seq+monitor
// uvm_sequencer : arbitrates sequence items to driver
// uvm_driver      : translates sequence items to pin-level activity
// uvm_monitor     : observes interface, captures transactions
// uvm_scoreboard: compares DUT output vs reference model
//
// Communication mechanisms:
// TLM (Transaction Level Modeling):
//    - seq_item_port/export: sequencer <-> driver (pull model)
//    - uvm_analysis_port: monitor -> scoreboard/coverage (broadcast)
//    - uvm_tlm_fifo: buffered data transfer between components
//

// Configuration:
//    uvm_config_db: pass virtual interfaces, objects, values
//                       across hierarchy without direct references
```

解释

UVM arch 是验证工程师的必备知识，面试几乎必考。

核心思路是分层解耦：Test 只关心"测什么"，Driver 只关心"怎么驱动"，

Monitor 只关心"怎么采样"，Scoreboard 只关心"对不对"，

各组件通过 TLM 通信而不是直接引用，便于复用和替换。

Analysis port 是广播模式（一对多），一个 monitor 的数据可以

同时送到 scoreboard 和 coverage collector。

## UVM

### Q69. UVM: run_phase Ordering and drop_objection Behavior

QUESTION

Given the following run_phase implementation, what will be

displayed and in what order? Explain why.

```
task run_phase(uvm_phase phase);
    v_seq vseq;
    phase.raise_objection(this);
    vseq = v_seq::type_id::create("vseq");
    vseq.start(env.vseqr);
    phase.drop_objection(this);
    #50;
    $display("TEST ENDS");
endtask
```

ANSWER

```
// Answer: "TEST ENDS" is NEVER displayed (or is dropped mid-simulation).
 //
 // Why:
 // drop_objection(this) signals UVM that this component is done.
 // When ALL objections across ALL components are dropped, the UVM
 // phase engine immediately advances to cleanup phases (extract,
 // check, report, final) and calls $finish internally.
 //
 // #50 and $display after drop_objection live in run_phase, but
 // simulation ends before #50 elapses because UVM has already
 // moved past run_phase.
 //
 // CORRECT pattern -- do ALL work BEFORE drop_objection:
 task run_phase(uvm_phase phase);
     v_seq vseq;
     phase.raise_objection(this);
     vseq = v_seq::type_id::create("vseq");
     vseq.start(env.vseqr);
     #50;                            // delay BEFORE drop
     $display("TEST ENDS");         // display BEFORE drop
     phase.drop_objection(this);    // drop LAST
 endtask
 //
 // Key rule:
 //    raise_objection as EARLY as possible.
 //    drop_objection as LATE as possible (last thing in task).
 //    Everything AFTER drop_objection is not guaranteed to execute.
```

解释

这道题考察对 UVM 仿真生命周期的理解。

drop_objection 是触发仿真结束的信号，不是"我做完了但继续等"。

一旦所有 objection 被 drop，UVM phase engine 立即推进到 cleanup phases，

调用 $finish，run_phase 中 drop 之后的代码被强制终止。

正确做法：所有工作（包括 #delay、$display）必须在 drop_objection 之前完成。

## Q70. UVM: All Phases -- Which are Tasks vs Functions

QUESTION

(a) List all UVM phases in execution order. For each phase,

state whether it is implemented as a task or a function,

and briefly describe its purpose.

(b) Why does run_phase (and other time-consuming phases) have

to be a task rather than a function?

(c) What is the difference between uvm_component and uvm_object

in terms of phase participation? Which UVM classes are

uvm_object vs uvm_component?

(d) What happens if build_phase is not called with super.build_phase(phase)?

What is the consequence?

ANSWER

```
// (a) UVM Phase execution order and task/function classification:
 //
 // --- BUILD-TIME PHASES (zero-time, functions, top-down) ---
 // build_phase(phase)           FUNCTION    Create/configure children
 // connect_phase(phase)         FUNCTION    Connect TLM ports/exports
 // end_of_elaboration_phase     FUNCTION    Final config checks, print topology
 // start_of_simulation_phase    FUNCTION    Print banner, final setup
 //
 // --- RUN-TIME PHASES (time-consuming, tasks, parallel) ---
 // run_phase(phase)             TASK        Main stimulus/checking loop
 //
 // run_phase runs in PARALLEL with these 12 fine-grain phases:
 //    pre_reset_phase           TASK        Before DUT comes out of reset
 //    reset_phase               TASK        Drive and release reset
 //    post_reset_phase          TASK        After reset released
 //    pre_configure_phase       TASK        Before configuration
 //    configure_phase           TASK        Configure DUT registers
 //    post_configure_phase      TASK        After configuration
 //    pre_main_phase            TASK        Before main stimulus
 //    main_phase                TASK        Main test stimulus
 //    post_main_phase           TASK        After main stimulus
 //    pre_shutdown_phase        TASK        Before shutdown
 //    shutdown_phase            TASK        Final drain/flush
 //    post_shutdown_phase       TASK        After shutdown
 //
 // --- CLEANUP PHASES (zero-time, functions, bottom-up) ---
 // extract_phase(phase)         FUNCTION    Extract results from components
 // check_phase(phase)           FUNCTION    Check for errors/warnings
 // report_phase(phase)          FUNCTION    Print final report/summary
 // final_phase(phase)           FUNCTION    Last chance cleanup
 //
 // Summary: 4 build functions + 13 run tasks + 4 cleanup functions
 // build/connect/elab = top-down; connect onward = bottom-up
 // run_phase = all components start simultaneously

 // (b) Why run_phase must be a task:
 // Functions in SystemVerilog are ZERO-TIME: they cannot consume
 // simulation time, block on events, or call @(posedge clk).
 // run_phase drives stimulus and waits for responses over many
 // clock cycles -> it MUST be able to block on time events.
 // Tasks allow time passage: @(posedge clk), #10, wait(...).
 // If run_phase were a function, it could never drive signals
 // or wait for DUT responses.

 // (c) uvm_component vs uvm_object -- phase participation:
 //
 // uvm_component:
 //   - Has a NAME and PARENT in the UVM hierarchy
 //   - PARTICIPATES in all UVM phases automatically
 //   - Exists for the entire simulation lifetime
 //   - Examples: uvm_driver, uvm_monitor, uvm_scoreboard,
 //               uvm_agent, uvm_env, uvm_test, uvm_sequencer
 //
 // uvm_object:
 //   - Has NO fixed place in hierarchy (no parent/child)
 //   - Does NOT participate in phases
 //   - Created and destroyed as needed (transient lifetime)
 //   - Examples: uvm_sequence_item, uvm_sequence,
 //               uvm_reg_block, uvm_transaction, uvm_config_object
```

```
//
// Rule of thumb:
//   "Does it need to run over time / respond to phases?" -> component
//   "Is it just a data container or algorithm?" -> object

// (d) Consequence of missing super.build_phase(phase):
//
// uvm_component::build_phase() does TWO critical things:
//   1. Calls uvm_config_db::get() for fields tagged with
//      `uvm_field_* macros (applies config automatically)
//   2. Triggers automatic creation of child components if using

//      factory-based creation macros
//
// If you omit super.build_phase(phase):
//   - Config values set via uvm_config_db may not be applied
//   - `uvm_field_* automation (copy/print/compare) may break
//   - In some UVM versions, the phase machinery itself may skip
//     propagation to children -> children never get built
//
// Best practice: ALWAYS call super.build_phase(phase) as the
// FIRST line in your overridden build_phase.
//   function void build_phase(uvm_phase phase);
//     super.build_phase(phase);    // MUST be first
//     // your code here
//   endfunction
```

解释

UVM Phase 是面试必考点，考官常问 "XXX 是 task 还是 function？为什么？"

核心记忆方法：

几乎所有 function phase 都在 run_phase 前后，

run_phase 及其 12 个细化 phase 全部是 task，

其余的全部是 function。

Task vs Function 的本质区别：

Function 是零时间的，不能消耗仿真时间（不能用 @、#、wait）。

Task 可以阻塞时间，可以驱动信号和等待响应。

uvm_component vs uvm_object 的区别是高频考点：

Component 有层次结构、有生命周期、参与 phase；

Object 是数据容器，无层次结构，不参与 phase。

super.build_phase() 必须调用：

很多同学会忘这一点，导致 config_db 的字段自动配置失效。

## Q71. UVM: Inter-process Synchronization Methods

QUESTION

What inter-process synchronization methods are available in

SystemVerilog and UVM? Explain each and their use cases.

Also explain what a barrier is.

ANSWER

```
// SV Built-in synchronization primitives:
 //
 // 1. event
 //     Lightweight signal between threads.
 //     -> e: trigger,     @e or wait(e.triggered): wait
 //     Use: simple notification (e.g. "phase done", "data ready")
 //
 // 2. semaphore
 //     Mutual exclusion / resource counting.
 //     sem.get(N): acquire N keys (blocks if unavailable)
 //     sem.put(N): release N keys
 //     Use: limit concurrent access (e.g. single-port memory model)
 //
 // 3. mailbox #(T)
 //     Typed message queue between producer and consumer threads.
 //     mbx.put(item): send (blocks if bounded and full)
 //     mbx.get(item): receive (blocks if empty)
 //     mbx.try_put/try_get: non-blocking versions
 //     Use: monitor -> scoreboard data passing
 //
 // UVM-specific synchronization:
 //
 // 4. uvm_event / uvm_event_pool
 //     Named events accessible by string key across components.
 //     Supports callbacks (add_callback) and status queries.
 //     Use: cross-component synchronization without handle passing
 //
 // 5. uvm_barrier
 //     A rendezvous point: N threads all call wait_for() and
 //     ALL are released only when ALL N have arrived.
 //     uvm_barrier b = new("b", 3); // 3 threads must arrive
 //     b.wait_for(); // each of 3 threads calls this
 //     Use: synchronize multiple sequences/agents before proceeding
 //          (e.g. "all agents initialized, start test simultaneously")
 //

 // 6. uvm_objection
 //     Phase-level synchronization: phase ends only when all
 //     objections are dropped.
 //     phase.raise_objection(this) / phase.drop_objection(this)
```

解释

进程间同步是 UVM 高级用法的重要考点。

event 最轻量，适合简单通知；semaphore 适合资源互斥；

mailbox 是最常用的生产者-消费者数据传递方式。

uvm_barrier 是面试中经常被单独问到的：它实现了"集合点"语义，

所有参与者都到达后才能继续，常用于多 agent 需要同步启动的场景。

uvm_event 与 SV event 的区别：uvm_event 有全局命名，

任何组件只要知道名字就能访问，不需要传递句柄。

## Q72. UVM: Code Coverage Types

QUESTION

What types of code coverage exist in verification?

Describe each type briefly.

ANSWER

```
// 1. Line Coverage (Statement Coverage)
 //      Was every line of RTL code executed?
 //      Simplest metric. Missing line coverage = dead code.
 //
 // 2. Branch Coverage (Decision Coverage)
 //      Was every branch of every if/case/ternary taken?
 //      Catches conditions where one path is never exercised.
 //      Example: if/else where else is never taken.
 //
 // 3. Condition Coverage
 //      Was each individual boolean sub-expression evaluated
 //      to both true and false?
 //      Finer-grained than branch coverage.
 //      Example: if (a && b) -> need {a=0,b=X} and {a=1,b=0} and {a=1,b=1}
 //
 // 4. Toggle Coverage
 //      Did each signal/bit toggle from 0->1 and 1->0?
 //      Catches stuck-at faults and unexercised data paths.
 //      Important for datapath verification.
 //
 // 5. FSM Coverage
 //      Were all states visited? Were all state transitions taken?
 //      Critical for control-heavy designs.
 //
 // 6. Functional Coverage (not code coverage, but related)
 //      User-defined covergroups measuring design-intent coverage.
 //      Complements code coverage to ensure meaningful scenarios tested.
 //
 // Hierarchy: Toggle < Statement < Branch < Condition < FSM
 // Higher levels generally imply lower levels are also covered.
```

解释

Code coverage 是验证完备性的量化指标，面试常考各类型的定义和区别。

重要认识：100% line coverage 不代表测试完备，

因为不同的条件组合可能执行相同的行。

Branch coverage 比 line coverage 更严格，

condition coverage 又比 branch 更细粒度。

Toggle coverage 是 datapath 验证的重点，发现信号被"卡住"的问题。

Functional coverage 是设计意图层面的覆盖，代码覆盖率无法替代它。

## Q73. UVM: DMA Scoreboard Implementation

QUESTION

A DMA module sends requests to memory (with ADDR and ID), and

memory responds out-of-order (by ID). The DUT then outputs data.

DMA Requests: Addr=h32 ID=h4245, Addr=h33 ID=h421,

Addr=h34 ID=h3234, Addr=h35 ID=h2415

DMA Responses: ID=h421 Data=h78DC1207,

ID=h2415 Data=hBC087A13,

ID=h4245 Data=h914758A1,

ID=h3234 Data=h4378ABCD

(a) What is the expected output order from the DUT?

(b) Implement the three scoreboard write functions:

write_dma_req(), write_dma_rsp(), write_dma_data()

ANSWER

```
// (a) Expected output order from DUT:
 // Requests were issued in address order: h32, h33, h34, h35
 // Corresponding IDs:                      h4245, h421, h3234, h2415
 // Memory responds out-of-order by ID, but DUT must output
 // data in the ORIGINAL REQUEST ORDER (in-order completion):
 //    Addr h32 -> ID h4245 -> Data h914758A1
 //    Addr h33 -> ID h421   -> Data h78DC1207
 //    Addr h34 -> ID h3234 -> Data h4378ABCD
 //    Addr h35 -> ID h2415 -> Data hBC087A13
 //
 // Expected DUT output order:
 //    h914758A1, h78DC1207, h4378ABCD, hBC087A13

 // (b) Scoreboard implementation
 class dma_scoreboard extends uvm_scoreboard;
   `uvm_component_utils(dma_scoreboard)

   // Map: ID -> expected data (populated from DMA responses)
   logic [31:0] id_to_data [logic [31:0]];

   // Queue: ordered list of IDs in request order
   logic [31:0] req_id_queue [$];

   // Queue: expected output data in order
   logic [31:0] expected_data_q [$];

   // Called when monitor sees a DMA request
   virtual function void write_dma_req(dma_req_pkt pkt);
      req_id_queue.push_back(pkt.id);
      `uvm_info("SCBD", $sformatf(
         "REQ: Addr=%0h ID=%0h", pkt.addr, pkt.id), UVM_LOW)
   endfunction

   // Called when monitor sees a DMA memory response
   virtual function void write_dma_rsp(dma_rsp_pkt pkt);

      // Store data keyed by ID
      id_to_data[pkt.id] = pkt.data;
      `uvm_info("SCBD", $sformatf(
         "RSP: ID=%0h Data=%0h", pkt.id, pkt.data), UVM_LOW)

      // Check if we can now resolve any pending requests in order
      while (req_id_queue.size() > 0 &&
             id_to_data.exists(req_id_queue[0])) begin
         logic [31:0] next_id    = req_id_queue.pop_front();
         logic [31:0] next_data = id_to_data[next_id];
         expected_data_q.push_back(next_data);
         id_to_data.delete(next_id);
      end
     endfunction
```

```
    // Called when monitor sees DUT output data
    virtual function void write_dma_data(dma_data pkt);
     if (expected_data_q.size() == 0) begin
        `uvm_error("SCBD", "Unexpected data: no pending expected")
        return;
     end
     logic [31:0] exp = expected_data_q.pop_front();
     if (pkt.data !== exp)
        `uvm_error("SCBD", $sformatf(
          "MISMATCH: got=%0h exp=%0h", pkt.data, exp))
     else
        `uvm_info("SCBD", $sformatf("PASS: data=%0h", pkt.data), UVM_LOW)
    endfunction

 endclass
```

解释

这道题考察 out-of-order DMA 的 scoreboard 设计，是 Apple 面试的核心考题。

关键难点：memory 按 ID 乱序返回，但 DUT 必须按原始请求顺序输出。

Scoreboard 需要维护两个数据结构：

1. req_id_queue：按请求顺序记录 ID（保序）。

2. id_to_data：按 ID 存储 response 数据（无序）。

每次收到 response，尝试从队头开始匹配，如果队头的 ID 已有 response 就弹出，

这样自然地将乱序 response 转换为有序期望数据。

write_dma_data 负责和实际 DUT 输出比较。

## Q74. UVM: Design a DMA Engine uvm_sequence_item

QUESTION

You are verifying a DMA (Direct Memory Access) engine.

(a) Design a uvm_sequence_item class for the DMA. What fields

should be included and why?

(b) Which fields should be rand vs non-rand?

(c) Write key constraints that reflect realistic DMA behavior.

(d) What utility macros should be included and why?

ANSWER

```
class dma_seq_item extends uvm_sequence_item;
  `uvm_object_utils_begin(dma_seq_item)
      `uvm_field_int(src_addr,     UVM_ALL_ON)
      `uvm_field_int(dst_addr,     UVM_ALL_ON)
      `uvm_field_int(length,       UVM_ALL_ON)
      `uvm_field_int(burst_type,   UVM_ALL_ON)
      `uvm_field_int(src_stride,   UVM_ALL_ON)
      `uvm_field_int(dst_stride,   UVM_ALL_ON)
      `uvm_field_int(interrupt_en, UVM_ALL_ON)
  `uvm_object_utils_end

  // rand: driver randomizes these to generate transactions
  rand bit [31:0] src_addr;        // source start address
```

```
  rand bit [31:0] dst_addr;        // destination address
  rand bit [15:0] length;          // transfer length in bytes
  rand bit [1:0]   burst_type;     // 0=SINGLE 1=INCR4 2=INCR8 3=INCR16
  rand bit [15:0] src_stride;      // source row stride (2D DMA)
  rand bit [15:0] dst_stride;      // destination row stride (2D DMA)
  rand bit         interrupt_en;   // assert interrupt on completion

  // non-rand: status driven by DUT response, captured by monitor
  bit              done;            // transfer completed
  bit              error;           // bus error or timeout
  bit [31:0]       bytes_transferred; // actual bytes moved

  // Address alignment per burst type
  constraint c_align {
      (burst_type == 2'd1) -> src_addr[1:0] == 2'b00; // INCR4: 4B align
      (burst_type == 2'd2) -> src_addr[2:0] == 3'b000;// INCR8: 8B align
      (burst_type == 2'd3) -> src_addr[3:0] == 4'h0;    // INCR16: 16B align
      (burst_type == 2'd1) -> dst_addr[1:0] == 2'b00;
      (burst_type == 2'd2) -> dst_addr[2:0] == 3'b000;
      (burst_type == 2'd3) -> dst_addr[3:0] == 4'h0;
  }

     // Realistic transfer sizes (word-aligned, max 4KB)
     constraint c_length {
         length inside {[1:4096]};
         length % 4 == 0;
     }

     // Prevent src/dst overlap (simplified)
     constraint c_no_overlap {
         (dst_addr >= src_addr + {16'b0, length}) ||
         (src_addr >= dst_addr + {16'b0, length});
     }

     // Stay within 32-bit address space
     constraint c_addr_range {
         src_addr + {16'b0, length} <= 32'hFFFF_FFFF;
         dst_addr + {16'b0, length} <= 32'hFFFF_FFFF;
     }

     function new(string name = "dma_seq_item");
         super.new(name);
     endfunction
  endclass

  // Why each field:
  // src/dst_addr : core descriptor; drives scoreboard reference model
  // length         : determines data volume; tests boundary conditions
  // burst_type     : exercises different AHB/AXI burst modes
  // stride         : 2D DMA for image/matrix copy (row pitch)
  // interrupt_en : tests interrupt generation path
  // done/error     : status fields; monitor writes these, not driver
  //
  // Why uvm_field macros:
  // Auto-generates copy(), compare(), print(), pack()/unpack()
  // Scoreboard uses compare() for golden check; print() for debug.
```

解释

DMA sequence_item 设计考察对 DMA 协议和 UVM 对象设计的综合理解。

四个设计原则：

1. rand 字段：所有驱动 DMA 操作的参数（地址、长度、类型）。

2. 非 rand 字段：DUT 返回的状态（done/error），由 monitor 填写。

3. 约束反映协议：地址对齐约束对 AHB/AXI burst 是强制要求。

4. uvm_field 宏：scoreboard compare() 和调试 print() 的基础。

面试进阶：能说出为什么地址对齐约束对 burst 传输是必要的。

## Q75. SV: Associative Array vs Regular Array in UVM Scoreboard

QUESTION

In a UVM scoreboard or reference model, you need to store
transaction data for later comparison.
Compare using an associative array vs a regular (fixed/dynamic)
array for this purpose:
(a) What are the advantages and disadvantages of each?

(b) In what scenarios would you choose one over the other?
(c) Give a concrete UVM scoreboard example showing both approaches.

ANSWER

```
// Associative array: bit[31:0] data[bit[31:0]]
 //    Key = any sparse value (address, transaction ID, etc.)
 //    Memory allocated on demand per entry
 //
 // Regular dynamic array: bit[31:0] data[]
 //    Key = integer index [0..N-1], contiguous allocation

 // (a) Comparison:
 //
 // +-----------------+-----------------------+-----------------------+
 // | Property            | Associative Array        | Regular/Dynamic Array   |
 // +-----------------+-----------------------+-----------------------+
 // | Index type          | Any type (string, int) | Integer [0..N-1] only     |
 // | Memory              | Sparse, on-demand        | Contiguous block        |
 // | Access time         | O(log N) hash/tree       | O(1) direct index       |
 // | Exists check        | .exists(key) built-in    | Manual bounds check     |
 // | Iteration           | foreach, first/next      | foreach [i]             |
 // | Memory overhead     | Higher per entry         | Lower, predictable      |
 // | Delete entry        | .delete(key)             | Not supported natively |
 // | Size known?         | .num() dynamic           | .size() dynamic         |
 // +-----------------+-----------------------+-----------------------+
 //
 // Associative array pros:
 //    - Natural key = transaction address or ID (no manual mapping needed)
 //    - SPARSE STORAGE: only allocates memory for entries that actually exist.
 //      Critical advantage: if your address space is 2^32 (4B addresses)
 //      but a test only touches 100 unique addresses, the associative array
 //      uses memory for 100 entries only.
 //      A regular array pre-allocated for 2^32 entries would require
 //      4G * 4 bytes = 16 GB -- completely impractical.
 //      This is the single most important reason to use associative arrays
 //      in a scoreboard or reference model.
 //    - Built-in .exists() prevents invalid key access
 //    - .delete() cleanly removes matched entries in scoreboard
 //
```

```
// Associative array cons:
//   - Slower access than regular array (hash lookup overhead)
//   - Higher memory overhead per entry (hash table pointers)
//   - Non-deterministic iteration order
//   - Harder to check "all entries consumed" at end of sim
//
// Regular/dynamic array pros:
//   - O(1) access by index (cache-friendly)
//   - Lower memory footprint for dense, sequential data
//   - Predictable iteration order
//   - Better for ordered queues (use queue $[] instead)
//
// Regular array cons:
//   - Index must be 0-based integer (need manual key-to-index mapping)
//   - Must pre-size or resize dynamically
//   - No built-in exists() -- out-of-bounds access is a bug
//   - Cannot "delete" a middle entry without shifting

// (b) When to choose:
//
// Use ASSOCIATIVE array when:
//   - Key is an address, transaction ID, or string (non-sequential)
//   - Transactions arrive out of order and are matched by ID/addr
//   - Data is sparse (few entries out of large key space)
//   - You need to check/delete individual entries frequently
//   Example: OOO DMA scoreboard, AXI ID-based matching
//
// Use REGULAR/QUEUE array when:
//   - Transactions arrive in order (FIFO matching)
//   - Index is naturally sequential (packet number 0,1,2...)
//   - Performance is critical (O(1) vs O(log N) access)
//   - Memory is constrained and density is high
//   Example: simple APB scoreboard (in-order), coverage collector

// (c) UVM scoreboard examples:

 //
 // Approach 1: Associative array (keyed by address)
 class scbd_assoc extends uvm_scoreboard;
     bit [31:0] exp_data [bit [31:0]];    // addr -> expected data

     function void write_expected(bit [31:0] addr, bit [31:0] data);
      exp_data[addr] = data;
     endfunction

     function void check_actual(bit [31:0] addr, bit [31:0] actual);
      if (!exp_data.exists(addr))
         `uvm_error("SCBD", $sformatf("No expected for addr %0h", addr))
      else begin
         if (exp_data[addr] !== actual)
            `uvm_error("SCBD", $sformatf(
              "MISMATCH addr=%0h exp=%0h act=%0h", addr, exp_data[addr], actual))
         exp_data.delete(addr);   // remove matched entry
      end
     endfunction

     function void check_phase(uvm_phase phase);
      if (exp_data.num() != 0)
         `uvm_error("SCBD", $sformatf(
            "%0d unmatched entries remain", exp_data.num()))
     endfunction
 endclass
```

```
// Approach 2: Queue (for in-order FIFO scoreboard)
class scbd_queue extends uvm_scoreboard;
    bit [31:0] exp_q [$];   // FIFO queue of expected values

    function void write_expected(bit [31:0] data);
     exp_q.push_back(data);
    endfunction

    function void check_actual(bit [31:0] actual);
     if (exp_q.size() == 0)
        `uvm_error("SCBD", "Unexpected transaction: queue empty")
     else begin
        bit [31:0] exp = exp_q.pop_front();
        if (exp !== actual)
           `uvm_error("SCBD", $sformatf("MISMATCH exp=%0h act=%0h", exp, actual))
     end
    endfunction
endclass
```

解释

这道题考察数据结构选择的工程判断，没有绝对答案。

关联数组最大的优势是稀疏存储，节省内存极其显著：

假设地址空间是 2^32（40亿个地址），但一个测试只访问了 100 个地址。

用普通数组预分配 2^32 个条目需要 16GB 内存，完全不可行。

用关联数组只分配这 100 个实际出现的条目，内存占用微乎其微。

这在 scoreboard 和 reference model 中是最常见的使用场景。

其他关联数组优势：任意类型作键（地址/ID/字符串）、内置 exists()、可删除单条目。

最适合乱序（OOO）匹配场景：DMA、AXI、PCIe 等有 ID 的协议。

队列的核心优势：O(1) 访问，顺序保证，内存紧凑（数据本身是密集的情况）。

最适合顺序（in-order）匹配场景：APB、SPI 等简单协议。

面试重点：能说出"为什么选这个"而不只是"这个怎么用"。

关键词：稀疏 -> 关联数组；密集有序 -> 队列或普通数组。

## Q76. UVM: Fix Scoreboard to Support Byte-Enable Writes

QUESTION

The following UVM scoreboard has a bug: it assumes full 32-bit

writes but the DUT supports byte-enable writes (individual bytes

can be written independently). The comment marks the bug location.

Fix it to correctly handle byte-enable (strobe) writes.

```
class mem_scoreboard extends uvm_scoreboard;
   bit [31:0] ref_mem [bit [31:0]];      // addr -> 32-bit reference memory


   // BUG: does not account for byte enables
   function void write_txn(
```

```
    bit [31:0] addr,
    bit [31:0] wdata,
    bit [3:0]   wstrb,     // byte enable: wstrb[i]=1 -> byte i is written
    bit         we         // write enable
  );
    if (we)
       ref_mem[addr] = wdata;    // BUG: overwrites all 4 bytes always
  endfunction


  function void check_read(bit [31:0] addr, bit [31:0] rdata);
    if (!ref_mem.exists(addr))
       `uvm_error("SCBD", "Read from uninitialized address")
    else if (ref_mem[addr] !== rdata)
       `uvm_error("SCBD", $sformatf(
         "MISMATCH: exp=%0h act=%0h", ref_mem[addr], rdata))
  endfunction
endclass
```

ANSWER

```
// Root cause of bug:
// ref_mem[addr] = wdata overwrites ALL 32 bits regardless of wstrb.
// If only byte 0 is enabled (wstrb=4'b0001), bytes 1,2,3 should
// retain their previous values in the reference model.
//
// Fix: apply wstrb byte-by-byte, only updating enabled bytes.
//
// wstrb[0] = 1 -> write wdata[7:0]      to ref_mem[addr][7:0]
// wstrb[1] = 1 -> write wdata[15:8]     to ref_mem[addr][15:8]
// wstrb[2] = 1 -> write wdata[23:16] to ref_mem[addr][23:16]
// wstrb[3] = 1 -> write wdata[31:24] to ref_mem[addr][31:24]

class mem_scoreboard_fixed extends uvm_scoreboard;
  bit [31:0] ref_mem [bit [31:0]];    // addr -> 32-bit reference memory

  function void write_txn(
    bit [31:0] addr,
    bit [31:0] wdata,
    bit [3:0]   wstrb,
    bit         we
  );
    if (!we) return;

    // Initialize to 0 if address not yet seen
    if (!ref_mem.exists(addr))
      ref_mem[addr] = 32'h0;

    // Apply byte enables: only update bytes where wstrb[i]==1
    for (int i = 0; i < 4; i++) begin
      if (wstrb[i]) begin
          // Update byte i: bits [8i+7 : 8i]
          ref_mem[addr][8*i +: 8] = wdata[8*i +: 8];
      end
      // wstrb[i]==0: byte i retains its current value -> no update
    end

    endfunction

    function void check_read(
      bit [31:0] addr,
      bit [31:0] rdata,
```

```
      bit [3:0]     rstrb = 4'hF   // which bytes are being read (optional)
    );
      bit [31:0] exp;
      if (!ref_mem.exists(addr)) begin
          `uvm_error("SCBD", $sformatf(
            "Read from uninitialized addr=%0h", addr))
          return;
      end
      exp = ref_mem[addr];
      // Only compare enabled bytes
      for (int i = 0; i < 4; i++) begin
          if (rstrb[i] && (rdata[8*i +: 8] !== exp[8*i +: 8]))
            `uvm_error("SCBD", $sformatf(
              "MISMATCH byte%0d addr=%0h exp=%0h act=%0h",
              i, addr, exp, rdata))
      end
    endfunction
endclass

// Example test scenario showing why bug matters:
// Write 0xAABBCCDD to addr=0x100 with wstrb=4'b1111 (all bytes)
//      ref_mem[0x100] = 0xAABBCCDD    (correct for both buggy and fixed)
//
// Write 0xFFFFFFFF to addr=0x100 with wstrb=4'b0001 (byte 0 only)
//      Buggy:   ref_mem[0x100] = 0xFFFFFFFF   (wrong! bytes 1,2,3 overwritten)
//      Fixed:   ref_mem[0x100] = 0xAABBCCFF   (correct: only byte 0 updated)
//
// Key SV syntax: [8*i +: 8] means "8 bits starting at bit 8*i"
// This is the variable part-select operator, essential for byte lanes.
```

解释

Byte enable（字节使能）/ write strobe 是总线协议（AHB、AXI）的核心特性，

scoreboard 不支持 byte enable 是实际工程中最常见的 TB bug 之一。

修复关键：用 for 循环按位检查 wstrb，只更新被使能的字节。

SV 的变长部分选择 [8*i +: 8] 是按字节操作的标准写法：

i=0: [0+:8] = [7:0] (byte 0)

i=1: [8+:8] = [15:8] (byte 1)

i=2: [16+:8] = [23:16] (byte 2)

i=3: [24+:8] = [31:24] (byte 3)

注意初始化问题：如果地址从未被写过，ref_mem.exists() 返回 false，

需要初始化为 0（或者用 X 表示未初始化更严格）。

## Q77. SV/C++: Data Structure for Sparse Memory in Verification

QUESTION

In a verification environment, you need to model a large memory

space (e.g. 64-bit address space) where only a small fraction

of addresses are ever accessed during simulation.

(a) What data structure would you use to implement this sparse

memory model? Why is a regular array not suitable?

(b) Compare hash map vs balanced tree (e.g. std::map) for this use case. What are the tradeoffs?

(c) Show a concrete SystemVerilog and C++ implementation.

(d) What additional considerations apply in a UVM scoreboard or reference model context?

ANSWER

```
// (a) Why NOT a regular array:
 //
 // A 64-bit address space has 2^64 possible addresses.
 // If each location stores 1 byte, a fully allocated array needs
 // 2^64 bytes = 16 exabytes -- physically impossible.
 // Even for a 32-bit space (4GB), pre-allocating a byte array
 // wastes enormous memory if only 1000 addresses are touched.
 //
 // Example: test accesses 500 unique addresses out of 2^32.
 //    Regular array: 4GB allocation
 //    Hash map:      ~500 entries * ~64 bytes overhead = ~32 KB
 // -> Hash map saves >100,000x memory for sparse access patterns.
 //
 // The fundamental issue: array index == address requires
 // contiguous allocation of the entire range, even for unused gaps.

 // (b) Hash map vs Balanced tree tradeoffs:
 //
 // +--------------------+--------------------------+--------------------------+
 // | Property           | Hash Map (unordered)     | Balanced Tree (ordered)  |
 // +--------------------+--------------------------+--------------------------+
 // | Insert             | O(1) average             | O(log N)                 |
 // | Lookup             | O(1) average             | O(log N)                 |
 // | Worst case         | O(N) on hash collision   | O(log N) guaranteed      |
 // | Ordered iteration  | No (random order)        | Yes (sorted by address)  |
 // | Range query        | Not supported natively   | Efficient (lower_bound)  |
 // | Memory overhead    | Higher (hash buckets)    | Lower per entry          |
 // | SV equivalent      | associative array [addr] | No direct equivalent     |
 // +--------------------+--------------------------+--------------------------+
 //
 // For verification reference memory:
 //    Hash map is usually preferred:
 //      - O(1) average access matches DUT's random access pattern
 //      - Simpler implementation in both SV and C++
 //      - SV associative array IS a hash map internally

//     Balanced tree is preferred when:
//      - You need to iterate addresses in order (e.g. dump memory)
//      - You need range queries (find all writes to [base, base+4KB])
//      - Worst-case latency matters (hash collisions unacceptable)

// (c) SystemVerilog implementation (associative array = hash map):
class sparse_mem_sv;
  // Key: 64-bit byte address, Value: 8-bit data
  bit [7:0] mem [bit [63:0]];

  function void write(bit [63:0] addr, bit [7:0] data);
     mem[addr] = data;
  endfunction

  function bit [7:0] read(bit [63:0] addr);
```

```
    if (!mem.exists(addr)) begin
       `uvm_warning("MEM", $sformatf(
          "Read uninit addr=0x%0h, returning 0", addr))
       return 8'h00;
    end
    return mem[addr];
  endfunction

  // Write N bytes (word write with byte enables)
  function void write_bytes(
    bit [63:0]   base_addr,
    bit [7:0]    data[],
    bit          be[]         // byte enables
  );
    foreach (data[i])
       if (be[i]) mem[base_addr + i] = data[i];
  endfunction

  function int unsigned num_entries();
    return mem.num();

  endfunction

  function void clear();
      mem.delete();
  endfunction
endclass

// C++ implementation (hash map and tree options):
#include <unordered_map>       // hash map
#include <map>                 // balanced BST (red-black tree)
#include <cstdint>

// Option 1: hash map (O(1) average, unordered)
class SparseMemHash {
  std::unordered_map<uint64_t, uint8_t> mem_;
public:
  void write(uint64_t addr, uint8_t data) {
      mem_[addr] = data;
  }
  uint8_t read(uint64_t addr) const {
      auto it = mem_.find(addr);
      return (it != mem_.end()) ? it->second : 0x00;
  }
  bool exists(uint64_t addr) const {
      return mem_.count(addr) > 0;
  }
  size_t size() const { return mem_.size(); }
};

// Option 2: balanced tree (O(log N), ordered iteration + range queries)
class SparseMemTree {
  std::map<uint64_t, uint8_t> mem_;
public:
  void write(uint64_t addr, uint8_t data) {
      mem_[addr] = data;

  }
  uint8_t read(uint64_t addr) const {
      auto it = mem_.find(addr);
      return (it != mem_.end()) ? it->second : 0x00;
  }
  // Range query: get all entries in [lo, hi]
```

```
  void dump_range(uint64_t lo, uint64_t hi) const {
      auto it = mem_.lower_bound(lo);
      while (it != mem_.end() && it->first <= hi) {
          printf("   addr=0x%lx data=0x%02x\n", it->first, it->second);
          ++it;
      }
  }
};

// (d) UVM scoreboard context:
//     - Use associative array in SV for simplicity and UVM integration
//     - Call mem.exists() before every read to catch uninitialized access
//     - In check_phase: verify mem.num() == 0 (all expected writes matched)
//       or dump remaining entries as errors
//     - For word-width memories (32/64-bit), store at word granularity:
//       bit [31:0] mem [bit [31:0]] to reduce entry count 4x or 8x
```

解释

稀疏内存的核心问题是键空间远大于实际使用量，数组会造成天文数字的内存浪费。

哈希表（SV 的 associative array）是首选：O(1) 平均访问，实现简单，

SV 内置支持 exists() 和 delete()，完全满足验证场景需求。

平衡树（std::map）的优势在于有序性和范围查询，适合需要按地址顺序迭代的场景，

例如在仿真结束时 dump 内存内容或验证某个地址区间的数据完整性。

面试回答要点：先说不用数组的原因（内存浪费），再说哈希表 vs 树的 tradeoff，

能说出 O(1) vs O(log N) 和有序性的区别就是加分项。

## Q78. Verification: Handling Concurrent Writes to Sparse Memory Model

QUESTION

Multiple agents or threads are writing to the same sparse memory reference model concurrently (e.g. multiple DMA channels, CPUs, or bus masters all issuing writes simultaneously in simulation).

(a) What problems arise when multiple threads write to the same reference memory model without synchronization?

(b) For a NON-golden-model component (e.g. a passive scoreboard that just records data), what is the simplest approach?

(c) For a GOLDEN MODEL (reference model that must match the DUT's exact operation order), how do you correctly handle concurrent writes? Why is simply adding a mutex insufficient?

(d) Show a concrete implementation of the golden model approach using a request FIFO / serialization queue.

ANSWER

```
// (a) Problems with unsynchronized concurrent writes:
//
// Race condition on shared state:
```

```
//   Thread A reads mem[addr], Thread B reads mem[addr] simultaneously.
//   Both compute a new value and write back -- one update is lost.
//   In SV: multiple processes updating an associative array entry
//   in the same time step can produce non-deterministic results.
//
// Non-deterministic ordering:
//   If Thread A and Thread B both write mem[0x100] at "the same time",
//   which value ends up in the model depends on scheduler ordering.
//   This is fine for a passive recorder but WRONG for a golden model
//   that must predict what the DUT will produce.
//
// Silent data corruption:
//   In C++ without mutexes, concurrent map writes cause undefined
//   behavior (iterator invalidation, partial writes, crashes).

// (b) Simple approach for NON-golden-model (passive scoreboard):
//
// If the component is single-threaded (one UVM process handles all
// write_* callbacks via analysis ports), no mutex is needed.
// UVM analysis port callbacks are already serialized by the
// UVM scheduler -- write() functions run one at a time.
//
// In SystemVerilog, multiple always/initial blocks run concurrently
// but associative array access within a single always block is safe.
// The key insight: UVM component callbacks (write functions) are
// functions, not tasks -- they execute atomically within a time step.
//
// Only add a mutex (semaphore) if:
//   - You use fork/join to spawn multiple threads that share the map
//   - You have explicit parallel processes writing the same model
//
// Simple passive scoreboard (no sync needed if single-threaded):

class passive_scbd extends uvm_scoreboard;
  bit [7:0] ref_mem [bit [63:0]];

  // Called from analysis port -- single-threaded by UVM framework
  function void write(mem_txn txn);
     if (txn.we)
       ref_mem[txn.addr] = txn.data;    // safe: only one writer at a time
  endfunction
endclass

// (c) Golden model: why mutex alone is insufficient:
//
// A mutex (semaphore) only prevents simultaneous access -- it does NOT
// define WHICH write wins or in WHAT ORDER writes are applied.
//
// Problem: the DUT processes writes in a specific order determined by
// its internal arbitration (bus arbiter, priority encoder, etc.).
// The golden model must apply writes in EXACTLY the same order.
//
// If Thread A (CPU write) and Thread B (DMA write) both arrive at
// the golden model "simultaneously" but the DUT's bus arbiter
// gives priority to CPU, the model must also apply CPU write first.
//
// A mutex might let Thread B win the lock first -> model state differs
// from DUT state -> false mismatch or missed real mismatch.
//
// Solution: SERIALIZE writes through a REQUEST FIFO.
// The order of entries in the FIFO must match the order the DUT
// actually processes them (same arbitration result).
//
```

```
// How to get the correct order:
//    - Monitor the DUT's bus/interconnect OUTPUT (after arbitration)
//    - Push the transaction into the golden model FIFO only AFTER
//      it is observed to win arbitration on the DUT side
//    - The FIFO then applies them one by one to the reference memory

//    - This guarantees model order == DUT order

// (d) Golden model implementation with serialization FIFO:
class golden_mem_model extends uvm_component;
  `uvm_component_utils(golden_mem_model)

  // Serialized request queue
  // Populated by monitor AFTER observing DUT arbitration result
  uvm_tlm_analysis_fifo #(mem_txn) req_fifo;

  // Reference memory (only accessed by the single consumer thread)
  bit [7:0] ref_mem [bit [63:0]];

  function void build_phase(uvm_phase phase);
     super.build_phase(phase);
     req_fifo = new("req_fifo", this);
  endfunction

  // Single consumer task: pops and applies one request at a time
  // This is the ONLY writer to ref_mem -> no mutex needed
  task run_phase(uvm_phase phase);
     mem_txn txn;
     forever begin
       // Blocking get: wait for next request in order
       req_fifo.get(txn);
       // Apply to reference model in the order they were enqueued
       apply_txn(txn);
     end
  endtask

  function void apply_txn(mem_txn txn);
     if (txn.we) begin
       // Byte-enable aware write
       for (int i = 0; i < txn.size; i++)
           if (txn.be[i])

               ref_mem[txn.addr + i] = txn.data[8*i +: 8];
      end
     endfunction

     // Monitor calls this after seeing the transaction WIN arbitration on DUT
     // The ORDER of calls to this function defines the model's write order
     function void write_request(mem_txn txn);
      req_fifo.write(txn);   // enqueue -- preserves arrival order
     endfunction

     // Scoreboard comparison: called after DUT produces a read result
     function void check_read(bit [63:0] addr, bit [7:0] actual);
      if (!ref_mem.exists(addr))
         `uvm_error("GOLDEN", $sformatf("Uninit read addr=0x%0h", addr))
      else if (ref_mem[addr] !== actual)
         `uvm_error("GOLDEN", $sformatf(
           "MISMATCH addr=0x%0h exp=0x%0h act=0x%0h",
           addr, ref_mem[addr], actual))
     endfunction
 endclass
```

```
// Key design decisions:
// 1. Monitor on DUT bus captures transactions POST-arbitration.
//      This is the critical observation point -- only after the DUT
//      decides who wins does the monitor push to the golden model FIFO.
//
// 2. Single consumer (run_phase task) is the ONLY thread that reads
//      or writes ref_mem. No mutex needed because there is exactly
//      one writer -- serialization is achieved by design, not by locking.
//
// 3. FIFO order == DUT arbitration order.
//      If DUT arbitrates A before B, monitor pushes A before B.
//      Golden model applies A before B. Model state matches DUT state.
//
// 4. Multiple producers (multiple monitors for multiple bus masters)

//      can all call write_request() -- uvm_tlm_analysis_fifo is
//      thread-safe for concurrent writes on the producer side.
```

解释

这道题考察验证架构设计能力，是高级 UVM 面试的核心题目。

非 golden model（被动 scoreboard）：

UVM 的 analysis port write() 回调本身就是串行的（function，不是 task），

所以单线程情况下不需要额外同步。只有在显式 fork 多线程时才需要 semaphore。

Golden model 的核心挑战：

互斥锁（mutex/semaphore）只解决"同时写"的问题，不解决"写的顺序"问题。

Golden model 必须和 DUT 的操作顺序完全一致，否则状态会diverge，

导致误报（false positive）或漏报（false negative）。

正确解法：Request FIFO 序列化。

关键设计：Monitor 在 DUT 总线仲裁之后才把事务推入 FIFO，

因此 FIFO 的顺序天然和 DUT 处理顺序一致。

Golden model 的 run_phase 单线程消费 FIFO，是唯一写入 ref_mem 的线程，

完全不需要锁，通过架构设计消除了竞争条件。

面试关键词：post-arbitration observation（仲裁后采样）、

single consumer（单消费者）、FIFO order == DUT order。

## Q79. UVM: Scoreboard Design for 10-Input to 20-Output Port Mapping

QUESTION

A DUT has 10 input ports and 20 output ports. Each input packet
contains an address and a unique ID. The DUT routes each input
packet to one or more output ports based on the address (e.g.
via hashing).

(a) Design a UVM scoreboard to verify this DUT.

Start simple: assume one input port and two output ports.

(b) Scale up: support all 10 input ports and 20 output ports.

What data structure should the scoreboard use?

(c) How does the scoreboard know which output port a packet

SHOULD go to? How do you model the routing logic (cmodel)?

(d) Important: the interviewer described using analysis_port

to "get" transactions. What is wrong with this, and what

is the correct UVM component to use for blocking receive?

ANSWER

```
// (d) FIRST: The analysis_port mistake -- read this carefully
//
// uvm_analysis_port is a BROADCAST port: it calls write() on all
// connected subscribers. It is PUSH-based and non-blocking.
// You CANNOT call get() on a uvm_analysis_port -- it has no get().
//
// To RECEIVE transactions in a scoreboard with blocking get():
//   Use uvm_tlm_analysis_fifo #(T) -- this implements BOTH:
//     uvm_analysis_imp (write() side, monitor connects here)
//     uvm_blocking_get_port (get() side, scoreboard reads here)
//   OR use uvm_analysis_imp + a mailbox/queue inside the scoreboard.
//
// Correct pattern:
//   Monitor   -> analysis_port -> [analysis_fifo] -> scoreboard get()
//   Monitor calls vif.analysis_port.write(txn)
//   Scoreboard calls fifo.get(txn)    -- blocking, waits for data
//
// The scoreboard logic below is correct; replace any
// "analysis_port.get()" with "tlm_fifo.get()" in implementation.

// (a) Simple scoreboard: 1 input port, 2 output ports
class simple_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(simple_scoreboard)

  // TLM FIFOs -- correct way to receive from monitors (NOT analysis_port.get)
  uvm_tlm_analysis_fifo #(packet_txn) input_fifo;      // from input monitor
  uvm_tlm_analysis_fifo #(packet_txn) output_fifo0; // from output port 0 monitor
  uvm_tlm_analysis_fifo #(packet_txn) output_fifo1; // from output port 1 monitor

  // Reference model: maps unique_id -> expected output port
  int unsigned exp_port [int unsigned];     // uid -> expected port_id
  // Reference model: maps unique_id -> expected packet (for data check)
  packet_txn    exp_pkt   [int unsigned];   // uid -> full expected packet

  function void build_phase(uvm_phase phase);

 super.build_phase(phase);
 input_fifo     = new("input_fifo",    this);
 output_fifo0 = new("output_fifo0", this);
 output_fifo1 = new("output_fifo1", this);
endfunction

task run_phase(uvm_phase phase);
 fork
    process_inputs();
    check_output(output_fifo0, 0);
    check_output(output_fifo1, 1);
 join
```

```
endtask

// Receive input packets and compute expected output port via cmodel
task process_inputs();
 packet_txn txn;
 forever begin
    input_fifo.get(txn);                        // blocking get -- correct
    exp_port[txn.unique_id] = hash(txn.addr);// cmodel routing decision
    exp_pkt [txn.unique_id] = txn;
 end
endtask

// Check each output port's packets against expectations
task check_output(uvm_tlm_analysis_fifo #(packet_txn) fifo,
                  int unsigned port_id);
 packet_txn txn;
 forever begin
    fifo.get(txn);
    if (!exp_port.exists(txn.unique_id)) begin
       `uvm_error("SCBD", $sformatf(
          "Unexpected pkt uid=%0h on port %0d", txn.unique_id, port_id))
    end else begin
       // Check correct port

           if (exp_port[txn.unique_id] != port_id)
             `uvm_error("SCBD", $sformatf(
              "Wrong port: uid=%0h got port=%0d exp port=%0d",
              txn.unique_id, port_id, exp_port[txn.unique_id]))
           // Check data integrity
           if (txn.addr != exp_pkt[txn.unique_id].addr)
             `uvm_error("SCBD", $sformatf(
              "Data mismatch uid=%0h", txn.unique_id))
           // Clean up matched entry
           exp_port.delete(txn.unique_id);
           exp_pkt.delete(txn.unique_id);
       end
     end
  endtask

  // Cmodel routing function -- must match DUT's algorithm exactly
  function int unsigned hash(bit [31:0] addr);
     // Example: simple modulo hash to 20 output ports
     return addr % 20;
  endfunction

  function void check_phase(uvm_phase phase);
     if (exp_port.num() != 0)
       `uvm_error("SCBD", $sformatf(
           "%0d input packets never appeared on any output port",
           exp_port.num()))
  endfunction
endclass

// (b) Scale to 10 input ports + 20 output ports
// Use associative arrays keyed by port_id instead of fixed FIFOs.
//
class full_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(full_scoreboard)

localparam NUM_IN   = 10;
localparam NUM_OUT = 20;

// One TLM FIFO per port -- use associative array for flexibility
```

```
uvm_tlm_analysis_fifo #(packet_txn) in_fifo   [int unsigned];
uvm_tlm_analysis_fifo #(packet_txn) out_fifo [int unsigned];

// Reference storage: uid -> expected {port_id, packet}
int unsigned exp_port [int unsigned];
packet_txn    exp_pkt   [int unsigned];

function void build_phase(uvm_phase phase);
 super.build_phase(phase);
 for (int i = 0; i < NUM_IN;     i++)
    in_fifo[i]   = new($sformatf("in_fifo_%0d",   i), this);
 for (int i = 0; i < NUM_OUT; i++)
    out_fifo[i] = new($sformatf("out_fifo_%0d", i), this);
endfunction

task run_phase(uvm_phase phase);
 // Fork one thread per input port and one per output port
 for (int i = 0; i < NUM_IN; i++) begin
    automatic int port_i = i;
    fork process_input(port_i); join_none
 end
 for (int i = 0; i < NUM_OUT; i++) begin
    automatic int port_i = i;
    fork check_output_port(port_i); join_none
 end
 wait fork;
endtask

task automatic process_input(int unsigned port_id);
 packet_txn txn;
 forever begin

    in_fifo[port_id].get(txn);
    // Cmodel: compute which output port this packet should go to
    begin
       automatic int unsigned expected = hash(txn.addr);
       exp_port[txn.unique_id] = expected;
       exp_pkt [txn.unique_id] = txn;
    end
 end
endtask

task automatic check_output_port(int unsigned port_id);
 packet_txn txn;
 forever begin
    out_fifo[port_id].get(txn);
    if (!exp_port.exists(txn.unique_id)) begin
       `uvm_error("SCBD", $sformatf(
          "Unexpected uid=%0h on out_port=%0d", txn.unique_id, port_id))
    end else begin
       if (exp_port[txn.unique_id] != port_id)
          `uvm_error("SCBD", $sformatf(
            "Routing error uid=%0h: expected out_port=%0d got=%0d",
            txn.unique_id, exp_port[txn.unique_id], port_id))
       if (txn.addr != exp_pkt[txn.unique_id].addr)
          `uvm_error("SCBD", $sformatf(
            "Data error uid=%0h", txn.unique_id))
       exp_port.delete(txn.unique_id);
       exp_pkt.delete(txn.unique_id);
    end
 end
endtask
```

```
function int unsigned hash(bit [31:0] addr);
 return addr % NUM_OUT;     // must mirror DUT's routing algorithm
endfunction

     function void check_phase(uvm_phase phase);
      if (exp_port.num() != 0)
         `uvm_error("SCBD", $sformatf(
           "%0d packets sent but never received on any output",
           exp_port.num()))
     endfunction
 endclass

 // (c) Cmodel routing design
 //
 // Key principle: the scoreboard's hash() function must be an EXACT
 // software replica of the DUT's routing logic.
 //
 // Verification flow:
 //    Step 1: Input monitor captures packet {addr, unique_id}
 //    Step 2: Scoreboard calls hash(addr) -> expected output port
 //    Step 3: Output monitor on every port captures outgoing packets
 //    Step 4: Scoreboard matches uid, checks:
 //              (a) Did it arrive on the correct port? (routing check)
 //              (b) Is the data (addr, payload) intact? (data integrity)
 //              (c) Was uid seen exactly once? (no duplication/loss)
 //
 // If routing algorithm changes (e.g. ECMP instead of modulo),
 // only hash() needs to change -- scoreboard structure stays the same.
```

解释

这道题是 Amazon 的综合 UVM 设计题，考察从简单到复杂的系统化思路。

analysis_port 的坑（面试中的关键误区）：

uvm_analysis_port 是 PUSH 广播模式，只有 write() 接口，没有 get()。

面试官说"用 analysis_port 去 get"是错误的说法，但全程没有纠正，

说明面试官自己也没有意识到（或者他确实想说 tlm_fifo）。

正确做法：uvm_tlm_analysis_fifo 同时实现 write 端（供 monitor 调用）

和 get 端（供 scoreboard 阻塞等待），是 scoreboard 接收事务的标准组件。

数据结构选择：

一个 FIFOs 的联合数组（key=port_id）比固定数量的 FIFO 成员变量

更灵活，端口数量变化时只需改常量，不需要改代码结构。

Cmodel 设计核心：

hash(addr) 必须和 DUT 的路由算法完全一致，这是正确性的基础。

在 check_output_port 里，不仅要验证数据正确，还要验证到达的端口正确，

unique_id 作为"事务身份证"贯穿整个验证流程。

## Q80. UVM: Phase Mechanism -- What It Is and How It Works

QUESTION

(a) What is a UVM phase? What problem does the phase mechanism solve?

(b) Walk through the key UVM phases in order, explaining what

each one is responsible for and whether it is a function

or a task (and why).

(c) How does the phase mechanism coordinate between multiple

components in a testbench hierarchy?

ANSWER

```
// (a) What is a UVM phase?
//
// A UVM phase is a named, ordered stage of simulation lifecycle.
// Problem it solves: in a complex TB with many components (agent,
// scoreboard, coverage, driver, monitor...), each component needs
// to initialize, connect, run, and clean up in a coordinated order.
// Without phases, one component might try to use an interface or
// port that has not been constructed yet.
// Phases guarantee that ALL components complete one stage before
// ANY component moves to the next stage.

// (b) Key UVM phases in order:
//
// BUILD PHASES (top-down, function):
//   build_phase:    Construct child components and set config.
//                   Called top-down so parents create children first.
//                   Function (no time passing, no blocking).
//
//   connect_phase:  Connect TLM ports (analysis ports, FIFOs).
//                   Bottom-up: children connect before parents.
//                   Function (no time).
//
//   end_of_elaboration_phase: Final checks after all connections made.
//                   Print TB topology, verify configuration.
//                   Function.
//
//   start_of_simulation_phase: One-time setup before time advances.
//                   Print config, set initial state.
//                   Function.
//
// RUN PHASE (task -- time consuming):
//   run_phase:      The main simulation phase. ALL parallel activity
//                   happens here: driving stimulus, monitoring,
//                   checking. This is a TASK because it must be able
//                   to consume simulation time (#delays, @events).
//                   All components run their run_phase concurrently
//                   (forked by the UVM framework).
//
// CLEANUP PHASES (bottom-up, function):
//   extract_phase:  Extract results from scoreboards, coverage.
//   check_phase:    Final pass/fail checks. Report errors found.
//   report_phase:   Print final summary (pass/fail counts, coverage).
//   final_phase:    Last cleanup before $finish.
//
// Why function vs task?
//   Functions: no simulation time, no blocking -- safe for
//              construction/connection where ordering matters.
//   Task:      run_phase must be able to wait for events,
//              drive signals, and consume time -- must be a task.

// (c) Coordination between components:
//
// The UVM phase engine manages execution:
//   1. For function phases: calls each component's phase method
```

```
//      in the correct order (top-down or bottom-up).
//      No component proceeds until all are done.
//
//   2. For run_phase: forks ALL components' run_phase tasks
//      simultaneously. They run in parallel.
//
//   3. Objection mechanism (see next Q) controls when run_phase ends:
//      The phase ends when ALL objections are dropped.
//      If no component raises an objection, run_phase ends immediately.
//
// Example flow:
//   build_phase   (top->down): env creates agent, agent creates driver/monitor
//   connect_phase (bot->up):   monitor.ap connects to scoreboard.analysis_export
//   run_phase     (parallel):  driver drives, monitor samples, scoreboard checks
//   check_phase   (bot->up):   scoreboard reports mismatches
```

解释

UVM phase 机制解决的核心问题：多个组件的初始化和执行顺序协调。

最重要的区别：function phase 不消耗仿真时间（构建/连接），

run_phase 是 task，可以消耗时间（驱动信号、等待事件）。

记忆要点：

build 自顶向下（父先于子）、connect 自底向上（子先于父）、

run_phase 所有组件并行 fork、cleanup 自底向上。

面试最常问：为什么 build 是 top-down？

答：父组件必须先创建子组件，子组件才能存在并被连接。

## Q81. UVM: Objection Mechanism -- What It Is and How It Works

QUESTION

(a) What is the UVM objection mechanism? Why is it needed?

(b) How do raise_objection and drop_objection work?

(c) What happens if no component raises an objection?

(d) What is the most common bug related to objections?

ANSWER

```
// (a) What is the objection mechanism?
//
// The objection mechanism is UVM's way of controlling when
// run_phase ends. It implements a reference-counting system:
//   - A component "objects to the phase ending" by raising an objection.
//   - The phase engine waits until ALL objections are dropped.
//   - When the objection count reaches 0, run_phase ends and
//     the UVM framework proceeds to cleanup phases.
//
// Why is it needed?
//   run_phase runs ALL components' tasks in parallel.
//   Without a coordination mechanism, the phase would end immediately
//   (since there is no inherent end condition for parallel tasks).
//   Objections let components say "I'm not done yet, keep running."

// (b) How raise/drop work:
```

```
//
task run_phase(uvm_phase phase);
  // RAISE: increment objection counter by 1
  // Call as early as possible (first thing in run_phase)
  phase.raise_objection(this);
  //
  // ... do all simulation work here ...
  // drive transactions, wait for responses, check results
  //
  // DROP: decrement objection counter by 1
  // Call LAST, after ALL work is done
  phase.drop_objection(this);
  // NOTHING should execute after drop_objection -- not guaranteed to run
endtask
//
// Internal mechanics:
//   raise_objection increments a global counter in the phase.
//   drop_objection decrements it.
//   When counter hits 0, UVM starts a drain_time timer (default 0).
//   After drain_time, if still 0, run_phase ends for ALL components.
//
// Multiple components can all raise objections independently:
//   env.agent.driver raises   -> count = 1
//   env.test raises           -> count = 2
//   env.agent.driver drops    -> count = 1
//   env.test drops            -> count = 0 -> run_phase ends

// (c) If no component raises an objection:
//
// run_phase ends IMMEDIATELY (at time 0 of the run phase).
// The UVM framework sees count = 0 from the start, so it
// immediately proceeds to extract/check/report phases.
// No stimulus is driven, no checking occurs.
// This is a common mistake for new UVM users: forgetting to
// raise an objection in the test's run_phase.

// (d) Most common objection bugs:
//
// Bug 1: Code after drop_objection
//   phase.drop_objection(this);
//   #50;
//   $display("This may never execute!"); // WRONG
//   // drop_objection signals end of phase; anything after it
//   // may be killed before it executes.
//
// Bug 2: Forgetting to raise objection
//   task run_phase(uvm_phase phase);
//     // no raise_objection -> phase ends immediately!
//     vseq.start(env.vseqr);  // never gets to run
//   endtask
//
// Bug 3: Raising but never dropping (simulation hangs forever)
//   phase.raise_objection(this);
//   // ... exception path exits without dropping
//   // Simulation never ends -> timeout
//
// Best practice:
//   raise_objection as the FIRST thing in run_phase.
//   drop_objection as the LAST thing in run_phase.
//   Use try/catch or always blocks to guarantee drop runs.
```

解释

Objection 是 UVM 最重要的控制机制之一，面试必考。

核心类比：reference counting（引用计数）。

计数 > 0：仿真继续运行。计数 = 0：run_phase 结束。

三大常见错误：drop 之后还写代码（不保证执行）、

忘记 raise（仿真立即结束）、raise 但忘记 drop（仿真挂死）。

面试要点：能说出 raise_objection 和 drop_objection 的位置原则——

raise 尽量早，drop 必须最后。

## Q82. UVM: Scoreboard with Timeout and Duplicate Detection (A/B Pairing)

QUESTION

Design a UVM scoreboard that receives transactions from two analysis ports (Agent A and Agent B). The scoreboard must:

- When a matching A+B pair arrives (in any order), call check_pair()
- If two consecutive A transactions arrive before a B (or vice versa), report a "duplicate transaction" error
- If one transaction arrives but its partner does not arrive within a timeout period, report a "timeout" error
- Use process::get() and process.kill() to manage the timer thread

ANSWER

```
class scoreboard extends uvm_scoreboard;
  `uvm_component_utils(scoreboard)

  // Analysis imp ports -- one per agent
  `uvm_analysis_imp_decl(_A)
  `uvm_analysis_imp_decl(_B)
  uvm_analysis_imp_A #(my_trans, scoreboard) imp_A;
  uvm_analysis_imp_B #(my_trans, scoreboard) imp_B;

  my_trans trans_A;       // pending A transaction (waiting for B)
  my_trans trans_B;       // pending B transaction (waiting for A)
  process  p_a;           // timer process for A
  process  p_b;           // timer process for B

  int timeout_time = 1000; // timeout in simulation time units
  longint timer = 0;       // timestamp when pending trans arrived

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    imp_A = new("imp_A", this);
    imp_B = new("imp_B", this);
  endfunction

  // Called by Agent A monitor
  function void write_A(my_trans tr);
    // Duplicate check: A arrived but we already have a pending A
    if (trans_A != null) begin
      `uvm_error(get_type_name(), "Duplicate trans A received")
```

```
    return;
  end
  // Pair found: A arrived and B is already waiting
  if (trans_B != null) begin
    check_pair(tr, trans_B);
    return;
  end
  // No B yet: store A and start timeout timer
  trans_A = tr;
  fork
    begin
      p_a = process::get();   // capture handle to this thread
      timer = $stime();
      count_time();           // blocks until timeout or killed
    end
  join_none
endfunction

// Called by Agent B monitor
function void write_B(my_trans tr);
  if (trans_B != null) begin
    `uvm_error(get_type_name(), "Duplicate trans B received")
    return;
  end
  if (trans_A != null) begin
    check_pair(trans_A, tr);
    return;
  end
  trans_B = tr;
  fork
    begin
      p_b = process::get();
      timer = $stime();
      count_time();
    end
  join_none
endfunction

// Timer task: wait in 1-unit steps, report timeout if exceeded
task count_time();
  forever begin
    #1;
    if ($stime() - timer > timeout_time) begin
      `uvm_error(get_type_name(), "Timeout: partner transaction never arrived")
      break;
    end
  end
endtask

// Called when a valid A+B pair is found
function void check_pair(my_trans A, my_trans B);
  `uvm_info(get_type_name(), "Pair matched -- checking...", UVM_MEDIUM)
  // ... your comparison logic here ...

  // Reset state: clear pending transactions
  trans_A = null;
  trans_B = null;
  // Kill timer threads so they don't fire spurious timeout errors
  if (p_a != null) begin p_a.kill(); p_a = null; end
  if (p_b != null) begin p_b.kill(); p_b = null; end
  timer = 0;
endfunction
```

```
endclass

// Key design points:
//
// 1. `uvm_analysis_imp_decl(_A/_B) creates two separate write() methods:
//    write_A() and write_B(), allowing one scoreboard to receive from
//    two different agents without port name conflicts.
//
// 2. process::get() captures the handle to the currently running thread.
//    This must be called from INSIDE the forked thread to get its handle.
//    The handle is then stored in p_a / p_b for later kill().
//
// 3. process.kill() terminates the timer thread cleanly when a pair is
//    found, preventing false timeout errors after successful matching.
//
// 4. Timer sharing: both write_A and write_B update the same 'timer'
//    variable. This works because only one pending transaction exists
//    at a time (the other is null). For concurrent timers, use
//    separate timer_a and timer_b variables.
```

解释

这道题综合考察 UVM scoreboard 的进阶设计能力。

三个核心机制：

1. 双 analysis imp：用 `uvm_analysis_imp_decl 宏声明两个带后缀的 imp，

使同一个 scoreboard 可以接收来自两个不同 agent 的事务，

分别进入 write_A() 和 write_B()。

2. process::get() + kill()：在 fork 内部调用 process::get() 获取

当前线程句柄，配对成功时调用 kill() 终止计时器线程，

防止配对成功后还触发超时报错。

3. 状态机逻辑：trans_A/trans_B 充当状态标志，

null 表示"空闲等待"，非 null 表示"有待匹配的事务"。

check_pair() 调用后必须将两者都清空，否则后续事务会误判为重复。

## Q83. Python: Parse comma-separated names into a list

QUESTION

Write a Python/Perl script that reads a string of

comma-separated names and stores them in a list/array.

Example input: "Alice, Bob, Charlie, Diana"

ANSWER

```
# Python solution

  def parse_names(input_str):
      """
      Parse a comma-separated string of names into a list.
      Handles extra whitespace around names.
```

```
      Args:
            input_str (str): Comma-separated names, e.g. "Alice, Bob, Charlie"

      Returns:
            list[str]: List of stripped name strings.
      """
      names = [name.strip() for name in input_str.split(',')]
      return [name for name in names if name]          # filter empty strings

  # Usage
  input_str = "Alice, Bob, Charlie, Diana"
  result = parse_names(input_str)
  print(result)    # ['Alice', 'Bob', 'Charlie', 'Diana']

  # One-liner version:
  names = [n.strip() for n in "Alice, Bob, Charlie".split(',')]

  # Perl equivalent:
  # my $str = "Alice, Bob, Charlie, Diana";
  # my @names = map { s/^\s+|\s+$//gr } split(/,/, $str);
  # print join("\n", @names);
```

解释

Python 的 split(',') 以逗号分割字符串，返回列表。

strip() 去掉每个元素前后的空格，处理 "Alice, Bob" 这种带空格的情况。

list comprehension [name.strip() for name in ...] 是最 Pythonic 的写法，

额外过滤空字符串避免输入末尾有逗号时产生空元素。

## Algorithms & Scripting

### Q84. Python: Extract error lines using regex

QUESTION

Given a simulation log file in the following format, write a Python script using regex to extract all lines that contain errors. The script should capture the timestamp, severity, module name, and error message.

```
module name, and error message.


Sample log format:
    [100ns] [INFO]   [driver]       Transaction sent: addr=0x1234
    [150ns] [ERROR] [scoreboard] Data mismatch: exp=0xAB got=0xCD
    [200ns] [INFO]   [monitor]      Captured txn id=3
    [250ns] [ERROR] [checker]       Protocol violation: valid without ready
    [300ns] [WARN]   [coverage]     Coverage below threshold: 72%
    [350ns] [FATAL] [scoreboard] Null pointer dereference in compare()
    [400ns] [error] [driver]        Timeout waiting for ack signal
    [450ns] [INFO]   [monitor]      Simulation complete
```

```
Requirements:
(a) Extract all lines with severity ERROR, FATAL, or error
     (case-insensitive match on the severity field).
(b) For each matched line, parse and print:
     timestamp, severity, module, and message separately.
(c) Write the matched lines to an output file.
```

ANSWER

```
import re

 # Sample log content (for reference)
 SAMPLE_LOG = """[100ns] [INFO]       [driver]     Transaction sent: addr=0x1234
 [150ns] [ERROR] [scoreboard] Data mismatch: exp=0xAB got=0xCD
 [200ns] [INFO]     [monitor]      Captured txn id=3
 [250ns] [ERROR] [checker]         Protocol violation: valid without ready
 [300ns] [WARN]     [coverage]     Coverage below threshold: 72%
 [350ns] [FATAL] [scoreboard] Null pointer dereference in compare()
 [400ns] [error] [driver]          Timeout waiting for ack signal
 [450ns] [INFO]     [monitor]      Simulation complete"""

 # Regex pattern to match a full log line and capture groups:
 # Group 1: timestamp     e.g. 150ns
 # Group 2: severity      e.g. ERROR
 # Group 3: module        e.g. scoreboard
 # Group 4: message       e.g. Data mismatch: exp=0xAB got=0xCD
 LOG_PATTERN = re.compile(
      r'\[(\w+)\]'          # [timestamp]
      r'\s+\[(\w+)\]'      # [severity]
      r'\s+\[(\w+)\]'      # [module]
      r'\s+(.+)',               # message (rest of line)
      re.IGNORECASE
 )

 # Pattern to identify error-level lines (ERROR, FATAL, error, etc.)
 ERROR_SEVERITY = re.compile(r'^(error|fatal)$', re.IGNORECASE)

 def extract_errors(log_text):
      """
      Parse log lines and extract all ERROR/FATAL entries.

      Args:
            log_text (str): Full log content as a string.

   Returns:
         list[dict]: Each dict has keys: timestamp, severity,
                     module, message.
   """
   results = []
   for line in log_text.strip().split('\n'):
         m = LOG_PATTERN.search(line)
         if m:
            timestamp, severity, module, message = m.groups()
            if ERROR_SEVERITY.match(severity):
                 results.append({
                      'timestamp': timestamp,
                      'severity' : severity.upper(),
                      'module'    : module,
                      'message'   : message.strip(),
                 })
   return results
```

```
def extract_errors_from_file(input_path, output_path=None):
   """
   Read a log file, extract error lines, optionally write to output.

   Args:
         input_path   (str): Path to input log file.
         output_path (str): Path to output file (optional).
   """
   error_pattern = re.compile(
         r'\[(\w+)\]\s+\[(error|fatal)\]\s+\[(\w+)\]\s+(.+)',
         re.IGNORECASE
   )
   matched_lines = []

   with open(input_path, 'r') as f:
         for line in f:
            if error_pattern.search(line):

                  matched_lines.append(line.rstrip('\n'))

      # Print results
      for line in matched_lines:
           print(line)

      # Write to output file if specified
      if output_path and matched_lines:
           with open(output_path, 'w') as out:
              out.write('\n'.join(matched_lines) + '\n')
           print(f"Written {len(matched_lines)} error lines to {output_path}")

      return matched_lines

  # Demo with sample log
  errors = extract_errors(SAMPLE_LOG)
  for e in errors:
      print(f"[{e['timestamp']}] [{e['severity']}] "
             f"[{e['module']}] {e['message']}")

  # Expected output:
  # [150ns] [ERROR] [scoreboard] Data mismatch: exp=0xAB got=0xCD
  # [250ns] [ERROR] [checker]      Protocol violation: valid without ready
  # [350ns] [FATAL] [scoreboard] Null pointer dereference in compare()
  # [400ns] [ERROR] [driver]       Timeout waiting for ack signal
```

解释

这道题考察实际工程中最常见的日志解析场景。

正则表达式分组 (\w+) 捕获各字段：

\[ 和 \] 匹配方括号（需要转义），\w+ 匹配字母数字下划线，

\s+ 匹配一个或多个空白字符（处理对齐空格），

.+ 匹配消息内容（任意字符到行尾）。

两个 pattern 分工：LOG_PATTERN 解析结构，ERROR_SEVERITY 过滤级别，

这样比把所有逻辑塞进一个超长 regex 更易读易维护。

re.IGNORECASE 保证 error/ERROR/Error 都能被捕获。

面试加分项：提到用 named groups 让代码更清晰：

r'\[(?P<timestamp>\w+)\]\s+\[(?P<severity>\w+)\]...'

然后用 m.group('timestamp') 而不是 m.group(1)。

## Q85. C: Check if the N-th bit is 1

QUESTION

Write a C function that checks whether the bit at position

'bit_pos' (0-indexed from LSB) in integer 'n' is set (1).

ANSWER

```
#include <stdbool.h>
  #include <stdio.h>

  bool is_bit_set(int n, int bit_pos) {
      /*
        * Check if bit at position bit_pos (0 = LSB) is 1.
        * Shift n right by bit_pos, then mask with 1.
        * Returns true if the bit is set, false otherwise.
        */
      return (n >> bit_pos) & 1;
  }

  /* Alternative: using bit mask */
  bool is_bit_set_v2(int n, int bit_pos) {
      return (n & (1 << bit_pos)) != 0;
  }

  int main() {
      int val = 0b10110100;      /* binary: bit 2, 4, 5, 7 are set */
      printf("bit 2: %d\n", is_bit_set(val, 2));      /* 1 */
      printf("bit 3: %d\n", is_bit_set(val, 3));      /* 0 */
      printf("bit 7: %d\n", is_bit_set(val, 7));      /* 1 */
      return 0;
  }
```

解释

(n >> bit_pos) & 1 是最简洁的写法：

先把目标 bit 右移到第0位，再与1做 AND，结果就是该 bit 的值（0或1）。

另一种写法 (n & (1 << bit_pos)) != 0 是生成一个只有目标位为1的 mask，

与 n 做 AND，非零则该位为1。

注意：对有符号 int 做右移时，算术右移会填充符号位，

但这里只关心某一个 bit，操作结果相同。

## Q86. Algorithm: Find the Element That Appears Once (XOR Trick)

QUESTION

Given an integer array where every element appears TWICE except

```
for exactly ONE element which appears once, find that element.
Requirement: O(n) time, O(1) extra space.

Example: arr = [4, 1, 2, 1, 2] -> output: 4
arr = [2, 2, 1] -> output: 1
```

ANSWER

```
// Key XOR properties:
  //    a XOR a == 0     (same value cancels)
  //    a XOR 0 == a     (identity)
  //    commutative and associative
  //
  // XOR all elements: pairs cancel to 0; unique element survives.
  // Example: 4^1^2^1^2 = 4^(1^1)^(2^2) = 4^0^0 = 4

  // Python version
  def find_unique(arr):
       result = 0
       for num in arr:
           result ^= num
       return result

  # Test
  # find_unique([4, 1, 2, 1, 2]) -> 4
  # find_unique([2, 2, 1])            -> 1

  // SystemVerilog version
  function automatic int find_unique_sv(int arr[]);
     int result = 0;
     foreach (arr[i])
       result ^= arr[i];
     return result;
  endfunction

  // Follow-up: one element appears 3x, rest appear 2x?
  // XOR fails (a^a^a = a, not 0).
  // Solution: count set bits at each bit position, take mod 3.
  // Remaining 1s at each position give the unique element.
```

解释

XOR 找唯一元素是 O(n)/O(1) 的经典算法。

核心：a^a=0，所有出现两次的元素两两抵消，剩下的就是唯一元素。

面试延伸：如果唯一元素出现 1 次而其他出现 3 次，XOR 不适用，

需要用按位计数 mod 3 的方法，每个 bit 位独立统计。

## Q87. C++ Algorithm: Count Pairs with Equal Digit Sum

QUESTION

Given an array of integers where each element is in range [10, 99]

(i.e. all two-digit numbers), return the count of pairs (i, j)

where i < j and the digit sum of nums[i] equals the digit sum of nums[j].

Example:

nums = [18, 27, 45, 36, 90]

digit sums: [9, 9, 9, 9, 9]

All 5 elements have digit sum 9 -> C(5,2) = 10 pairs

Output: 10

nums = [14, 21, 13, 31]

digit sums: [5, 3, 4, 4]

Only 13 and 31 share digit sum 4 -> 1 pair

Output: 1

Constraint: 10 <= nums[i] <= 99, 1 <= n <= 10^5

Goal: O(n) time, O(1) extra space (digit sum only ranges [2,18]).

ANSWER

```
// Key insight: digit sum of a 2-digit number ab = a + b.
 // For nums[i] in [10,99]: digit_sum = nums[i]/10 + nums[i]%10
 // Range of digit sums: [1+0, 9+9] = [1, 18]
 // So we can use a fixed-size count array of size 19.
 //
 // Algorithm:
 // 1. Count frequency of each digit sum.
 // 2. For each digit sum s with frequency f[s], the number of valid
 //       pairs is C(f[s], 2) = f[s] * (f[s]-1) / 2.
 // 3. Sum over all digit sums.
 // Time: O(n), Space: O(1) (count array has fixed size 19)

 #include <vector>
 #include <numeric>
 using namespace std;

 int countEqualDigitSumPairs(vector<int>& nums) {
      int freq[19] = {};   // freq[s] = count of numbers with digit sum s

      for (int x : nums) {
           int s = x / 10 + x % 10;    // digit sum of two-digit number
           freq[s]++;
      }

      int count = 0;
      for (int f : freq) {
           count += f * (f - 1) / 2;    // C(f, 2) pairs
      }
      return count;
 }

 // Test cases
 // nums = {18,27,45,36,90}: all digit sum 9, C(5,2)=10 -> output 10
 // nums = {14,21,13,31}:       digit sums {5,3,4,4}, only 13+31 -> output 1
 // nums = {10,20,30}:          digit sums {1,2,3}, all different -> output 0

 // nums = {19,28,37,46}:          all digit sum 10, C(4,2)=6 -> output 6

 // Generalization for arbitrary integers (follow-up):
 // If range is not [10,99], digit sum can be larger.
 // Use unordered_map<int,int> instead of fixed array.
 // Still O(n) time, O(distinct_digit_sums) space.
 int countPairsGeneral(vector<int>& nums) {
      unordered_map<int, int> freq;
      for (int x : nums) {
           int s = 0, tmp = abs(x);
           while (tmp) { s += tmp % 10; tmp /= 10; }
           freq[s]++;
      }
      int count = 0;
      for (auto& [s, f] : freq)
```

```
            count += f * (f - 1) / 2;
        return count;
    }
```

解释

这道题的核心技巧是把问题转化为"计算频率数组中每个桶的组合数"。

两位数的数字之和范围是固定的 [1,18]，所以可以用大小为 19 的数组代替哈希表，

实现真正的 O(1) 空间复杂度。

C(f,2) = f*(f-1)/2 是计算同一组内两两配对数的标准公式。

面试进阶：如果数字范围不限，改用 unordered_map，时间仍然 O(n)，

空间变为 O(不同数字和的个数)。

## Q88. C++ Design: Container with ADD, EXISTS, GET_COUNT

QUESTION

Design a container class that supports the following operations:

- ADD(val): Add value val to the container.

- EXISTS(val): Return true if val exists, false otherwise.

- GET_COUNT(val): Return how many times val has been added.

Example:

ADD(3); ADD(3); ADD(5);

EXISTS(3) -> true

EXISTS(7) -> false

GET_COUNT(3) -> 2

GET_COUNT(5) -> 1

GET_COUNT(7) -> 0

Follow-up: support REMOVE(val) that removes ONE occurrence of val.

ANSWER

```
#include <unordered_map>
 using namespace std;

 class ValueContainer {
 private:
      unordered_map<int, int> freq;    // value -> count

 public:
      // Add val to container. O(1) average.
      void add(int val) {
          freq[val]++;
      }

      // Return true if val exists (count >= 1). O(1) average.
      bool exists(int val) const {
          auto it = freq.find(val);
          return it != freq.end() && it->second > 0;
      }
```

```
    // Return how many times val has been added. O(1) average.
    int getCount(int val) const {
        auto it = freq.find(val);
        return (it != freq.end()) ? it->second : 0;
    }

    // Follow-up: remove ONE occurrence. O(1) average.
    // Returns true if removal succeeded, false if val not present.
    bool remove(int val) {
        auto it = freq.find(val);
        if (it == freq.end() || it->second == 0)
            return false;
        if (--(it->second) == 0)
            freq.erase(it);   // clean up zero-count entries
        return true;
    }

};

// Usage example:
// ValueContainer c;
// c.add(3); c.add(3); c.add(5);
// c.exists(3)      -> true
// c.exists(7)      -> false
// c.getCount(3)    -> 2
// c.getCount(5)    -> 1
// c.getCount(7)    -> 0
// c.remove(3);
// c.getCount(3)    -> 1
// c.remove(3);
// c.exists(3)      -> false

// Complexity:
// All operations: O(1) average, O(n) worst case (hash collision)
// Space: O(distinct values added)

// Thread-safe version (follow-up):
// Add mutex m; wrap each operation with lock_guard<mutex> lk(m);
```

解释

这道题考察数据结构设计能力，核心是选择合适的底层结构。

unordered_map<int,int> 是最直接的选择：key 是值，value 是出现次数。

EXISTS 和 GET_COUNT 都是 O(1) 的查找，ADD 是 O(1) 的插入/更新。

REMOVE 的细节：要处理 count 减到 0 后是否清理 map entry，

否则 map 会随时间累积大量零计数条目，浪费内存。

面试加分：主动提到线程安全（加 mutex 或用 concurrent_unordered_map）。

## Q89. C++ Algorithm: Palindrome Vowel Removal

QUESTION

Given an array of strings, for each string that is a palindrome,

replace all vowels (a, e, i, o, u, case-insensitive) with empty

string (i.e. remove them). Leave non-palindromes unchanged.

Return the modified array.

Example:

Input: ["racecar", "hello", "level", "world", "madam"]

Output: ["rccr", "hello", "lvl", "world", "mm"]

Note: "racecar" is palindrome -> remove a,e -> "rccr"

"level" is palindrome -> remove e -> "lvl"

"madam" is palindrome -> remove a -> "mm"

"hello", "world" are not palindromes -> unchanged

Case-insensitive palindrome check (but preserve original case in output).

ANSWER

```
#include <vector>
#include <string>
#include <algorithm>
#include <cctype>
using namespace std;

bool isPalindrome(const string& s) {
    int l = 0, r = (int)s.size() - 1;
    while (l < r) {
        if (tolower(s[l]) != tolower(s[r]))
            return false;
        l++; r--;
    }
    return true;
}

string removeVowels(const string& s) {
    // Vowels set for O(1) lookup
    const string vowels = "aeiouAEIOU";
    string result;
    result.reserve(s.size());
    for (char c : s) {
        if (vowels.find(c) == string::npos)
            result += c;
    }
    return result;
}

vector<string> processStrings(vector<string>& words) {
    vector<string> result;
    result.reserve(words.size());
    for (const string& w : words) {
        if (isPalindrome(w))
            result.push_back(removeVowels(w));
        else

                result.push_back(w);
       }
       return result;
  }

  // Optimized vowel check using unordered_set:
  #include <unordered_set>
  string removeVowelsFast(const string& s) {
       static const unordered_set<char> v =
            {'a','e','i','o','u','A','E','I','O','U'};
       string r;
```

```
        r.reserve(s.size());
        for (char c : s)
            if (!v.count(c)) r += c;
        return r;
    }

    // Complexity:
    // isPalindrome: O(n) where n = string length
    // removeVowels: O(n)
    // processStrings: O(total characters across all strings)
    // Overall: O(sum of all string lengths)

    // Edge cases:
    // Empty string "" -> isPalindrome returns true (empty is palindrome)
    //     -> removeVowels("") = "" (no vowels to remove)
    // Single char "a" -> palindrome, removeVowels -> "" (empty)
    // All vowels "aeiou" -> palindrome? "aeiou" reversed is "uoiea" != "aeiou"
    //     -> NOT palindrome -> unchanged
```

解释

这道题综合考察字符串处理的多个基础操作。

关键点 1：大小写不敏感的回文检测——用 tolower 统一比较，但输出保留原始大小写。

关键点 2：元音删除——用 unordered_set 做 O(1) 查找比 string::find 更快。

边界情况：空字符串和单字符字符串都是回文（面试常考边界）。

"aeiou" 不是回文，注意不要想当然认为全元音就是回文。

## Q90. C++ Algorithm: Array Pair Product Query with Point Update

QUESTION

Given two arrays a[] and b[], and a list of operations:

- [0, idx, val]: update a[idx] = val

- [1, target]: count pairs (i, j) where a[i] * b[j] == target

Return results for all type-1 (query) operations.

Example:

a = [2, 3, 6], b = [1, 2, 3]

ops = [[1,6], [0,1,4], [1,6], [1,12]]

Query 1 (target=6): pairs: (2,3),(6,1) -> a[0]*b[2]=6, a[2]*b[0]=6 -> count=3

also a[1]*b[1]=3*2=6 -> count=3

Update: a[1]=4, a=[2,4,6]

Query 2 (target=6): a[0]*b[2]=6, a[2]*b[0]=6 -> count=2 (4*? gives no 6)

Query 3 (target=12): a[1]*b[2]=4*3=12, a[2]*b[1]=6*2=12 -> count=2

Output: [3, 2, 2]

Constraints: n,m <= 10^4, q <= 10^5, values <= 10^6.

Optimize for fast queries after updates.

ANSWER

```cpp
#include <vector>
  #include <unordered_map>
  using namespace std;

  // --- Approach 1: Naive O(n*m) per query, O(1) update ---
  // For each query, scan all pairs. Simple but slow for large inputs.
  vector<int> solveNaive(
      vector<int>& a, vector<int>& b,
      vector<vector<int>>& ops)
  {
      vector<int> res;
      int n = a.size(), m = b.size();
      for (auto& op : ops) {
           if (op[0] == 0) {
                a[op[1]] = op[2];             // update a[idx] = val
           } else {
                long long target = op[1];
                int cnt = 0;
                for (int i = 0; i < n; i++) {
                     if (a[i] == 0) {
                         if (target == 0) cnt += m;
                         continue;
                     }
                     if (target % a[i] == 0) {
                         long long need = target / a[i];
                         // count how many b[j] == need
                         for (int j = 0; j < m; j++)
                             if (b[j] == need) cnt++;
                     }
                }
                res.push_back(cnt);
           }
      }
      return res;
  }

// --- Approach 2: Precompute freq_b, O(n) per query, O(1) update ---
// Precompute frequency map of b (b never changes).
// For each query(target): for each a[i], check if target%a[i]==0,
// then look up freq_b[target/a[i]] in O(1).
// Query: O(n), Update: O(1), Total: O(n*q + m)
vector<int> solveFast(
    vector<int>& a, vector<int>& b,
    vector<vector<int>>& ops)
{
    // Precompute frequency of elements in b (b is static)
    unordered_map<long long, int> freq_b;
    for (int x : b) freq_b[x]++;

    vector<int> res;
    int n = a.size();

    for (auto& op : ops) {
       if (op[0] == 0) {
           a[op[1]] = op[2];            // update: O(1)
       } else {
           long long target = op[1];
           int cnt = 0;
           for (int i = 0; i < n; i++) {       // O(n) per query
               if (a[i] == 0) {
                    // a[i]*b[j]=0=target only if target==0
                    if (target == 0) cnt += (int)b.size();
                    continue;
```

```
               }
               if (target % a[i] == 0) {
                    long long need = target / a[i];
                    auto it = freq_b.find(need);
                    if (it != freq_b.end())
                         cnt += it->second;
               }

             }
             res.push_back(cnt);
         }
     }
     return res;
}

// --- Approach 3: Further optimize with freq_a, O(D(target)) per query ---
// Maintain freq_a (frequency of elements in a, updated on each update op).
// For each query(target): enumerate divisors d of target,
// for each divisor d: if freq_a[d]>0 and freq_b[target/d]>0,
//    cnt += freq_a[d] * freq_b[target/d].
// Query: O(sqrt(target)) divisor enumeration, Update: O(1).
// Best when target is small but n and q are large.
vector<int> solveDivisor(
     vector<int>& a, vector<int>& b,
     vector<vector<int>>& ops)
{
     unordered_map<long long, int> freq_a, freq_b;
     for (int x : a) freq_a[x]++;
     for (int x : b) freq_b[x]++;

     vector<int> res;
     for (auto& op : ops) {
         if (op[0] == 0) {
             int idx = op[1], val = op[2];
             if (--freq_a[a[idx]] == 0) freq_a.erase(a[idx]);
             freq_a[val]++;
             a[idx] = val;               // update: O(1)
         } else {
             long long target = op[1];
             int cnt = 0;
             // enumerate divisors of target in O(sqrt(target))
             for (long long d = 1; d * d <= target; d++) {
                   if (target % d != 0) continue;

                   // pair (d, target/d)
                   auto ia = freq_a.find(d);
                   auto ib = freq_b.find(target / d);
                   if (ia != freq_a.end() && ib != freq_b.end())
                       cnt += ia->second * ib->second;
                   // pair (target/d, d) if different
                   if (d != target / d) {
                       auto ia2 = freq_a.find(target / d);
                       auto ib2 = freq_b.find(d);
                       if (ia2 != freq_a.end() && ib2 != freq_b.end())
                           cnt += ia2->second * ib2->second;
                   }
               }
               res.push_back(cnt);
           }
     }
     return res;
 }
```

```
// Complexity summary:
// Approach 1: O(n*m) per query -- simplest, TLE for large input
// Approach 2: O(n) per query, O(1) update -- good when n is small
// Approach 3: O(sqrt(target)) per query -- best when target is bounded
```

解释

这道题是典型的"在线更新+查询"设计题，面试考察的是选择合适的权衡方案。

三种方法的核心思路：

方法1（暴力）：O(n*m) 查询，适合小数据，面试中先说这个展示思路。

方法2（预计算 freq_b）：b 不变所以预处理 b 的频率表，

查询时只遍历 a，对每个 a[i] 在 O(1) 内查 b 中有多少配对，O(n) 查询。

方法3（枚举因子）：同时维护 freq_a 和 freq_b，查询时枚举 target 的因子，

O(sqrt(target)) 查询，适合 n,m 大但 target 有界的情况。

面试技巧：先说暴力，再分析 b 不变的特性提出方法2，最后如果追问才说方法3。

注意零值的特判（a[i]=0 时除法无效）。

## Q91. C++: Reverse a Linked List

QUESTION

Implement a function to reverse a singly linked list in C++.

(a) Iterative solution.

(b) Recursive solution.

(c) What is the time and space complexity of each?

(d) Follow-up: reverse only nodes from position m to n (1-indexed).

ANSWER

```
#include <iostream>
using namespace std;

struct ListNode {
     int val;
     ListNode* next;
     ListNode(int x) : val(x), next(nullptr) {}
};

// (a) Iterative -- O(n) time, O(1) space
ListNode* reverseList_iter(ListNode* head) {
     ListNode* prev = nullptr;
     ListNode* curr = head;
     while (curr != nullptr) {
         ListNode* next_node = curr->next;     // save next
         curr->next = prev;                     // reverse pointer
         prev = curr;                           // advance prev
         curr = next_node;                      // advance curr
     }
     return prev;   // prev is new head
}

// Step-by-step trace for [1->2->3->4->5]:
// Start:   prev=null, curr=1
```

```
// Step1:   1->null,    prev=1, curr=2
// Step2:   2->1,       prev=2, curr=3
// Step3:   3->2,       prev=3, curr=4
// Step4:   4->3,       prev=4, curr=5
// Step5:   5->4,       prev=5, curr=null    -> return 5
// Result: 5->4->3->2->1->null

// (b) Recursive -- O(n) time, O(n) space (call stack)
ListNode* reverseList_rec(ListNode* head) {
     // Base case: empty list or single node
     if (head == nullptr || head->next == nullptr)

          return head;
      // Recursively reverse the rest
      ListNode* new_head = reverseList_rec(head->next);
      // Make head->next point back to head
      head->next->next = head;
      head->next = nullptr;   // break original forward link
      return new_head;
 }

 // (c) Complexity:
 // Iterative: Time O(n), Space O(1)      -- preferred in production
 // Recursive: Time O(n), Space O(n)      -- elegant but stack overflow
 //             risk for very long lists (n > ~10k on typical systems)

 // (d) Reverse nodes from position m to n (1-indexed)
 // Example: [1->2->3->4->5], m=2, n=4      =>   [1->4->3->2->5]
 ListNode* reverseBetween(ListNode* head, int m, int n) {
      if (!head || m == n) return head;

      // Dummy node simplifies edge case where m==1
      ListNode dummy(0);
      dummy.next = head;
      ListNode* pre = &dummy;

      // Move pre to node just before position m
      for (int i = 1; i < m; i++)
           pre = pre->next;

      ListNode* curr = pre->next;   // node at position m
      // Perform (n-m) reversals using head-insertion
      for (int i = 0; i < n - m; i++) {
           ListNode* next_node = curr->next;
           curr->next = next_node->next;
           next_node->next = pre->next;
           pre->next = next_node;

      }
      return dummy.next;
 }
```

解释

反转链表是最经典的链表题，面试必须烂熟于心。

迭代解法的核心：三指针（prev、curr、next_node），

每次把 curr->next 从指向后继改为指向前驱，然后三指针同时向前移动。

递归解法的核心：先递归到尾部，回溯时让后继节点指向当前节点，

再把当前节点的 next 置 null 断开原链接。

区间反转（follow-up）：用 dummy 节点避免 m=1 的边界处理，

头插法：每次把 curr 后面的节点摘下来插到 pre 后面，

重复 (n-m) 次即可完成区间内的反转。

## Q92. C: memcpy Safety Issues and Best Practices

QUESTION

In C, what are the key safety issues and pitfalls when using memcpy()? Cover:

(a) Overlapping source and destination buffers.

(b) Null pointer and invalid pointer issues.

(c) Size miscalculation pitfalls.

(d) Alignment and hardware-specific concerns.

(e) What should you use instead in each problematic case?

ANSWER

```
#include <string.h>      // memcpy, memmove
  #include <stdint.h>
  #include <assert.h>

  // memcpy signature:
  // void* memcpy(void* dest, const void* src, size_t n);
  // Copies n bytes from src to dest. Returns dest.

  // (a) OVERLAPPING MEMORY -- most common and dangerous issue
  //
  // memcpy() has UNDEFINED BEHAVIOR when src and dest overlap.
  // The C standard explicitly states memcpy is for non-overlapping regions.
  //
  // Example of silent data corruption:
  char buf[] = "ABCDEFGH";
  // Shift left by 2: copy buf[2..7] to buf[0..5]
  memcpy(buf, buf + 2, 6);       // UNDEFINED BEHAVIOR: regions overlap!
  // May produce "ABABCDGH" or "CDEFGHGH" depending on implementation

  // CORRECT: use memmove() for overlapping regions
  // memmove() guarantees correct behavior even when src and dest overlap.
  // It detects overlap direction and copies accordingly
  // (forward or backward) to avoid overwriting data before it is copied.
  memmove(buf, buf + 2, 6);          // SAFE: "CDEFGHGH" -> "CDEFGH\0\0"

  // Rule: if you are not 100% certain regions do not overlap, use memmove.
  // memmove is slightly slower than memcpy but safe in all cases.

  // (b) NULL POINTER AND INVALID POINTER
  //
  // memcpy(NULL, src, n) and memcpy(dest, NULL, n) are both
  // undefined behavior, even if n == 0 (in strict C standard).
  // Always validate pointers before calling memcpy.
  void safe_memcpy(void* dest, const void* src, size_t n) {
       assert(dest != NULL && src != NULL);

     assert(n > 0);
     memcpy(dest, src, n);
}
```

```
// Also watch for:
//    - Dangling pointers (pointer to freed memory)
//    - Pointers to stack-allocated memory that went out of scope
//    - Hardware register addresses that must not be bulk-copied

// (c) SIZE MISCALCULATION PITFALLS
//
// Pitfall 1: using sizeof(pointer) instead of sizeof(object)
int* arr = malloc(10 * sizeof(int));
int buf2[10];
memcpy(buf2, arr, sizeof(arr));     // WRONG: copies sizeof(int*) = 8 bytes
memcpy(buf2, arr, sizeof(buf2));    // CORRECT: copies 10*4 = 40 bytes
memcpy(buf2, arr, 10 * sizeof(int)); // CORRECT: explicit element count

// Pitfall 2: string copy -- memcpy does NOT add null terminator
char src_str[] = "hello";
char dst_str[10];
memcpy(dst_str, src_str, 5);      // copies 'h','e','l','l','o' -- no '\0'!
memcpy(dst_str, src_str, 6);      // correct: include the null terminator
// Or: use strcpy / strncpy for strings

// Pitfall 3: integer overflow in size calculation
// memcpy(dest, src, count * element_size) -- if count is large,
// count * element_size can overflow size_t -> copies wrong amount
size_t count = 1000000000UL;
size_t elem   = 8;
// count * elem = 8e9 which fits in size_t (64-bit) but may exceed
// actual buffer size -> buffer overflow
// Always bounds-check before computing the size argument.

// (d) ALIGNMENT AND HARDWARE CONCERNS

  //
  // On most modern CPUs (x86, ARM with NEON), memcpy handles misaligned
  // addresses correctly but with a performance penalty.
  //
  // On some embedded/DSP targets:
  //   - Misaligned access causes a hardware exception (bus fault)
  //   - memcpy to/from memory-mapped I/O registers can trigger
  //       side effects (e.g. reading a FIFO register consumes the data)
  //   - DMA register writes require specific access width (32-bit only)
  //       -- memcpy may use byte or 16-bit accesses internally -> WRONG
  //
  // Embedded rule: never use memcpy on MMIO register arrays.
  // Use explicit volatile pointer reads/writes or register-specific macros.
  //
  // volatile uint32_t* reg = (volatile uint32_t*)0x40001000;
  // uint32_t val = *reg;              // correct: single 32-bit read
  // memcpy(&val, (void*)reg, 4); // DANGEROUS: may split into byte reads

  // (e) Summary -- what to use:
  //
  // +----------------------------+--------------------------------+
  // | Situation                  | Use instead of memcpy          |
  // +----------------------------+--------------------------------+
  // | Overlapping buffers        | memmove()                      |
  // | String copy                | strncpy() / strlcpy()          |
  // | Struct copy                | direct assignment (s1 = s2)    |
  // | MMIO registers             | volatile pointer read/write    |
  // | Security-sensitive buffers | memcpy_s() (C11 Annex K)       |
  // | Known non-overlapping      | memcpy() -- fastest option     |
  // +----------------------------+--------------------------------+
```

解释

memcpy 的安全问题在嵌入式和系统编程中非常重要，面试常考。

最高频考点：

1. 重叠问题：memcpy 重叠区域是未定义行为，必须用 memmove。

memmove 内部检测重叠方向，选择从前往后或从后往前复制。

2. 大小计算：sizeof(指针) 不等于 sizeof(数组)，这是最常见的 bug。

3. MMIO 寄存器：memcpy 不保证访问宽度，对寄存器操作是危险的。

在验证/芯片设计上下文中，MMIO 这一点尤其重要：

用 memcpy 读取 FIFO 状态寄存器可能因为多次字节读取

而意外多次触发 FIFO pop 操作，造成数据丢失。

## Q93. Test Plan: SoC with Encryption/Decryption Module

QUESTION

You are given a SoC containing an encryption/decryption module.

(a) Describe your overall test plan for this SoC.

(b) For different failure scenarios, identify which module is

likely at fault and what might have gone wrong.

(c) The encryption module is a black box - how do you verify it?

(d) If you have white-box access, what additional checks can you do?

ANSWER

```
// (a) Overall Test Plan:
 //
 // Level 1 - Module-level (block-level simulation):
 //    - Encrypt known plaintext, verify ciphertext against golden reference
 //    - Decrypt known ciphertext, verify recovered plaintext
 //    - Boundary: empty input, max-length input, single-byte input
 //    - Key variations: all-zero key, all-one key, random keys
 //    - Error injection: invalid key length, malformed input
 //
 // Level 2 - Integration:
 //    - End-to-end: encrypt -> transmit -> decrypt -> compare original
 //    - Back-to-back transactions with no gap
 //    - Concurrent encrypt + decrypt operations
 //    - DMA transfer of data through crypto engine
 //    - Interrupt handling during crypto operation
 //
 // Level 3 - System-level:
 //    - Boot sequence with secure boot using crypto module
 //    - Power state transitions (active -> sleep -> wake) during crypto
 //    - Reset during encryption mid-operation
 //    - Performance: measure throughput (MB/s), latency per block
 //
 // Coverage goals:
 //    - All key sizes (128/192/256-bit if AES)
 //    - All modes (ECB/CBC/CTR/GCM if applicable)
 //    - All interface signals toggle coverage
 //    - FSM state coverage for crypto engine control
```

```
// (b) Failure scenario -> likely cause:
//    Encrypted output always same regardless of input
//      -> Key register not being loaded correctly
//    Decrypted output != original plaintext
//      -> Key mismatch between encrypt/decrypt, or mode mismatch
//    Crypto operation hangs (done signal never asserts)
//      -> FSM stuck, clock/reset issue, or interface handshake bug

 //    Data corruption only on certain lengths
 //       -> Padding logic bug, block boundary handling error
 //    Intermittent failures under load
 //       -> DMA arbitration issue, FIFO overflow, timing closure

 // (c) Black-box verification:
 //    - Use known-answer tests (KAT): standard test vectors from
 //       NIST (for AES, SHA, etc.) with published plaintext/key/ciphertext
 //    - Encrypt with module, compare output bit-for-bit with golden
 //    - Verify interface protocol compliance (valid/ready handshake,
 //       correct timing of done/interrupt signals)
 //    - Stress test: random data, random keys, measure correctness rate
 //    - Negative testing: invalid inputs, unexpected reset

 // (d) White-box verification (internal access):
 //    - Internal state checking: verify round keys, intermediate states
 //    - Check each pipeline stage output against reference model
 //    - Toggle coverage on internal signals (catch stuck bits)
 //    - Formal verification of key schedule correctness
 //    - Scan chain / ATPG for stuck-at fault coverage
 //    - Power analysis: check crypto does not leak through power side-channel
```

解释

加解密模块的验证计划是一道综合考题，考察从模块级到系统级的完整验证思路。

黑盒验证的核心是 Known-Answer Tests (KAT)：用 NIST 等标准机构发布的

测试向量验证实现正确性，这些向量有公认的 plaintext/key/ciphertext 三元组。

白盒验证可以深入内部状态，配合 formal verification 和 DFT 手段。

面试时能系统说出分层测试思路（模块->集成->系统）、覆盖率目标、

以及黑白盒的不同策略是加分项。

## Verification — Test Plan

### Q94. Algorithm: 100 Switches Toggle Problem -- How Many Are ON

QUESTION

There are 100 switches, all initially OFF.

In visit i (i = 1 to 100), you toggle every switch that is a

multiple of i:

Visit 1: toggle switches 1, 2, 3, 4, ... 100

Visit 2: toggle switches 2, 4, 6, 8, ...

Visit 3: toggle switches 3, 6, 9, 12, ...

...

Visit 100: toggle switch 100 only.

After all 100 visits, how many switches are ON?

Explain why, and which switches are ON.

ANSWER

```
// Answer: 10 switches are ON.
//
// Key insight: a switch N ends up ON if and only if it is toggled
// an ODD number of times. Switch N is toggled once for each of its
// divisors (because visit i toggles switch N iff i divides N).
//
// When does N have an odd number of divisors?
//   Divisors normally come in pairs: if d divides N, then N/d also divides N.
//   Example: divisors of 12 = {1,12}, {2,6}, {3,4} -> 6 divisors (even).
//   Exception: when d == N/d, i.e. d = sqrt(N), i.e. N is a PERFECT SQUARE.
//   For perfect squares, sqrt(N) is its own pair -> odd number of divisors.
//
// Therefore: switch N is ON after all visits iff N is a perfect square.
//
// Perfect squares from 1 to 100:
//   1, 4, 9, 16, 25, 36, 49, 64, 81, 100
//   (1^2, 2^2, 3^2, 4^2, 5^2, 6^2, 7^2, 8^2, 9^2, 10^2)
//
// Count: exactly 10 switches are ON.
//
// Verification for switch 9 (perfect square):
//   Divisors of 9: 1, 3, 9  (odd count = 3)
//   Toggles: visit 1 -> ON, visit 3 -> OFF, visit 9 -> ON
//   Final state: ON  (correct)
//
// Verification for switch 12 (not a perfect square):
//   Divisors of 12: 1, 2, 3, 4, 6, 12  (even count = 6)
//   Toggles: ON, OFF, ON, OFF, ON, OFF
//   Final state: OFF  (correct)
//
// Code to verify (Python):
// switches = [0] * 101          # index 0 unused
// for visit in range(1, 101):
//     for switch in range(visit, 101, visit):
//         switches[switch] ^= 1  # toggle
// on_count = sum(switches)
// on_list  = [i for i in range(1, 101) if switches[i]]
// print(on_count)   # 10
// print(on_list)    # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

解释

这道题是经典算法/数学面试题，核心洞察是把"被拨动奇数次"转化为"因数个数为奇数"。

因数总是成对出现（d 和 N/d），唯一的例外是完全平方数——

此时 sqrt(N) 和自身配对，因数个数为奇数。

所以最终 ON 的开关 = 1到100内的完全平方数 = 10个。

面试中能快速说出"完全平方数"就是关键，不需要暴力枚举。

## Q95. Verification: SoC-Level Verification with CPU, DMA, and Peripherals

QUESTION

You are given a SoC containing: a CPU core, a DMA engine,
on-chip SRAM, peripheral controllers (UART, SPI, I2C), and
an AXI/AHB interconnect.

(a) Describe your overall verification strategy for this SoC.
What levels of verification would you use and why?

(b) Focus on DMA verification specifically:
- What are the key functional scenarios to test?
- What corner cases are most likely to reveal bugs?
- How would you structure the UVM environment for DMA?
- How do you verify DMA correctness (data integrity)?

(c) How do you verify the interaction between DMA and CPU
(e.g. both accessing the same memory region simultaneously)?

(d) What coverage metrics would you track for DMA verification?

## Q96. Verification: Test Plan for Address-Based Routing DUT (A/B Client)

QUESTION

A DUT receives a 32-bit address and routes it to Client A or Client B
based on address ranges programmed into configuration registers (regs).
(Assume A and B ranges do not overlap.)

(a) Write a test plan covering functional scenarios and corner cases.

(b) How would you use assertions to protect register programming?

(c) Design the UVM environment: how many agents, how does the
scoreboard work, and what data structures does it use?

ANSWER

```
// (a) Test Plan
//
// --- Functional Tests ---
//
// 1. Register Initialization and Reset
//    - Verify regs come up with correct default values after reset.
//    - Assert reset and verify all internal state clears.
//    - Write known values to A/B range regs and read back (RAL model).
//
// 2. Basic Routing (directed tests)
//    - Program A_base, A_top, B_base, B_top via config regs.
//    - Send address in A range -> verify arrives at Client A, not B.
//    - Send address in B range -> verify arrives at Client B, not A.
//    - Send random addresses -> verify each goes to correct client.
//
// 3. Randomized Testing
//    - Randomize A and B range registers (non-overlapping constraint).
//    - Send random stream of addresses, scoreboard checks routing.
//    - Run with many seeds to maximize coverage.
```

```
//
// --- Corner Cases ---
//
// 4. Boundary Addresses
//    - Address == A_base     (first address of A range)
//    - Address == A_top      (last  address of A range)
//    - Address == A_base - 1 (just below A range -- should NOT go to A)
//    - Address == A_top  + 1 (just above A range -- should NOT go to A)
//    - Same for B range boundaries.
//
// 5. Full Range / Zero Range
//    - A covers full 32-bit space (A_base=0x0, A_top=0xFFFF_FFFF), B range = 0.
//    - B covers full 32-bit space, A range = 0.
//    - Verify DUT handles degenerate ranges correctly.
//
// 6. Register Programming During Active Traffic
//    - Start sending transactions to A range.
//    - Reprogram A/B range regs mid-flight.
//    - Use assertions: while reg is being programmed (PSEL high),
//      PREADY must be LOW (DUT must stall incoming traffic).
//    - Verify no transaction is misrouted or lost during reprogramming.
//
// 7. Back-to-back Transactions
//    - Consecutive transactions: A, B, A, B alternating.
//    - All-A burst, all-B burst, then switch.
//    - Verify no routing errors across bursts.
//
// 8. Coverage Goals
//    - Each address range (A, B) exercised.
//    - All boundary addresses hit.
//    - Register reprogramming mid-traffic.
//    - Functional coverage: cross(address_range, transaction_type).

// (b) Assertion: PREADY must be LOW during register programming
//
// While the config reg is being written (psel=1, penable=1, pwrite=1),
// the DUT must stall incoming address traffic (pready=0).
property p_stall_during_reg_write;
  @(posedge clk) disable iff (!rst_n)
  // While config reg is being programmed (APB write in progress)
  (cfg_psel && cfg_penable && cfg_pwrite) |->
    !dut_pready;  // incoming traffic must be stalled (pready low)
endproperty
assert property (p_stall_during_reg_write)
  else `uvm_error("ROUTE", "DUT accepted traffic during reg programming!")

// (c) UVM Environment Design
//
// Three agents:
//   1. Input Agent:    drives address transactions into the DUT.
//                      Passive monitor captures all sent transactions.
//   2. Client A Agent: passive monitor only -- observes transactions
//                      arriving at Client A port.
//   3. Client B Agent: passive monitor only -- observes transactions
//                      arriving at Client B port.
//   (4. Config Reg Agent: APB master to program A/B range registers.)
//
// Scoreboard design:
//
//   Input side:
//     - Use uvm_tlm_analysis_fifo for Input Agent (blocking get).
//     - Transactions stored in a QUEUE (not associative array) because:
//       * Addresses are NOT unique -- same address can be sent multiple times.
```

```
//       * Ordering matters: transactions are processed in order.
//     - Input monitor pushes each txn into input_queue.
//
//   Output side:
//     - Client A and Client B monitors write to scoreboard via analysis ports.
//     - When a transaction arrives at Client A or B:
//       1. Compute expected destination using the same routing logic
//          (cmodel: if addr in A_range -> expect A, else expect B).
//       2. Pop from input_queue (FIFO order -- first sent, first checked).
//       3. Verify: popped address matches received address.
//       4. Verify: received at correct client (A or B).
//     - No need to store Client A/B transactions: compare immediately and discard.
//
//   Data structure choice rationale:
//     - Input queue: queue not associative array because address is NOT unique.
//       If address were a unique ID, use associative array keyed by ID.
//     - Client A/B: no storage needed -- match-on-arrival, then discard.
//
// class routing_scoreboard extends uvm_scoreboard;
//   uvm_tlm_analysis_fifo #(addr_txn) input_fifo;
//   uvm_analysis_imp_a    #(addr_txn, routing_scoreboard) a_imp;
//   uvm_analysis_imp_b    #(addr_txn, routing_scoreboard) b_imp;
//
//   addr_txn input_queue[$];  // ordered queue for input transactions
//
//   // Input monitor callback: store in queue
//   function void write_input(addr_txn txn);
//     input_queue.push_back(txn);
//   endfunction
//
//   // Client A monitor callback: verify and pop
//   function void write_a(addr_txn txn);
//     addr_txn expected;
//     if (input_queue.size() == 0)
//       `uvm_error("SCBD", "Client A received unexpected transaction")
//     else begin
//       expected = input_queue.pop_front();
//       if (expected.addr != txn.addr)
//         `uvm_error("SCBD", "Data mismatch at Client A")
//       if (!in_a_range(txn.addr))
//         `uvm_error("SCBD", "Routing error: addr should not go to A")
//     end
//   endfunction
//
//   // Client B: same pattern as write_a
// endclass
```

解释

这道题考察从测试计划到 UVM 架构的完整设计能力。

面试官的关键 feedback：

不要一开始就深入 vif/driver/sequencer 等细节组件，

先把大框架说清楚：几个 agent，scoreboard 怎么设计，用什么数据结构。

被追问了再深入细节。思路清晰比细节完整更重要。

Scoreboard 数据结构选择口诀：

有 unique ID -> 用联合数组（key=ID，快速查找）。

无 unique ID，保持顺序 -> 用 queue（先入先出）。

本题地址不唯一，按序比较 -> 用 queue。

关键测试点不要漏：

边界地址（base, top, base-1, top+1）、全范围/零范围、

寄存器编程过程中的流量处理（用断言保护）。

断言的作用：在 reg 编程期间用 pready=0 来保护，防止 DUT

在配置变化瞬间产生错误路由。

## Behavior Questions

### Q97. Behavior Question: Describe Your Most Proud Project

QUESTION

Tell me about the project you are most proud of.

Walk me through: what the project was, what your specific contribution was, what challenges you faced, how you solved them, and what the outcome or impact was.

### Q98. Behavior Question: Describe an Interesting Bug You Encountered

QUESTION

Tell me about an interesting or challenging bug you encountered during your work or projects.

Walk me through: how you discovered it, what made it difficult to find, what steps you took to debug it, and what you learned from the experience.